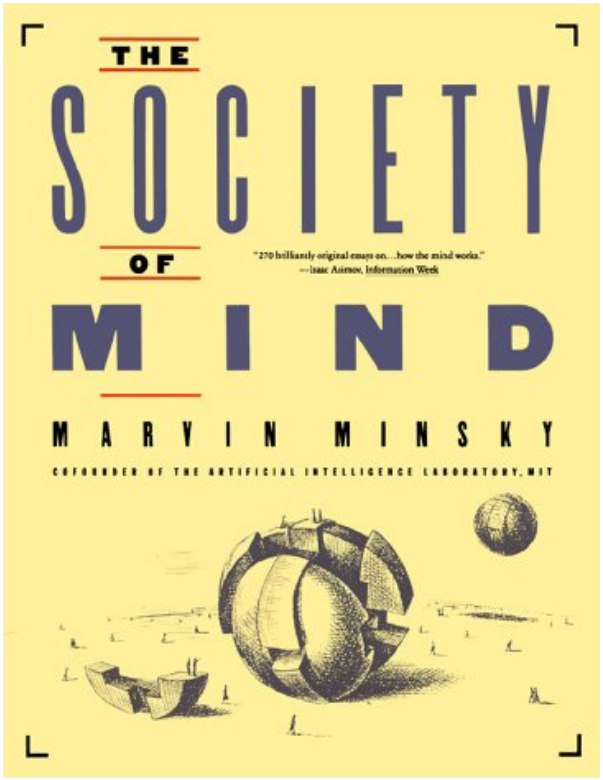
Intelligent Systems

January 2017

The society of mind

Marvin Minsky



2.5 EASY THINGS ARE HARD

In the late 1960s *Builder* was embodied in the form of a computer program at the MIT Artificial Intelligence Laboratory. Both my collaborator, Seymour Papert, and I had long desired to combine a mechanical hand, a television eye, and a computer into a robot that could build with children's building-blocks. It took several years for us and our students to develop *Move*, *See*, *Grasp*, and hundreds of other little programs we needed to make a working *Builder*-agency. I like to think that this project gave us glimpses of what happens inside certain parts of children's minds when they learn to "play" with simple toys. The project left us wondering if even a thousand microskills would be enough to enable a child to fill a pail with sand. It was this body of experience, more than anything we'd learned about psychology, that led us to many ideas about societies of mind.

To do those first experiments, we had to build a mechanical Hand, equipped with sensors for pressure and touch at its fingertips. Then we had to interface a television camera with our computer and write programs with which that Eye could discern the edges of the building-blocks. It also had to recognize the Hand itself. When those programs didn't work so well, we added more programs that used the fingers' feeling-sense to verify that things were where they visually seemed to be. Yet other programs were needed to enable the computer to move the Hand from place to place while using the Eye to see that there was nothing in its way. We also had to write higher-level programs that the robot could use for planning what to do—and still more programs to make sure that those plans were actually carried out. To make this all work reliably, we needed programs to verify at every step (again by using Eye and Hand) that what had been planned inside the mind did actually take place outside—or else to correct the mistakes that occurred.

In attempting to make our robot work, we found that many everyday problems were much more complicated than the sorts of problems, puzzles, and games adults consider hard. At every point, in that world of blocks, when we were forced to look more carefully than usual, we found an unexpected universe of complications. Consider just the seemingly simple problem of not reusing blocks already built into the tower. To a person, this seems simple common sense: *"Don't use an object to satisfy a new goal if that object is already involved in accomplishing a prior goal."* No one knows exactly how human minds do this. Clearly we learn from experience to recognize the situations in which difficulties are likely to occur, and when we're older we learn to plan ahead to avoid such conflicts. But since we cannot be sure what will work, we must learn policies for dealing with uncertainty. Which strategies are best to try, and which will avoid the worst mistakes? Thousands and, perhaps, millions of little processes must be involved in how we anticipate, imagine, plan, predict, and prevent—and yet all this proceeds so automatically that we regard it as "ordinary common sense." But if thinking is so complicated, what makes it seem so simple? At first it may seem incredible that our minds could use such intricate machinery and yet be unaware of it.

In general, we're least aware of what our minds do best.

It's mainly when our other systems start to fail that we engage the special agencies involved with what we call "consciousness." Accordingly, we're more aware of simple processes that don't work well than of complex ones that work flawlessly. This means that we cannot trust our offhand judgments about which of the things we do are simple, and which require complicated machinery. Most times, each portion of the mind can only sense how quietly the other portions do their jobs.

2.6 ARE PEOPLE MACHINES?

Many people feel offended when their minds are likened to computer programs or machines. We've seen how a simple tower-building skill can be composed of smaller parts. But could anything like a real mind be made of stuff so trivial?

"*Ridiculous,*" most people say. "*I certainly don't feel like a machine!*"

But if you're not a machine, what makes you an authority on what it feels like to be a machine? A person might reply, "*I think, therefore I know how the mind works.*" But that would be suspiciously like saying, "*I drive my car, therefore I know how its engine works.*" Knowing how to use something is not the same as knowing how it works.

"*But everyone knows that machines can behave only in lifeless, mechanical ways.*"

This objection seems more reasonable: indeed, a person *ought* to feel offended at being likened to any *trivial* machine. But it seems to me that the word "machine" is getting to be out of date. For centuries, words like "mechanical" made us think of simple devices like pulleys, levers, locomotives, and typewriters. (The word "computerlike" inherited a similar sense of pettiness, of doing dull arithmetic by little steps.) But we ought to recognize that we're still in an early era of machines, with virtually no idea of what they may become. What if some visitor from Mars had come a billion years ago to judge the fate of earthly life from watching clumps of cells that hadn't even learned to crawl? In the same way, we cannot grasp the range of what machines may do in the future from seeing what's on view right now.

Our first intuitions about computers came from experiences with machines of the 1940s, which contained only thousands of parts. But a human brain contains billions of cells, each one complicated by itself and connected to many thousands of others. Present-day computers represent an intermediate degree of complexity; they now have millions of parts, and people already are building billion-part computers for research on Artificial Intelligence. And yet, in spite of what is happening, we continue to use old words as though there had been no change at all. We need to adapt our attitudes to phenomena that work on scales never before conceived. The term "machine" no longer takes us far enough.

But rhetoric won't settle anything. Let's put these arguments aside and try instead to understand what the vast, unknown mechanisms of the brain may do. Then we'll find more self-respect in knowing what wonderful machines we are.

4.4 THE CONSERVATIVE SELF

How do we control our minds? Ideally, we first choose what we want to do, then make ourselves do it. But that's harder than it sounds: we spend our lives in search of schemes for self-control. We celebrate when we succeed, and when we fail, we're angry with ourselves for not behaving as we wanted to—and then we try to scold or shame or bribe ourselves to change our ways. But wait! How could a self be angry with itself? Who would be mad at whom? Consider an example from everyday life.

> *I was trying to concentrate on a certain problem but was getting bored and sleepy. Then I imagined that one of my competitors, Professor Challenger, was about to solve the same problem. An angry wish to frustrate Challenger then kept me working on the problem for a while. The strange thing was, this problem was not of the sort that ever interested Challenger.*

What makes us use such roundabout techniques to influence ourselves? Why be so indirect, inventing misrepresentations, fantasies, and outright lies? Why can't we simply tell ourselves to do the things we want to do?

To understand how something works, one has to know its purposes. Once, no one understood the heart. But as soon as it was seen that hearts move blood, a lot of other things made sense: those things that looked like pipes and valves were really pipes and valves indeed—and anxious, pounding, pulsing hearts were recognized as simple pumps. New speculations could then be formed: was this to give our tissues drink or food? Was it to keep our bodies warm or cool? For sending messages from place to place? In fact, all those hypotheses were correct, and when that surge of functional ideas led to the guess that blood can carry air as well, more puzzle parts fell into place.

To understand what we call the Self, we first must see what Selves are for. *One function of the Self is to keep us from changing too rapidly.* Each person must make some long-range plans in order to balance single-purposeness against attempts to do everything at once. But it is not enough simply to instruct an agency to start to carry out our plans. We also have to find some ways to constrain the changes we might later make—to prevent ourselves from turning those plan-agents off again! If we changed our minds too recklessly, we could never know what we might want next. We'd never get much done because we could never depend on ourselves.

Those ordinary views are wrong that hold that Selves are magic, self-indulgent luxuries that enable our minds to break the bonds of natural cause and law. Instead, those Selves are practical necessities. The myths that say that Selves embody special kinds of liberty are merely masquerades. Part of their function is to hide from us the nature of our self-ideals—the chains we forge to keep ourselves from wrecking all the plans we make.

7.1 INTELLIGENCE

Many people insist on having some definition of "intelligence."

> *CRITIC: How can we be sure that things like plants and stones, or storms and streams, are not intelligent in ways that we have not yet conceived?*

It doesn't seem a good idea to use the same word for different things, unless one has in mind important ways in which they are the same. Plants and streams don't seem very good at solving the kinds of problems we regard as needing intelligence.

> *CRITIC: What's so special about solving problems? And why don't you define "intelligence" precisely, so that we can agree on what we're discussing?*

That isn't a good idea, either. An author's job is using words the ways other people do, not telling others how to use them. In the few places the word "intelligence" appears in this book, it merely means what people usually mean—the ability to solve hard problems.

> *CRITIC: Then you should define what you mean by a "hard" problem. We know it took a lot of human intelligence to build the pyramids—yet little coral reef animals build impressive structures on even larger scales. So don't you have to consider them intelligent? Isn't it hard to build gigantic coral reefs?*

Yes, but it is only an illusion that animals can "solve" those problems! No individual bird *discovers* a way to fly. Instead, each bird exploits a solution that evolved from countless reptile years of evolution. Similarly, although a person might find it very hard to design an oriole's nest or a beaver's dam, no oriole or beaver ever figures out such things at all. Those animals don't "solve" such problems themselves; they only exploit procedures available within their complicated gene-built brains.

> *CRITIC: Then wouldn't you be forced to say that evolution itself must be intelligent, since it solved those problems of flying and building reefs and nests?*

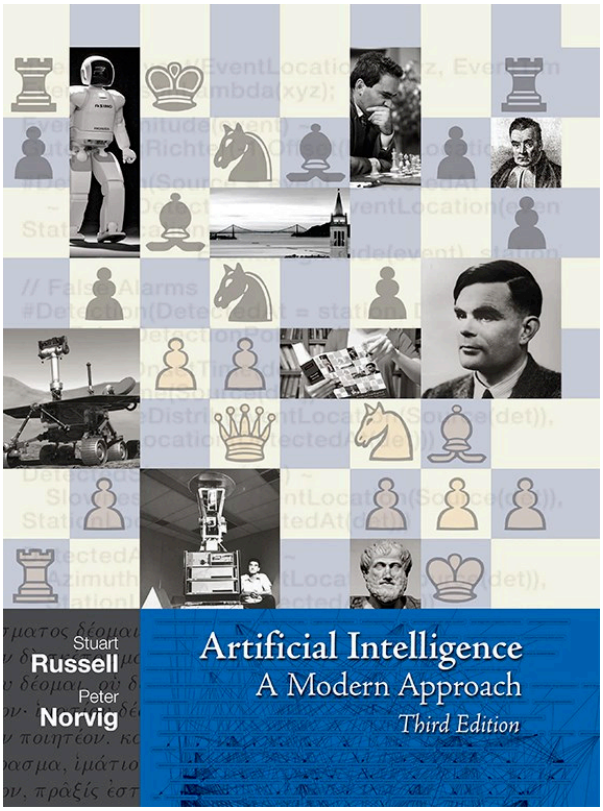
No, because people also use the word "intelligence" to emphasize swiftness and efficiency. Evolution's time rate is so slow that we don't see it as intelligent, even though it finally produces wonderful things we ourselves cannot yet make. Anyway, it isn't wise to treat an old, vague word like "intelligence" as though it must define any definite thing. Instead of trying to say what such a word "means," it is better simply to try to explain how we use it.

> *Our minds contain processes that enable us to solve problems we consider difficult. "Intelligence" is our name for whichever of those processes we don't yet understand.*

Some people dislike this "definition" because its meaning is doomed to keep changing as we learn more about psychology. But in my view that's exactly how it ought to be, because the very concept of intelligence is like a stage magician's trick. Like the concept of "*the unexplored regions of Africa*," it disappears as soon as we discover it.

Artificial intelligence: a modern approach

Stuart J. Russell, Peter Norvig



3.5 SEARCH STRATEGIES

The majority of work in the area of search has gone into finding the right **search strategy** for a problem. In our study of the field we will evaluate strategies in terms of four criteria:

COMPLETENESS
- ◇ **Completeness:** is the strategy guaranteed to find a solution when there is one?

TIME COMPLEXITY
- ◇ **Time complexity:** how long does it take to find a solution?

SPACE COMPLEXITY
- ◇ **Space complexity:** how much memory does it need to perform the search?

OPTIMALITY
- ◇ **Optimality**: does the strategy find the highest-quality solution when there are several different solutions?[7]

UNINFORMED SEARCH

BLIND SEARCH

This section covers six search strategies that come under the heading of **uninformed search.** The term means that they have no information about the number of steps or the path cost from the current state to the goal—all they can do is distinguish a goal state from a nongoal state. Uninformed search is also sometimes called **blind search.**

Consider again the route-finding problem. From the initial state in Arad, there are three actions leading to three new states: Sibiu, Timisoara, and Zerind. An uninformed search has no preference among these, but a more clever agent might notice that the goal, Bucharest, is southeast of Arad, and that only Sibiu is in that direction, so it is likely to be the best choice.

INFORMED SEARCH
HEURISTIC SEARCH

Strategies that use such considerations are called **informed search** strategies or **heuristic search** strategies, and they will be covered in Chapter 4. Not surprisingly, uninformed search is less effective than informed search. Uninformed search is still important, however, because there are many problems for which there is no additional information to consider.

The six uninformed search strategies are distinguished by the *order* in which nodes are expanded. It turns out that this difference can matter a great deal, as we shall shortly see.

---

[7] This is the way "optimality" is used in the theoretical computer science literature. Some AI authors use "optimality" to refer to time of execution and "admissibility" to refer to solution optimality.

Breadth-first search

BREADTH-FIRST SEARCH

One simple search strategy is a **breadth-first search.** In this strategy, the root node is expanded first, then all the nodes generated by the root node are expanded next, and then *their* successors, and so on. In general, all the nodes at depth d in the search tree are expanded before the nodes at depth $d + 1$. Breadth-first search can be implemented by calling the GENERAL-SEARCH algorithm with a queuing function that puts the newly generated states at the end of the queue, after all the previously generated states:

```
function BREADTH-FIRST-SEARCH(problem) returns a solution or failure
  return GENERAL-SEARCH(problem,ENQUEUE-AT-END)
```

Breadth-first search is a very systematic strategy because it considers all the paths of length 1 first, then all those of length 2, and so on. Figure 3.11 shows the progress of the search on a simple binary tree. If there is a solution, breadth-first search is guaranteed to find it, and if there are several solutions, breadth-first search will always find the shallowest goal state first. In terms of the four criteria, breadth-first search is complete, and it is optimal *provided the path cost is a nondecreasingfunction of the depth of the node.* (This condition is usually satisfied only when all operators have the same cost. For the general case, see the next section.)

So far, the news about breadth-first search has been good. To see why it is not always the strategy of choice, we have to consider the amount of time and memory it takes to complete a search. To do this, we consider a hypothetical state space where every state can be expanded to yield b new states. We say that the **branching factor** of these states (and of the search tree) is b. The root of the search tree generates b nodes at the first level, each of which generates b more nodes, for a total of b^2 at the second level. Each of *these* generates b more nodes, yielding b^3 nodes at the third level, and so on. Now suppose that the solution for this problem has a path length of d. Then the maximum number of nodes expanded before finding a solution is

BRANCHING FACTOR

$$1 + b + b^2 + b^3 + \bullet\bullet\bullet + b^d$$

This is the maximum number, but the solution could be found at any point on the dth level. In the best case, therefore, the number would be smaller.

Those who do complexity analysis get nervous (or excited, if they are the sort of people who like a challenge) whenever they see an exponential complexity bound like $O(b^d)$. Figure 3.12 shows why. It shows the time and memory required for a breadth-first search with branching factor $b = 10$ and for various values of the solution depth d. The space complexity is the same as the time complexity, because all the leaf nodes of the tree must be maintained in memory

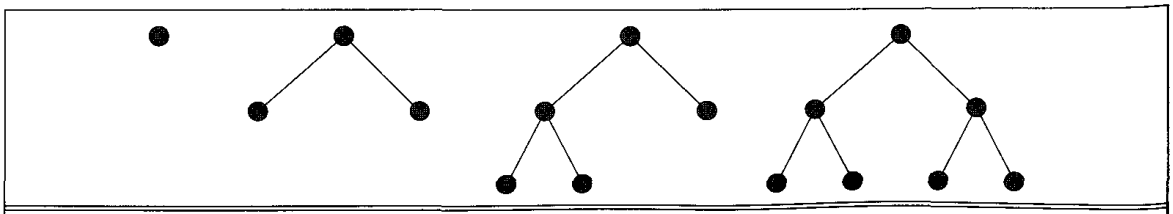


Figure 3.11 Breadth-first search trees after 0, 1, 2, and 3 node expansions.

at the same time. Figure 3.12 assumes that 1000 nodes can be goal-checked and expanded per second, and that a node requires 100 bytes of storage. Many puzzle-like problems fit roughly within these assumptions (give or take a factor of 100) when run on a modern personal computer or workstation.

Depth	Nodes	Time	Memory
0	1	1 millisecond	100 bytes
2	111	.1 seconds	11 kilobytes
4	11,111	11 seconds	1 megabyte
6	10^6	18 minutes	111 megabytes
8	10^8	31 hours	11 gigabytes
10	10^{10}	128 days	1 terabyte
12	10^{12}	35 years	111 terabytes
14	10^{14}	3500 years	11,111 terabytes

Figure 3.12 Time and memory requirements for breadth-first search. The figures shown assume branching factor $b = 10$; 1000 nodes/second; 100 bytes/node.

There are two lessons to be learned from Figure 3.12. First, *the memory requirements are a bigger problem for breadth-first search than the execution time*. Most people have the patience to wait 18 minutes for a depth 6 search to complete, assuming they care about the answer, but not so many have the 111 megabytes of memory that are required. And although 31 hours would not be too long to wait for the solution to an important problem of depth 8, very few people indeed have access to the 11 gigabytes of memory it would take. Fortunately, there are other search strategies that require less memory.

The second lesson is that the time requirements are still a major factor. If your problem has a solution at depth 12, then (given our assumptions) it will take 35 years for an uninformed search to find it. Of course, if trends continue then in 10 years, you will be able to buy a computer that is 100 times faster for the same price as your current one. Even with that computer, however, it will still take 128 days to find a solution at depth 12—and 35 years for a solution at depth 14. Moreover, there are no other uninformed search strategies that fare any better. *In general, exponential complexity search problems cannot be solved for any but the smallest instances.*

Uniform cost search

UNIFORM COST SEARCH

Breadth-first search finds the *shallowest* goal state, but this may not always be the least-cost solution for a general path cost function. **Uniform cost search** modifies the breadth-first strategy by always expanding the lowest-cost node on the fringe (as measured by the path cost $g(n)$), rather than the lowest-depth node. It is easy to see that breadth-first search is just uniform cost search with $g(n) = \text{DEPTH}(n)$.

When certain conditions are met, the first solution that is found is guaranteed to be the cheapest solution, because if there were a cheaper path that was a solution, it would have been expanded earlier, and thus would have been found first. A look at the strategy in action will help explain. Consider the route-finding problem in Figure 3.13. The problem is to get from *S* to G,

and the cost of each operator is marked. The strategy first expands the initial state, yielding paths to A, *B,* and *C.* Because the path to A is cheapest, it is expanded next, generating the path *SAG,* which is in fact a solution, though not the optimal one. However, the algorithm does not yet recognize this as a solution, because it has cost 11, and thus is buried in the queue below the path *SB,* which has cost 5. It seems a shame to generate a solution just to bury it deep in the queue, but it is necessary if we want to find the optimal solution rather than just any solution. The next step is to expand *SB,* generating *SBG,* which is now the cheapest path remaining in the queue, so it is goal-checked and returned as the solution.

Uniform cost search finds the cheapest solution provided a simple requirement is met: the cost of a path must never decrease as we go along the path. In other words, we insist that

$$g(\text{SUCCESSOR}(n)) > g(n)$$

for every node n.

The restriction to nondecreasing path cost makes sense if the path cost of a node is taken to be the sum of the costs of the operators that make up the path. If every operator has a nonnegative cost, then the cost of a path can never decrease as we go along the path, and uniform-cost search can find the cheapest path without exploring the whole search tree. But if some operator had a negative cost, then nothing but an exhaustive search of all nodes would find the optimal solution, because we would never know when a path, no matter how long and expensive, is about to run into a step with high negative cost and thus become the best path overall. (See Exercise 3.5.)

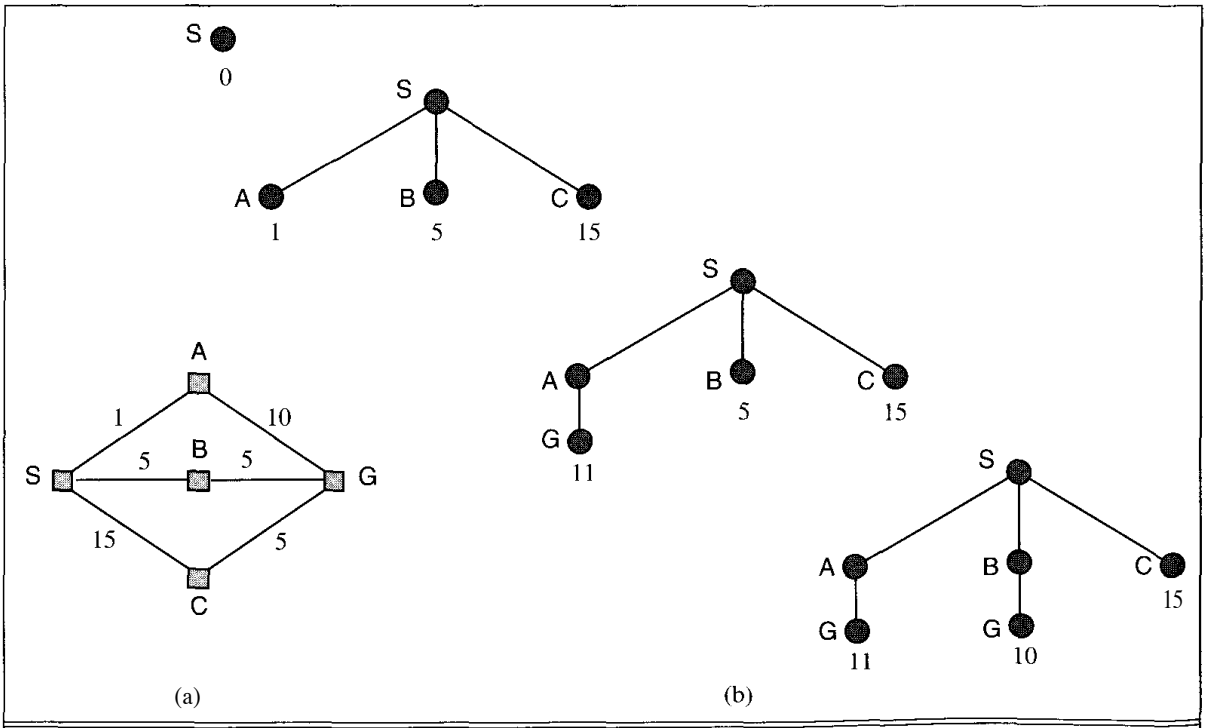


Figure 3.13 A route-finding problem. (a) The state space, showing the cost for each operator. (b) Progression of the search. Each node is labelled with $g(n)$. At the next step, the goal node with $g = 10$ will be selected.

Depth-first search

DEPTH-FIRST SEARCH

Depth-first search always expands one of the nodes at the deepest level of the tree. Only when the search hits a dead end (a nongoal node with no expansion) does the search go back and expand nodes at shallower levels. This strategy can be implemented by GENERAL-SEARCH with a queuing function that always puts the newly generated states at the front of the queue. Because the expanded node was the deepest, its successors will be even deeper and are therefore now the deepest. The progress of the search is illustrated in Figure 3.14.

Depth-first search has very modest memory requirements. As the figure shows, it needs to store only a single path from the root to a leaf node, along with the remaining unexpanded sibling nodes for each node on the path. For a state space with branching factor b and maximum depth m, depth-first search requires storage of only bm nodes, in contrast to the b^d that would be required by breadth-first search in the case where the shallowest goal is at depth d. Using the same assumptions as Figure 3.12, depth-first search would require 12 kilobytes instead of 111 terabytes at depth $d = 12$, a factor of 10 billion times less space.

The time complexity for depth-first search is $O(b^m)$. For problems that have very many solutions, depth-first may actually be faster than breadth-first, because it has a good chance of

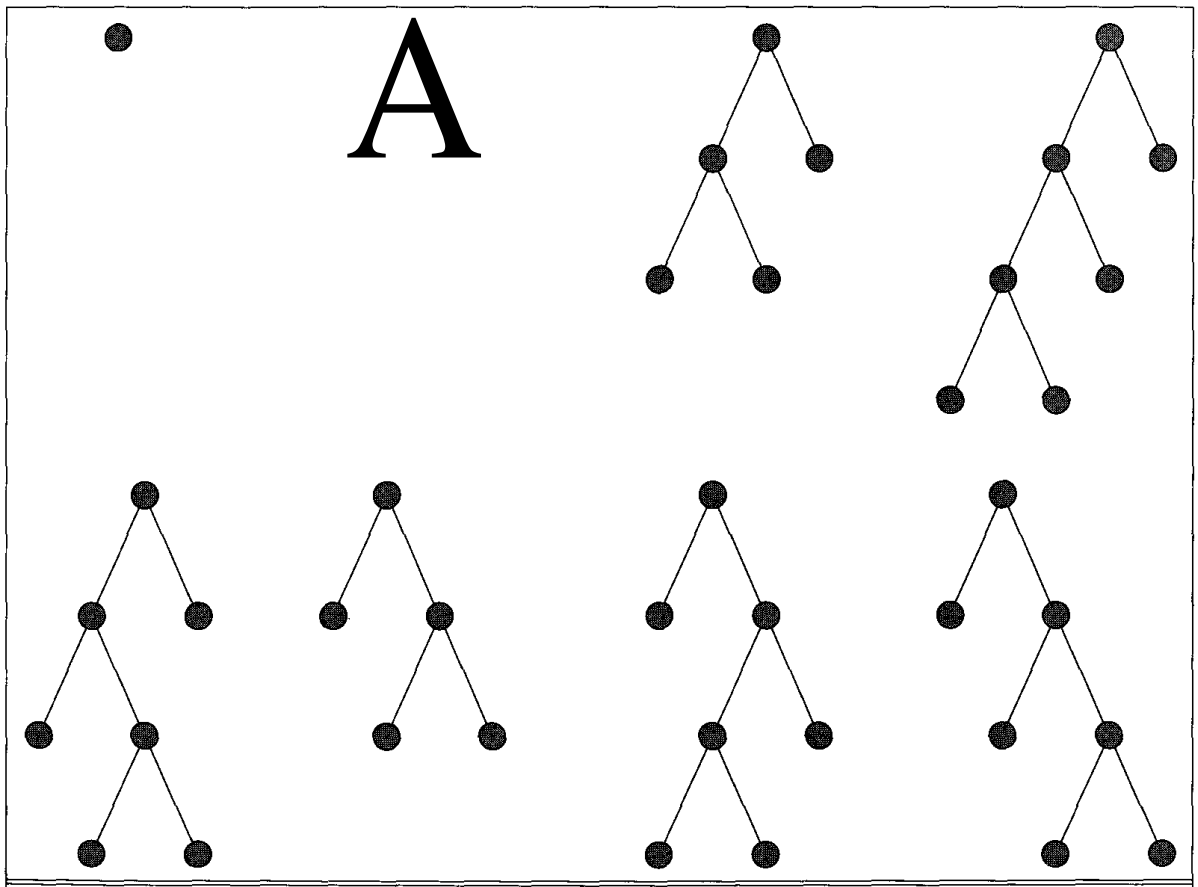


Figure 3.14 Depth-first search trees for a binary search tree. Nodes at depth 3 are assumed to have no successors.

finding a solution after exploring only a small portion of the whole space. Breadth-first search would still have to look at all the paths of length d - 1 before considering any of length d. Depth-first search is still $O(b^m)$ in the worst case.

The drawback of depth-first search is that it can get stuck going down the wrong path. Many problems have very deep or even infinite search trees, so depth-first search will never be able to recover from an unlucky choice at one of the nodes near the top of the tree. The search will always continue downward without backing up, even when a shallow solution exists. Thus, on these problems depth-first search will either get stuck in an infinite loop and never return a solution, or it may eventually find a solution path that is longer than the optimal solution. That means depth-first search is neither complete nor optimal. Because of this, *depth-first search should be avoidedfor search trees with large or infinite maximum depths.*

It is trivial to implement depth-first search with GENERAL-SEARCH:

```
function DEPTH-FIRST-SEARCH(problem) returns a solution, or failure
  GENERAL-SEARCH(problem,ENQUEUE-AT-FRONT)
```

It is also common to implement depth-first search with a recursive function that calls itself on each of its children in turn. In this case, the queue is stored implicitly in the local state of each invocation on the calling stack.

Depth-limited search

DEPTH-LIMITED SEARCH

Depth-limited search avoids the pitfalls of depth-first search by imposing a cutoff on the maximum depth of a path. This cutoff can be implemented with a special depth-limited search algorithm, or by using the general search algorithm with operators that keep track of the depth. For example, on the map of Romania, there are 20 cities, so we know that if there is a solution, then it must be of length 19 at the longest. We can implement the depth cutoff using operators of the form "If you are in city A and have travelled a path of less than 19 steps, then generate a new state in city B with a path length that is one greater." With this new operator set, we are guaranteed to find the solution if it exists, but we are still not guaranteed to find the shortest solution first: depth-limited search is complete but not optimal. If we choose a depth limit that is too small, then depth-limited search is not even complete. The time and space complexity of depth-limited search is similar to depth-first search. It takes $O(b^l)$ time and $O(bl)$ space, where l is the depth limit.

Iterative deepening search

The hard part about depth-limited search is picking a good limit. We picked 19 as an "obvious" depth limit for the Romania problem, but in fact if we studied the map carefully, we would discover that any city can be reached from any other city in at most 9 steps. This number, known as the **diameter** of the state space, gives us a better depth limit, which leads to a more efficient depth-limited search. However, for most problems, we will not know a good depth limit until we have solved the problem.

DIAMETER

ITERATIVE DEEPENING SEARCH

Iterative deepening search is a strategy that sidesteps the issue of choosing the best depth limit by trying all possible depth limits: first depth 0, then depth 1, then depth 2, and so on. The algorithm is shown in Figure 3.15. In effect, iterative deepening combines the benefits of depth-first and breadth-first search. It is optimal and complete, like breadth-first search, but has only the modest memory requirements of depth-first search. The order of expansion of states is similar to breadth-first, except that some states are expanded multiple times. Figure 3.16 shows the first four iterations of ITERATIVE-DEEPENING-SEARCH on a binary search tree.

Iterative deepening search may seem wasteful, because so many states are expanded multiple times. For most problems, however, the overhead of this multiple expansion is actually

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution sequence
  inputs: problem, a problem

  for depth ← 0 to ∞ do
    if DEPTH-LIMITED-SEARCH(problem, depth) succeeds then return its result
  end
  return failure
```

Figure 3.15 The iterative deepening search algorithm.

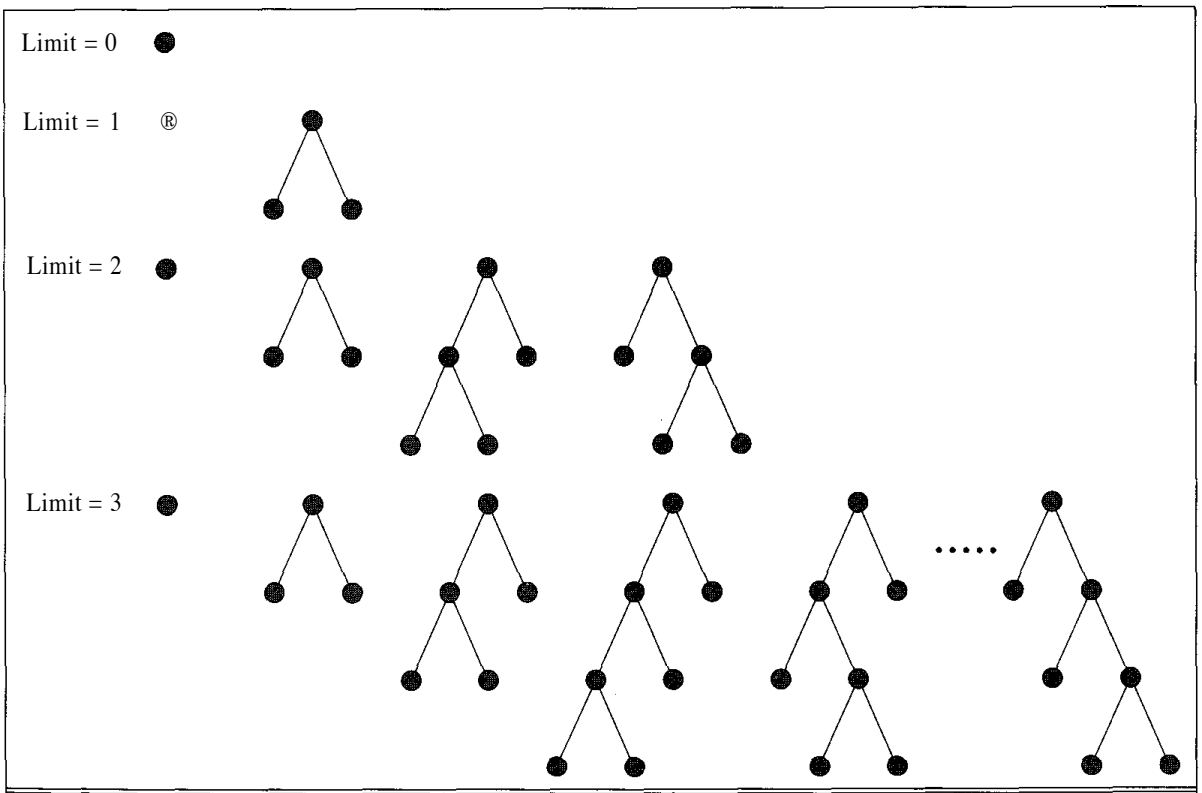


Figure 3.16 Four iterations of iterative deepening search on a binary tree.

rather small. Intuitively, the reason is that in an exponential search tree, almost all of the nodes are in the bottom level, so it does not matter much that the upper levels are expanded multiple times. Recall that the number of expansions in a depth-limited search to depth d with branching factor b is

$$1 + b + b^2 + \cdots + b^{d-2} + b^{d-1} + b^d$$

To make this concrete, for $b = 10$ and $d = 5$. the number is

$$1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$$

In an iterative deepening search, the nodes on the bottom level are expanded once, those on the next to bottom level are expanded twice, and so on, up to the root of the search tree, which is expanded $d + 1$ times. So the total number of expansions in an iterative deepening search is

$$(d + 1)1 + (d)b + (d-1)b^2 + \cdots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

Again, for $b = 10$ and $d = 5$ the number is

$$6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$$

All together, an iterative deepening search from depth 1 all the way down to depth d expands only about 11% more nodes than a single breadth-first or depth-limited search to depth d, when $b = 10$. The higher the branching factor, the lower the overhead of repeatedly expanded states, but even when the branching factor is 2, iterative deepening search only takes about twice as long as a complete breadth-first search. This means that the time complexity of iterative deepening is still $O(b^d)$, and the space complexity is $O(bd)$. *In general, iterative deepening is the preferred search method when there is a large search space and the depth of the solution is not known.*

Bidirectional search

The idea behind bidirectional search is to simultaneously search both forward from the initial state and backward from the goal, and stop when the two searches meet in the middle (Figure 3.17). For problems where the branching factor is b in both directions, bidirectional search can make a big difference. If we assume as usual that there is a solution of depth d, then the solution will be found in $O(2b^{d/2}) = O(b^{d/2})$ steps, because the forward and backward searches each have to go only half way. To make this concrete: for $b = 10$ and $d = 6$, breadth-first search generates 1,111,111 nodes, whereas bidirectional search succeeds when each direction is at depth 3, at which point 2,222 nodes have been generated. This sounds great in theory. Several issues need to be addressed before the algorithm can be implemented.

PREDECESSORS

- The main question is, what does it mean to search backwards from the goal? We define the **predecessors** of a node n to be all those nodes that have n as a successor. Searching backwards means generating predecessors successively starting from the goal node.
- When all operators are reversible, the predecessor and successor sets are identical; for some problems, however, calculating predecessors can be very difficult.
- What can be done if there are many possible goal states? If there is an *explicit* list of goal states, such as the two goal states in Figure 3.2, then we can apply a predecessor function to the state set just as we apply the successor function in multiple-state search. If we only

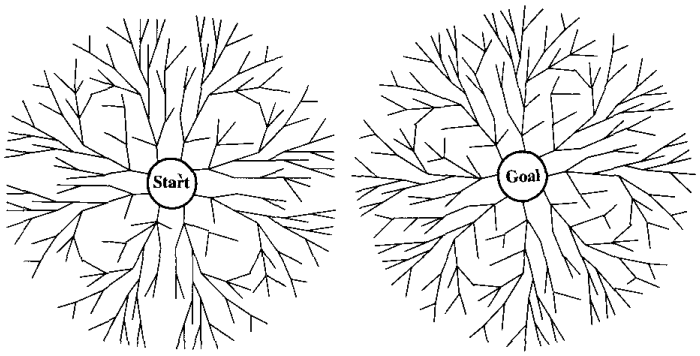


Figure 3.17 A schematic view of a bidirectional breadth-first search that is about to succeed, when a branch from the start node meets a branch from the goal node.

have a *description of* the set, it may be possible to figure out the possible descriptions of "sets of states that would generate the goal set," but this is a very tricky thing to do. For example, what are the states that are the predecessors of the checkmate goal in chess?

- There must be an efficient way to check each new node to see if it already appears in the search tree of the other half of the search.
- We need to decide what kind of search is going to take place in each half. For example, Figure 3.17 shows two breadth-first searches. Is this the best choice?

The $O(b^{d/2})$ complexity figure assumes that the process of testing for intersection of the two frontiers can be done in constant time (that is, is independent of the number of states). This often can be achieved with a hash table. In order for the two searches to meet at all, the nodes of at least one of them must all be retained in memory (as with breadth-first search). This means that the space complexity of uninformed bidirectional search is $O(b^{d/2})$.

Comparing search strategies

Figure 3.18 compares the six search strategies in terms of the four evaluation criteria set forth in Section 3.5.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Time	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	b^d	bm	bl	bd	$b^{d/2}$
Optimal?	Yes	Yes	No	No	Yes	Yes
Complete?	Yes	Yes	No	Yes, if $l > d$	Yes	Yes

Figure 3.18 Evaluation of search strategies. b is the branching factor; d is the depth of solution; m is the maximum depth of the search tree; l is the depth limit.

4 INFORMED SEARCH METHODS

In which we see how information about the state space can prevent algorithms from blundering about in the dark.

Chapter 3 showed that uninformed search strategies can find solutions to problems by systematically generating new states and testing them against the goal. Unfortunately, these strategies are incredibly inefficient in most cases. This chapter shows how an informed search strategy—one that uses problem-specific knowledge—can find solutions more efficiently. It also shows how optimization problems can be solved.

4.1 BEST-FIRST SEARCH

In Chapter 3, we found several ways to apply knowledge to the process of formulating a problem in terms of states and operators. Once we are given a well-defined problem, however, our options are more limited. If we plan to use the GENERAL-SEARCH algorithm from Chapter 3, then the only place where knowledge can be applied is in the queuing function, which determines the node to expand next. Usually, the knowledge to make this determination is provided by an

EVALUATION FUNCTION

evaluation function that returns a number purporting to describe the desirability (or lack thereof) of expanding the node. When the nodes are ordered so that the one with the best evaluation is

BEST-FIRST SEARCH

expanded first, the resulting strategy is called **best-first search.** It can be implemented directly with GENERAL-SEARCH, as shown in Figure 4.1.

The name "best-first search" is a venerable but inaccurate one. After all, if we could really expand the best node first, it would not be a search at all; it would be a straight march to the goal. All we can do is choose the node that *appears* to be best according to the evaluation function. If the evaluation function is omniscient, then this will indeed be the best node; in reality, the evaluation function will sometimes be off, and can lead the search astray. Nevertheless, we will stick with the name "best-first search," because "seemingly-best-first search" is a little awkward.

Just as there is a whole family of GENERAL-SEARCH algorithms with different ordering functions, there is also a whole family of BEST-FIRST-SEARCH algorithms with different evaluation

```
function BEST-FIRST-SEARCH(problem, EVAL-FN) returns a solution sequence
  inputs: problem, a problem
          Eval-Fn, an evaluation function

  Queueing-Fn ← a function that orders nodes by EVAL-FN
  return GENERAL-SEARCH(problem, Queueing-Fn)
```

Figure 4.1 An implementation of best-first search using the general search algorithm.

functions. Because they aim to find low-cost solutions, these algorithms typically use some estimated measure of the cost of the solution and try to minimize it. We have already seen one such measure: the use of the path cost g to decide which path to extend. This measure, however, does not direct search *toward the goal. In order to focus the search, the measure must incorporate some estimate of the cost of the path from a state to the closest goal state.* We look at two basic approaches. The first tries to expand the node closest to the goal. The second tries to expand the node on the least-cost solution path.

Minimize estimated cost to reach a goal: Greedy search

One of the simplest best-first search strategies is to minimize the estimated cost to reach the goal. That is, the node whose state is judged to be closest to the goal state is always expanded first. For most problems, the cost of reaching the goal from a particular state can be estimated but cannot be determined exactly. A function that calculates such cost estimates is called a **heuristic function,** and is usually denoted by the letter h:

HEURISTIC FUNCTION

$h(n) =$ estimated cost of the cheapest path from the state at node n to a goal state.

GREEDY SEARCH

A best-first search that uses h to select the next node to expand is called **greedy search,** for reasons that will become clear. Given a heuristic function h, the code for greedy search is just the following:

```
function GREEDY-SEARCH(problem) returns a solution or failure
  return BEST-FIRST-SEARCH(problem, h)
```

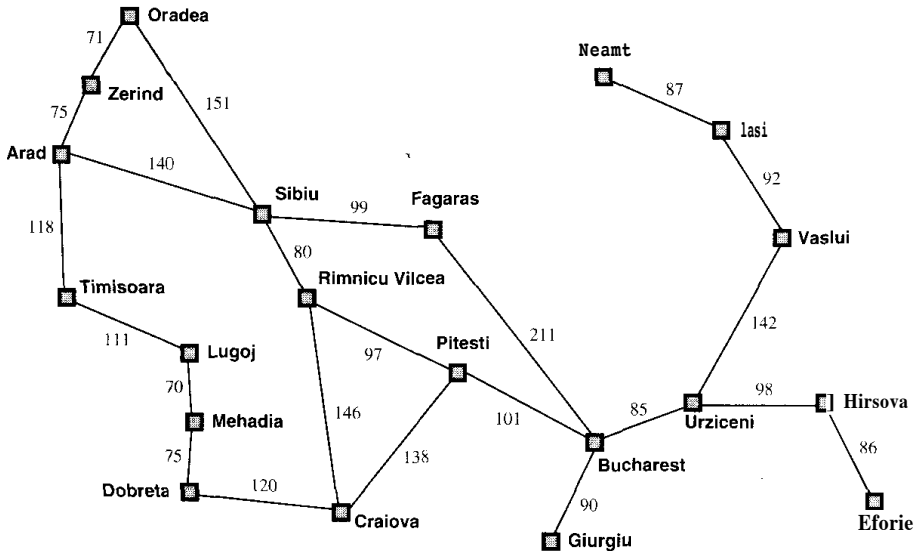
Formally speaking, h can be any function at all. We will require only that $h(n) = 0$ if n is a goal.

To get an idea of what a heuristic function looks like, we need to choose a particular problem, because heuristic functions are problem-specific. Let us return to the route-finding problem from Arad to Bucharest. The map for that problem is repeated in Figure 4.2.

STRAIGHT-LINE DISTANCE

A good heuristic function for route-finding problems like this is the **straight-line distance** to the goal. That is,

$h_{SLD}(n) =$ straight-line distance between n and the goal location.



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Figure 4.2 Map of Romania with road distances in km, and straight-line distances to Bucharest.

Notice that we can only calculate the values of h_{SLD} if we know the map coordinates of the cities in Romania. Furthermore, h_{SLD} is only useful because a road from A to B usually tends to head in more or less the right direction. This is the sort of extra information that allows heuristics to help in reducing search cost.

Figure 4.3 shows the progress of a greedy search to find a path from Arad to Bucharest. With the straight-line-distance heuristic, the first node to be expanded from Arad will be Sibiu, because it is closer to Bucharest than either Zerind or Timisoara. The next node to be expanded will be Fagaras, because it is closest. Fagaras in turn generates Bucharest, which is the goal. For this particular problem, the heuristic leads to minimal search cost: it finds a solution without ever expanding a node that is not on the solution path. However, it is not perfectly optimal: the path it found via Sibiu and Fagaras to Bucharest is 32 miles longer than the path through Rimnicu Vilcea and Pitesti. This path was not found because Fagaras is closer to Bucharest in straight-line distance than Rimnicu Vilcea, so it was expanded first. The strategy prefers to take the biggest bite possible out of the remaining cost to reach the goal, without worrying about whether this will be best in the long run—hence the name "greedy search." Although greed is considered one of the seven deadly sins, it turns out that greedy algorithms often perform quite well. They tend to find solutions quickly, although as shown in this example, they do not always find the optimal solutions: that would take a more careful analysis of the long-term options, not just the immediate best choice.

Greedy search is susceptible to false starts. Consider the problem of getting from Iasi to Fagaras. The heuristic suggests that Neamt be expanded first, but it is a dead end. The solution is to go first to Vaslui—a step that is actually farther from the goal according to the heuristic—and then to continue to Urziceni, Bucharest, and Fagaras. Hence, in this case, the heuristic causes unnecessary nodes to be expanded. Furthermore, if we are not careful to detect repeated states, the solution will never be found—the search will oscillate between Neamt and Iasi.

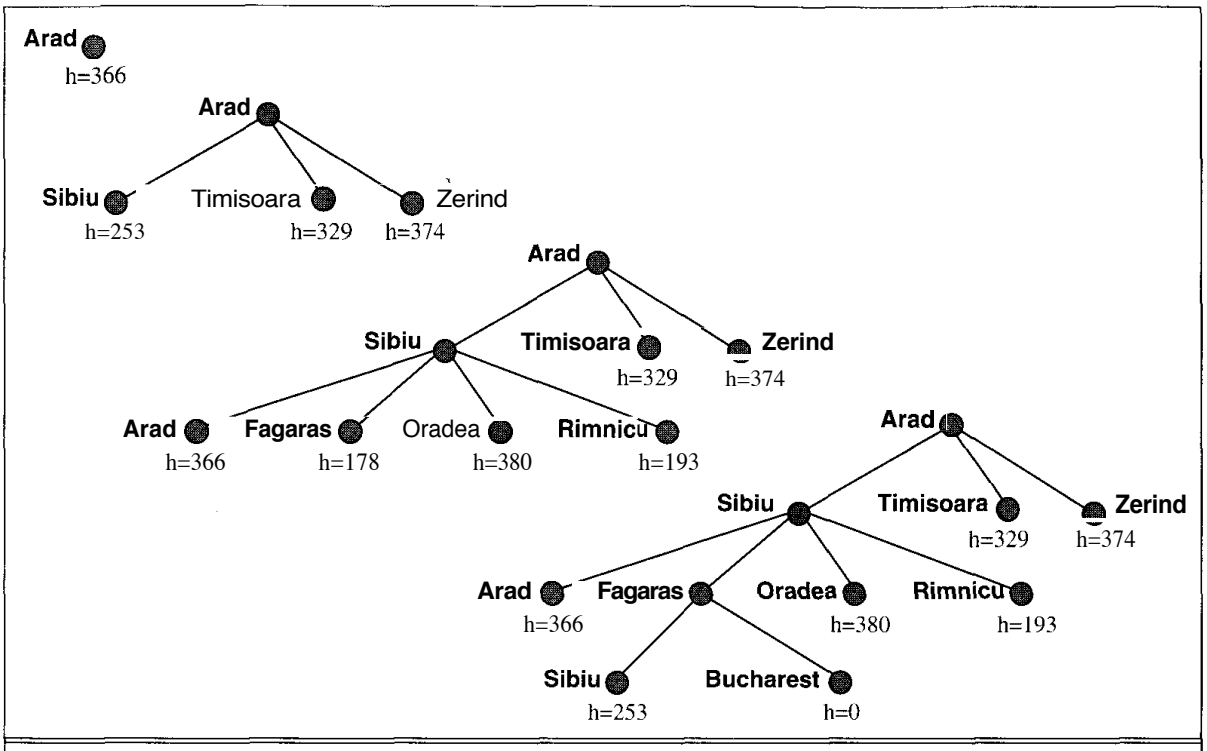


Figure 4.3 Stages in a greedy search for Bucharest, using the straight-line distance to Bucharest as the heuristic function h_{SLD}. Nodes are labelled with their h-values.

Greedy search resembles depth-first search in the way it prefers to follow a single path all the way to the goal, but will back up when it hits a dead end. It suffers from the same defects as depth-first search—it is not optimal, and it is incomplete because it can start down an infinite path and never return to try other possibilities. The worst-case time complexity for greedy search is $O(b^m)$, where m is the maximum depth of the search space. Because greedy search retains all nodes in memory, its space complexity is the same as its time complexity. With a good heuristic function, the space and time complexity can be reduced substantially. The amount of the reduction depends on the particular problem and quality of the h function.

Minimizing the total path cost: A* search

Greedy search minimizes the estimated cost to the goal, $h(n)$, and thereby cuts the search cost considerably. Unfortunately, it is neither optimal nor complete. Uniform-cost search, on the other hand, minimizes the cost of the path so far, $g(n)$; it is optimal and complete, but can be very inefficient. It would be nice if we could combine these two strategies to get the advantages of both. Fortunately, we can do exactly that, combining the two evaluation functions simply by summing them:

$$f(n) = g(n) + h(n).$$

Since $g(n)$ gives the path cost from the start node to node n, and $h(n)$ is the estimated cost of the cheapest path from n to the goal, we have

$f(n)$ = estimated cost of the cheapest solution through n

Thus, if we are trying to find the cheapest solution, a reasonable thing to try first is the node with the lowest value of f. The pleasant thing about this strategy is that it is more than just reasonable. We can actually prove that it is complete and optimal, given a simple restriction on the h function.

ADMISSIBLE HEURISTIC

The restriction is to choose an h function that *never overestimates* the cost to reach the goal. Such an h is called an **admissible heuristic.** Admissible heuristics are by nature optimistic, because they think the cost of solving the problem is less than it actually is. This optimism transfers to the f function as well: *If his admissible, $f(n)$ never overestimates the actual cost of the best solution through n.* Best-first search using f as the evaluation function and an admissible h function is known as A* **search**

A* SEARCH

```
function A*-SEARCH(problem) returns a solution or failure
    return BEST-FIRST-SEARCH(problem,g + h)
```

Perhaps the most obvious example of an admissible heuristic is the straight-line distance h_{SLD} that we used in getting to Bucharest. Straight-line distance is admissible because the shortest path between any two points is a straight line. In Figure 4.4, we show the first few steps of an A* search for Bucharest using the h_{SLD} heuristic. Notice that the A* search prefers to expand from Rimnicu Vilcea rather than from Fagaras. Even though Fagaras is closer to Bucharest, the path taken to get to Fagaras is not as *efficient* in getting close to Bucharest as the path taken to get to Rimnicu. The reader may wish to continue this example to see what happens next.

The behavior of A* search

Before we prove the completeness and optimality of A*, it will be useful to present an intuitive picture of how it works. A picture is not a substitute for a proof, but it is often easier to remember and can be used to generate the proof on demand. First, a preliminary observation: if you examine the search trees in Figure 4.4, you will notice an interesting phenomenon. Along any path from the root, the f-cost never decreases. This is no accident. It holds true for almost all admissible heuristics. A heuristic for which it holds is said to exhibit **monotonicity**.[1]

MONOTONICITY

If the heuristic is one of those odd ones that is not monotonic, it turns out we can make a minor correction that restores monotonicity. Let us consider two nodes n and n', where n is the parent of n'. Now suppose, for example, that $g(n) = 3$ and $h(n) = 4$. Then $f(n) = g(n)+h(n)$- 7— that is, we know that the true cost of a solution path through n is at least 7. Suppose also that $g(n') = 4$ and $h(n')$- 2, so that $f(n') = 6$. Clearly, this is an example of a nonmonotonic heuristic. Fortunately, from the fact that *any path through n' is also a path through* n, we can see that the value of 6 is meaningless, because we already know the true cost is at least 7. Thus, we should

[1] It can be proved (Pearl, 1984) that a heuristic is monotonic if and only if it obeys the triangle inequality. The triangle inequality says that two sides of a triangle cannot add up to less than the third side (see Exercise 4.7). Of course, straight-line distance obeys the triangle inequality and is therefore monotonic.

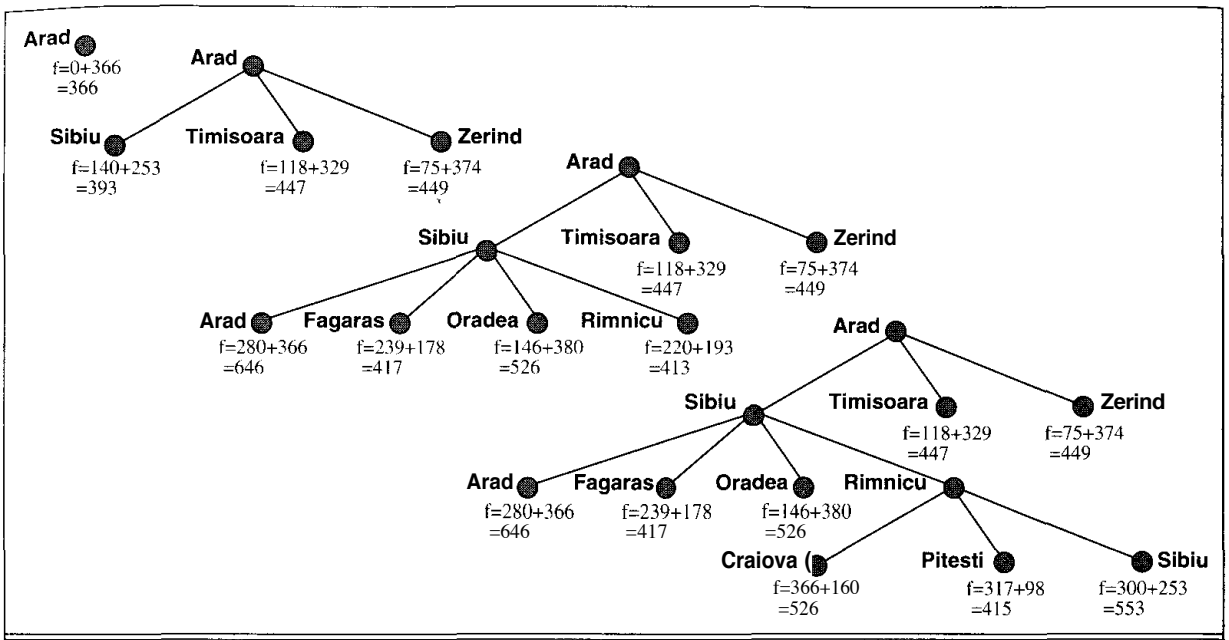


Figure 4.4 Stages in an A* search for Bucharest. Nodes are labelled with / = g + *h*. The *h* values are the straight-line distances to Bucharest taken from Figure 4.1.

check, each time we generate a new node, to see if its/-cost is less than its parent's *f*-cost; if it is, we use the parent's *f*-cost instead:

$$f(n') = max(f(n)g(n') + h(n')).$$

In this way, we ignore the misleading values that may occur with a nonmonotonic heuristic. This equation is called the **pathmax** equation. If we use it, then *f* will always be nondecreasing along any path from the root, provided *h* is admissible.

PATHMAX

The purpose of making this observation is to legitimize a certain picture of what A* does. If *f* never decreases along any path out from the root, we can conceptually draw **contours** in the state space. Figure 4.5 shows an example. Inside the contour labelled 400, all nodes have $f(n)$ less than or equal to 400, and so on. Then, because A* expands the leaf node of lowest *f*, we can see that an A* search fans out from the start node, adding nodes in concentric bands of increasing *f*-cost.

CONTOURS

With uniform-cost search (A* search using $h = 0$), the bands will be "circular" around the start state. With more accurate heuristics, the bands will stretch toward the goal state and become more narrowly focused around the optimal path. If we define f^* to be the cost of the optimal solution path, then we can say the following:

- A* expands all nodes with $f(n) < f^*$.
- A* may then expand some of the nodes right on the "goal contour," for which $f(n) =/*$, before selecting a goal node.

Intuitively, it is obvious that the first solution found must be the optimal one, because nodes in all subsequent contours will have higher *f*-cost, and thus higher *g*-cost (because all goal states have $h(n) = 0$). Intuitively, it is also obvious that A* search is complete. As we add bands of

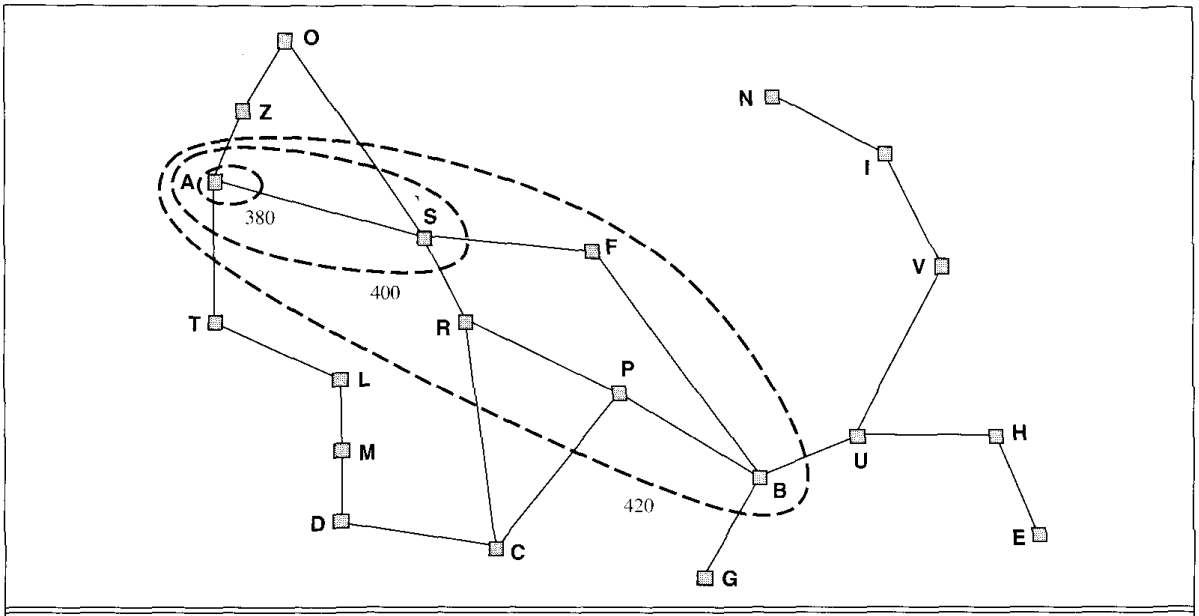


Figure 4.5 Map of Romania showing contours at $f = 380$, $f = 400$ and $f = 420$, with Arad as the start state. Nodes inside a given contour have f-costs lower than the contour value.

increasing f, we must eventually reach a band where f is equal to the cost of the path to a goal state. We will turn these intuitions into proofs in the next subsection.

OPTIMALLY EFFICIENT

One final observation is that among optimal algorithms of this type—algorithms that extend search paths from the root—A* is **optimally efficient** for any given heuristic function. That is, no other optimal algorithm is guaranteed to expand fewer nodes than A*. We can explain this as follows: any algorithm that *does not* expand all nodes in the contours between the root and the goal contour runs the risk of missing the optimal solution. A long and detailed proof of this result appears in Dechter and Pearl (1985).

Proof of the optimality of A*

Let G be an optimal goal state, with path cost f^*. Let G_2 be a suboptimal goal state, that is, a goal state with path cost $g(G_2) > f^*$. The situation we imagine is that A* has selected G_2 from the queue. Because G_2 is a goal state, this would terminate the search with a suboptimal solution (Figure 4.6). We will show that this is not possible.

Consider a node n that is currently a leaf node on an optimal path to G (there must be some such node, unless the path has been completely expanded—in which case the algorithm would have returned G). Because h is admissible, we must have

$$f \geq f(n).$$

Furthermore, if n is not chosen for expansion over G_2, we must have

$$f(n) \geq f(G_2).$$

Combining these two together, we get

$$f > f(G_2).$$

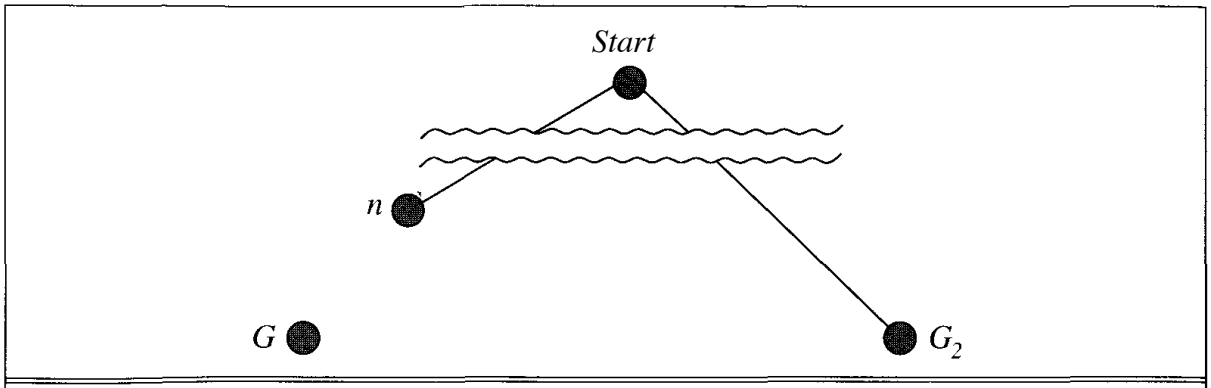


Figure 4.6 Situation at the point where a suboptimal goal state G_2 is about to be expanded. Node n is a leaf node on an optimal path from the start node to the optimal goal state G.

But because G_2 is a goal state, we have $h(G_2) = 0$; hence $f(G_2) = g(G_2)$. Thus, we have proved, from our assumptions, that

$$f^* \geq g(G_2).$$

This contradicts the assumption that G_2 is suboptimal, so it must be the case that A* never selects a suboptimal goal for expansion. Hence, because it only returns a solution after selecting it for expansion, A* is an optimal algorithm.

Proof of the completeness of A*

We said before that because A* expands nodes in order of increasing f, it must eventually expand to reach a goal state. This is true, of course, unless there are infinitely many nodes with $f(n) < f^*$. The only way there could be an infinite number of nodes is either (a) there is a node with an infinite branching factor, or (b) there is a path with a finite path cost but an infinite number of nodes along it.[2]

LOCALLY FINITE GRAPHS

Thus, the correct statement is that A* is complete on **locally finite graphs** (graphs with a finite branching factor) provided there is some positive constant δ such that every operator costs at least δ.

Complexity of A*

That A* search is complete, optimal, and optimally efficient among all such algorithms is rather satisfying. Unfortunately, it does not mean that A* is the answer to all our searching needs. The catch is that, for most problems, the number of nodes within the goal contour search space is still exponential in the length of the solution. Although the proof of the result is beyond the scope of this book, it has been shown that exponential growth will occur unless the error in the heuristic

[2] Zeno's paradox, which purports to show that a rock thrown at a tree will never reach it, provides an example that violates condition (b). The paradox is created by imagining that the trajectory is divided into a series of phases, each of which covers half the remaining distance to the tree; this yields an infinite sequence of steps with a finite total cost.

function grows no faster than the logarithm of the actual path cost. In mathematical notation, the condition for subexponential growth is that

$$|h(n) - h^*(n)| \leq O(\log h^*(n)),$$

where $h^*(n)$ is the *true* cost of getting from n to the goal. For almost all heuristics in practical use, the error is at least proportional to the path cost, and the resulting exponential growth eventually overtakes any computer. Of course, the use of a good heuristic still provides enormous savings compared to an uninformed search. In the next section, we will look at the question of designing good heuristics.

Computation time is not, however, A*'s main drawback. Because it keeps all generated nodes in memory, A* usually runs out of space long before it runs out of time. Recently developed algorithms have overcome the space problem without sacrificing optimality or completeness. These are discussed in Section 4.3.

4.2 HEURISTIC FUNCTIONS

So far we have seen just one example of a heuristic: straight-line distance for route-finding problems. In this section, we will look at heuristics for the 8-puzzle. This will shed light on the nature of heuristics in general.

The 8-puzzle was one of the earliest heuristic search problems. As mentioned in Section 3.3, the object of the puzzle is to slide the tiles horizontally or vertically into the empty space until the initial configuration matches the goal configuration (Figure 4.7).

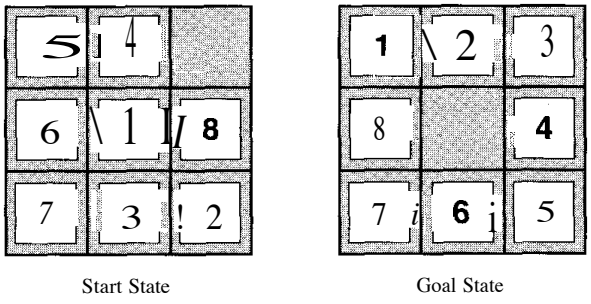


Figure 4.7 A typical instance of the 8-puzzle.

The 8-puzzle is just the right level of difficulty to be interesting. A typical solution is about 20 steps, although this of course varies depending on the initial state. The branching factor is about 3 (when the empty tile is in the middle, there are four possible moves; when it is in a corner there are two; and when it is along an edge there are three). This means that an exhaustive search to depth 20 would look at about $3^{20} = 3.5 \times 10^9$ states. By keeping track of repeated states, we could cut this down drastically, because there are only $9! = 362,880$ different arrangements of 9 squares. This is still a large number of states, so the next order of business is to find a good

5 GAME PLAYING

In which we examine the problems that arise when we try to plan ahead in a world that includes a hostile agent.

5.1 INTRODUCTION: GAMES AS SEARCH PROBLEMS

Games have engaged the intellectual faculties of humans—sometimes to an alarming degree—for as long as civilization has existed. Board games such as chess and Go are interesting in part because they offer pure, abstract competition, without the fuss and bother of mustering up two armies and going to war. It is this abstraction that makes game playing an appealing target of AI research. The state of a game is easy to represent, and agents are usually restricted to a fairly small number of well-defined actions. *That makes game playing an idealization of worlds in which hostile agents act so as to diminish one's well-being*. Less abstract games, such as croquet or football, have not attracted much interest in the AI community.

Game playing is also one of the oldest areas of endeavor in artificial intelligence. In 1950, almost as soon as computers became programmable, the first chess programs were written by Claude Shannon (the inventor of information theory) and by Alan Turing. Since then, there has been steady progress in the standard of play, to the point where current systems can challenge the human world champion without fear of gross embarrassment.

Early researchers chose chess for several reasons. A chess-playing computer would be an existence proof of a machine doing something thought to require intelligence. Furthermore, the simplicity of the rules, and the fact that the world state is fully accessible to the program[1] means that it is easy to represent the game as a search through a space of possible game positions. The computer representation of the game actually can be correct in every relevant detail—unlike the representation of the problem of fighting a war, for example.

[1] Recall from Chapter 2 that **accessible** means that the agent can perceive everything there is to know about the environment. In game theory, chess is called a game of **perfect information.**

The presence of an opponent makes the decision problem somewhat more complicated than the search problems discussed in Chapter 3. The opponent introduces *uncertainty,* because one never knows what he or she is going to do. In essence, all game-playing programs must deal with the **contingency problem** defined in Chapter 3. The uncertainty is not like that introduced, say, by throwing dice or by the weather. The opponent will try as far as possible to make the least benign move, whereas the dice arid the weather are assumed (perhaps wrongly!) to be indifferent to the goals of the agent. This complication is discussed in Section 5.2.

But what makes games *really* different is that they are usually much too hard to solve. Chess, for example, has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{100} nodes (although there are "only" about 10^{40} *different* legal positions). Tic-Tac-Toe (noughts and crosses) is boring for adults precisely because it is easy to determine the right move. The complexity of games introduces a completely new kind of uncertainty that we have not seen so far; the uncertainty arises not because there is missing information, but because one does not have time to calculate the exact consequences of any move. Instead, one has to make one's best guess based on past experience, and act before one is sure of what action to take. In this respect, *games are much more like the real world than the standard search problems we have looked at so far.*

Because they usually have time limits, games also penalize inefficiency very severely. Whereas an implementation of A* search that is 10% less efficient will simply cost a little bit extra to run to completion, a chess program that is 10% less effective in using its available time probably will be beaten into the ground, other things being equal. Game-playing research has therefore spawned a number of interesting ideas on how to make the best use of time to reach good decisions, when reaching optimal decisions is impossible. These ideas should be kept in mind throughout the rest of the book, because the problems of complexity arise in every area of AI. We will return to them in more detail in Chapter 16.

We begin our discussion by analyzing how to find the theoretically best move. We then look at techniques for choosing a good move when time is limited. **Pruning** allows us to ignore portions of the search tree that make no difference to the final choice, and heuristic **evaluation functions** allow us to approximate the true utility of a state without doing a complete search. Section 5.5 discusses games such as backgammon that include an element of chance. Finally, we look at how state-of-the-art game-playing programs succeed against strong human opposition.

5.2 PERFECT DECISIONS IN TWO-PERSON GAMES

We will consider the general case of a game with two players, whom we will call MAX and MIN, for reasons that will soon become obvious. MAX moves first, and then they take turns moving until the game is over. At the end of the game, points are awarded to the winning player (or sometimes penalties are given to the loser). A game can be formally defined as a kind of search problem with the following components:

- **The initial state,** which includes the board position and an indication of whose move it is.
- **A set** of **operators,** which define the legal moves that a player can make.

TERMINAL TEST

- A **terminal test,** which determines when the game is over. States where the game has ended are called **terminal states.**

PAYOFF FUNCTION

- A **utility function** (also called a **payoff function),** which gives a numeric value for the outcome of a game. In chess, the outcome is a win, loss, or draw, which we can represent by the values +1, −1, or 0. Some games have a wider variety of possible outcomes; for example, the payoffs in backgammon range from +192 to −192.

If this were a normal search problem, then all MAX would have to do is search for a sequence of moves that leads to a terminal state that is a winner (according to the utility function), and then go ahead and make the first move in the sequence. Unfortunately, MIN has something to say about it. MAX therefore must find a **strategy** that will lead to a winning terminal state regardless of what MIN does, where the strategy includes the correct move for MAX for each possible move by MIN. We will begin by showing how to find the optimal (or rational) strategy, even though normally we will not have enough time to compute it.

STRATEGY

Figure 5.1 shows part of the search tree for the game of Tic-Tac-Toe. From the initial state, MAX has a choice of nine possible moves. Play alternates between MAX placing x's and MIN placing o's until we reach leaf nodes corresponding to terminal states: states where one player has three in a row or all the squares are filled. The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names). It is MAX'S job to use the search tree (particularly the utility of terminal states) to determine the best move.

Even a simple game like Tic-Tac-Toe is too complex to show the whole search tree, so we will switch to the absolutely trivial game in Figure 5.2. The possible moves for MAX are labelled A_1, A_2, and A_3. The possible replies to A_1 for MIN are A_{11}, $A\backslash 2$, A_{13}, and so on. This particular game ends after one move each by MAX and MIN. (In game parlance, we say this tree is one move deep, consisting of two half-moves or two **ply.)** The utilities of the terminal states in this game range from 2 to 14.

PLY

MINIMAX

The **minimax** algorithm is designed to determine the optimal strategy for MAX, and thus to decide what the best first move is. The algorithm consists of five steps:

- Generate the whole game tree, all the way down to the terminal states.
- Apply the utility function to each terminal state to get its value.
- Use the utility of the terminal states to determine the utility of the nodes one level higher up in the search tree. Consider the leftmost three leaf nodes in Figure 5.2. In the V node above it, MIN has the option to move, and the best MIN can do is choose A_{11}, which leads to the minimal outcome, 3. Thus, even though the utility function is not immediately applicable to this V node, we can assign it the utility value 3, under the assumption that MIN will do the right thing. By similar reasoning, the other two V nodes are assigned the utility value 2.
- Continue backing up the values from the leaf nodes toward the root, one layer at a time.
- Eventually, the backed-up values reach the top of the tree; at that point, MAX chooses the move that leads to the highest value. In the topmost A node of Figure 5.2, MAX has a choice of three moves that will lead to states with utility 3, 2, and 2, respectively. Thus, MAX's best opening move is A_1. This is called the **mininiax decision,** because it maximizes the utility under the assumption that the opponent will play perfectly to minimize it.

MINIMAX DECISION

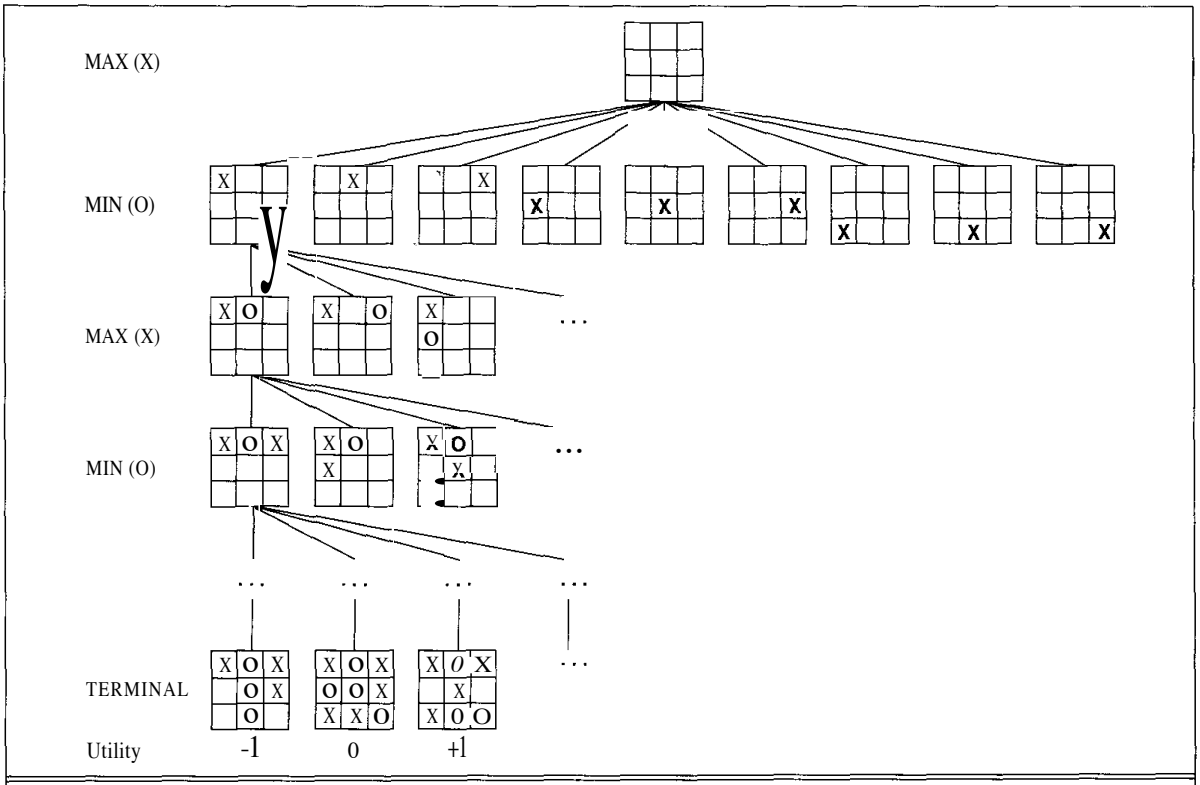


Figure 5.1 A (partial) search tree for the game of Tic-Tac-Toe. The top node is the initial state, and MAX moves first, placing an x in some square. We show part of the search tree, giving alternating moves by MIN (o) and MAX, until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

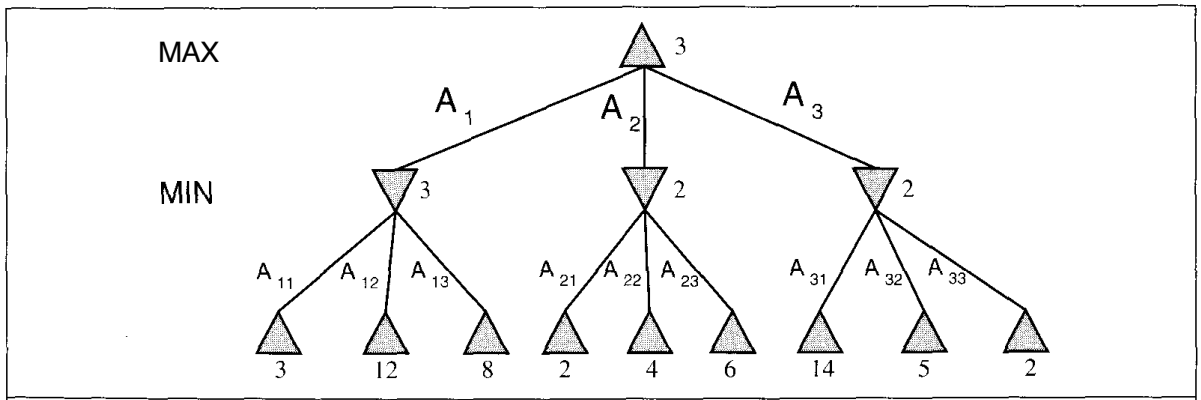


Figure 5.2 A two-ply game *tree* as generated by the minimax algorithm. The A nodes *ars* moves by MAX and the V nodes aremoves by MIN. The terminal nodes show the utility valuefor MAX computed by the utility function (i.e., by the rules of the game), whereas the utilities of the other nodes are computed by the minimax algorithm from the utilities of their successors. MAX'S best move is *A*\, and MIN's best reply is *A*\\.

Figure 5.3 shows a more formal description of the minimax algorithm. The top level function, MINIMAX-DECISION, selects from the available moves, which are evaluated in turn by the MINIMAX-VALUE function.

If the maximum depth of the tree is m, and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$. The algorithm is a depth-first search (although here the implementation is through recursion rather than using a queue of nodes), so its space requirements are only linear in m and b. For real games, of course, the time cost is totally impractical, but this algorithm serves as the basis for more realistic methods and for the mathematical analysis of games.

```
function MINIMAX-DECISION(game) returns an operator

  for each op in OPERATORS[game] do
      VALUE[op] — MINIMAX-VALUE(APPLY(op, game), game)
  end
  return the op with the highest VALUE[op]

function MINIMAX-VALUE(state, game) returns a utility value

  if TERMINAL-TEST[game](state) then
      return  UTILITY[game](state)
  else if MAX is to move in state then
      return the highest MINIMAX-VALUE of SUCCESSORS(state)
  else
      return the lowest MINIMAX-VALUE of SUCCESSORS(state)
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the operator that corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The function MINIMAX-VALUE goes through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

5.3 IMPERFECT DECISIONS

The minimax algorithm assumes that the program has time to search all the way to terminal states, which is usually not practical. Shannon's original paper on chess proposed that instead of going all the way to terminal states and using the utility function, the program should cut off the search earlier and apply a heuristic **evaluation function** to the leaves of the tree. In other words, the suggestion is to alter minimax in two ways: the utility function is replaced by an evaluation function EVAL, and the terminal test is replaced by a cutoff test CUTOFF-TEST.

Evaluation functions

An evaluation function returns an *estimate* of the expected utility of the game from a given position. The idea was not new when Shannon proposed it. For centuries, chess players (and, of course, aficionados of other games) have developed ways of judging the winning chances of each side based on easily calculated features of a position. For example, introductory chess books give an approximate **material value** for each piece: each pawn is worth 1, a knight or bishop is worth 3, a rook 5, and the queen 9. Other features such as "good pawn structure" and "king safety" might be worth half a pawn, say. All other things being equal, a side that has a secure material advantage of a pawn or more will probably win the game, and a 3-point advantage is sufficient for near-certain victory. Figure 5.4 shows four positions with their evaluations.

MATERIAL VALUE

It should be clear that the performance of a game-playing program is extremely dependent on the quality of its evaluation function. If it is inaccurate, then it will guide the program toward positions that are apparently "good," but in fact disastrous. How exactly do we measure quality?

First, the evaluation function must agree with the utility function on terminal states. Second, it must not take too long! (As mentioned in Chapter 4, if we did not limit its complexity, then it could call minimax as a subroutine and calculate the exact value of the position.) Hence, there is a trade-off between the accuracy of the evaluation function and its time cost. Third, an evaluation function should accurately reflect the actual chances of winning.

One might well wonder about the phrase "chances of winning." After all, chess is not a game of chance. But if we have cut off the search at a particular nonterminal state, then we do not know what will happen in subsequent moves. For concreteness, assume the evaluation function counts only material value. Then, in the opening position, the evaluation is 0, because both sides have the same material. All the positions up to the first capture will also have an evaluation of 0. If MAX manages to capture a bishop without losing a piece, then the resulting position will have an evaluation value of 3. The important point is that a given evaluation value covers many different positions—all the positions where MAX is up by a bishop are grouped together into a *category* that is given the label "3." Now we can see how the word "chance" makes sense: the evaluation function should reflect the chance that a position chosen at random from such a category leads to a win (or draw or loss), based on previous experience.[2]

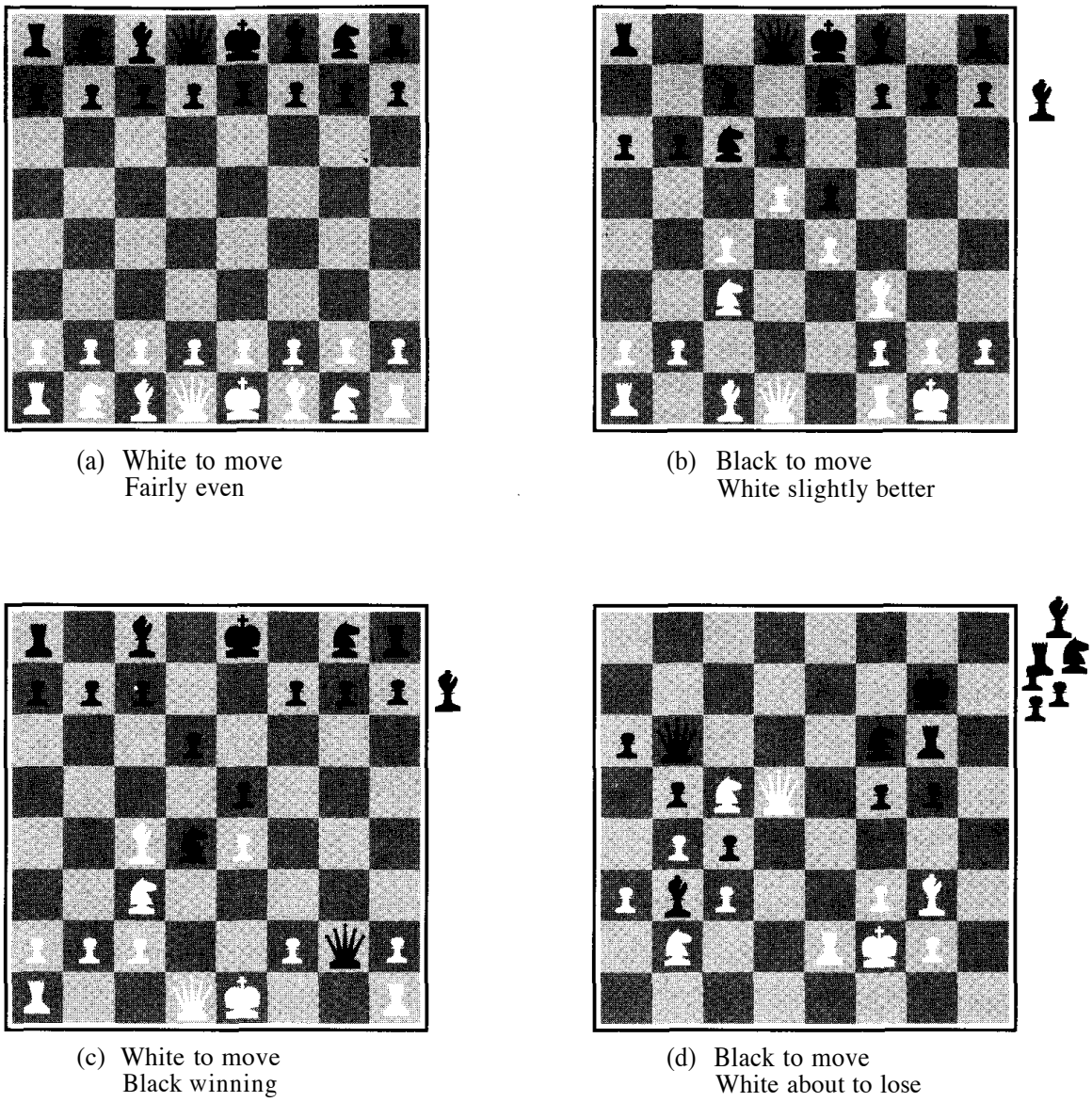
This suggests that the evaluation function should be specified by the rules of probability: if position A has a 100% chance of winning, it should have the evaluation 1.00, and if position B has a 50% chance of winning, 25% of losing, and 25% of being a draw, its evaluation should be $+1 \times .50 + -1 \times .25 + 0 \times .25 = .25$. But in fact, we need not be this precise; the actual numeric values of the evaluation function are not important, as long as A is rated higher than B.

The material advantage evaluation function assumes that the value of a piece can be judged independently of the other pieces present on the board. This kind of evaluation function is called a **weighted linear function,** because it can be expressed as

$$w_1 f_1 + w_2 f_2 + \cdots + w_n f_n$$

where the w's are the weights, and the f's are the features of the particular position. The w's would be the values of the pieces (1 for pawn, 3 for bishop, etc.), and the f's would be the numbers

2 Techniques for automatically constructing evaluation functions with this property are discussed in Chapter 18. In assessing the value of a category, more normally occurring positions should be given more weight.



(a) White to move
Fairly even

(b) Black to move
White slightly better

(c) White to move
Black winning

(d) Black to move
White about to lose

Figure 5.4 Some chess positions and their evaluations.

of each kind of piece on the board. Now we can see where the particular piece values come from: they give the best approximation to the likelihood of winning in the individual categories.

Most game-playing programs use a linear evaluation function, although recently nonlinear functions have had a good deal of success. (Chapter 19 gives an example of a neural network that is trained to learn a nonlinear evaluation function for backgammon.) In constructing the linear formula, one has to first pick the features, and then adjust the weights until the program plays well. The latter task can be automated by having the program play lots of games against itself, but at the moment, no one has a good idea of how to pick good features automatically.

Cutting off search

The most straightforward approach to controlling the amount of search is to set a fixed depth limit, so that the cutoff test succeeds for all nodes at or below depth d. The depth is chosen so that the amount of time used will not exceed what the rules of the game allow. A slightly more robust approach is to apply iterative deepening, as defined in Chapter 3. When time runs out, the program returns the move selected by the deepest completed search.

These approaches can have some disastrous consequences because of the approximate nature of the evaluation function. Consider again the simple evaluation function for chess based on material advantage. Suppose the program searches to the depth limit, reaching the position shown in Figure 5.4(d). According to the material function, white is ahead by a knight and therefore almost certain to win. However, because it is black's move, white's queen is lost because the black knight can capture it without any recompense for white. Thus, in reality the position is won for black, but this can only be seen by looking ahead one more ply.

Obviously, a more sophisticated cutoff test is needed. The evaluation function should only be applied to positions that are **quiescent,** that is, unlikely to exhibit wild swings in value in the near future. In chess, for example, positions in which favorable captures can be made are not quiescent for an evaluation function that just counts material. Nonquiescent positions can be expanded further until quiescent positions are reached. This extra search is called a **quiescence search;** sometimes it is restricted to consider only certain types of moves, such as capture moves, that will quickly resolve the uncertainties in the position.

QUIESCENT

QUIESCENCE SEARCH

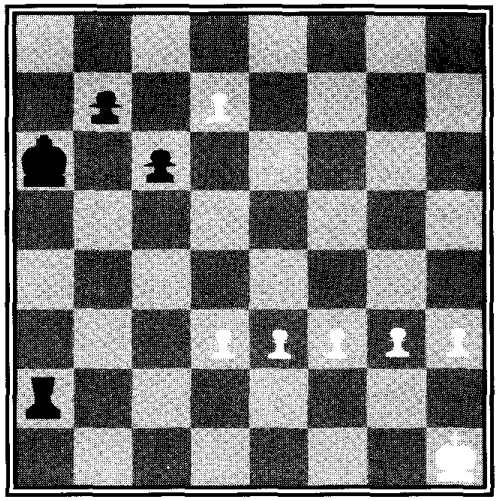
HORIZON PROBLEM

The horizon problem is more difficult to eliminate. It arises when the program is facing a move by the opponent that causes serious damage and is ultimately unavoidable. Consider the chess game in Figure 5.5. Black is slightly ahead in material, but if white can advance its pawn from the seventh row to the eighth, it will become a queen and be an easy win for white. Black can forestall this for a dozen or so ply by checking white with the rook, but inevitably the pawn will become a queen. The problem with fixed-depth search is that it believes that these stalling moves have avoided the queening move—we say that the stalling moves push the inevitable queening move "over the horizon" to a place where it cannot be detected. At present, no general solution has been found for the horizon problem.

5.4 ALPHA-BETA PRUNING

Let us assume we have implemented a minimax search with a reasonable evaluation function for chess, and a reasonable cutoff test with a quiescence search. With a well-written program on an ordinary computer, one can probably search about 1000 positions a second. How well will our program play? In tournament chess, one gets about 150 seconds per move, so we can look at 150,000 positions. In chess, the branching factor is about 35, so our program will be able to look ahead only three or four ply, and will play at the level of a complete novice! Even average human players can make plans six or eight ply ahead, so our program will be easily fooled.

Fortunately, it is possible to compute the correct minimax decision without looking at every node in the search tree. The process of eliminating a branch of the search tree from consideration



Black to move

Figure 5.5 The horizon problem. A series of checks by the black rook forces the inevitable queening move by white "over the horizon" and makes this position look like a slight advantage for black, when it is really a sure win for white.

PRUNING
ALPHA-BETA PRUNING

without examining it is called **pruning** the search tree. The particular technique we will examine is called **alpha-beta pruning.** When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

Consider the two-ply game tree from Figure 5.2, shown again in Figure 5.6. The search proceeds as before: $A\backslash$, then $A\backslash\backslash$, $A\backslash 2$, A_{13}, and the node under A_1 gets minimax value 3. Now we follow A2, and A_{21}, which has value 2. At this point, we realize that if MAX plays A_2, MIN has the option of reaching a position worth 2, and some other options besides. Therefore, we can say already that move A_2 is worth *at most 2* to MAX. Because we already know that move $A\backslash$ is worth 3, there is no point at looking further under A_2. In other words, we can prune the search tree at this point and be confident that the pruning will have no effect on the outcome.

The general principle is this. Consider a node n somewhere in the tree (see Figure 5.7), such that Player has a choice of moving to that node. If Player has a better choice m either at the parent node of n, or at any choice point further up, then *n will never be reached in actual play*. So once we have found out enough about n (by examining some of its descendants) to reach this conclusion, we can prune it.

Remember that minimax search is depth-first, so at any one time we just have to consider the nodes along a single path in the tree. Let a be the value of the best choice we have found so far at any choice point along the path for MAX, and β be the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN. Alpha-beta search updates the value of a and 8 as it goes along, and prunes a subtree (i.e., terminates the recursive call) as soon as it is known to be worse than the current α or 13 value.

The algorithm description in Figure 5.8 is divided into a MAX-VALUE function and a MIN-VALUE function. These apply to MAX nodes and MIN nodes, respectively, but each does the same thing: return the minimax value of the node, except for nodes that are to be pruned (in

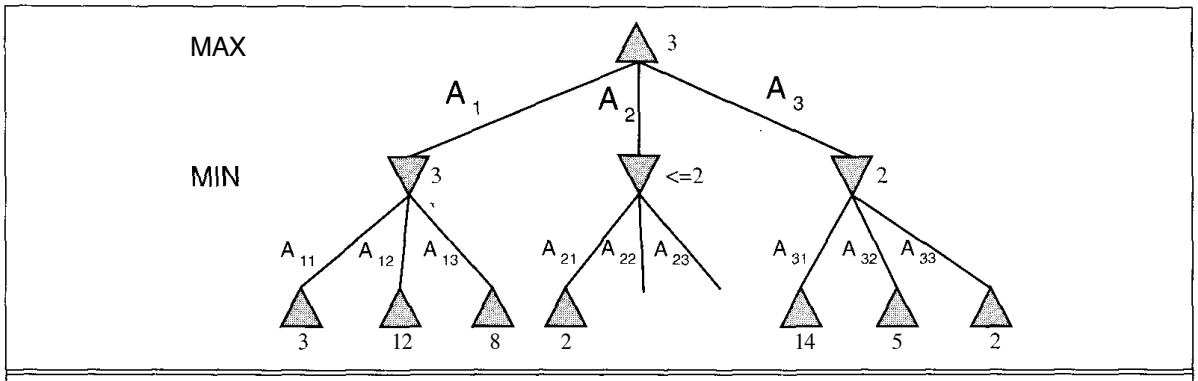


Figure 5.6 The two-ply game tree as generated by alpha-beta.

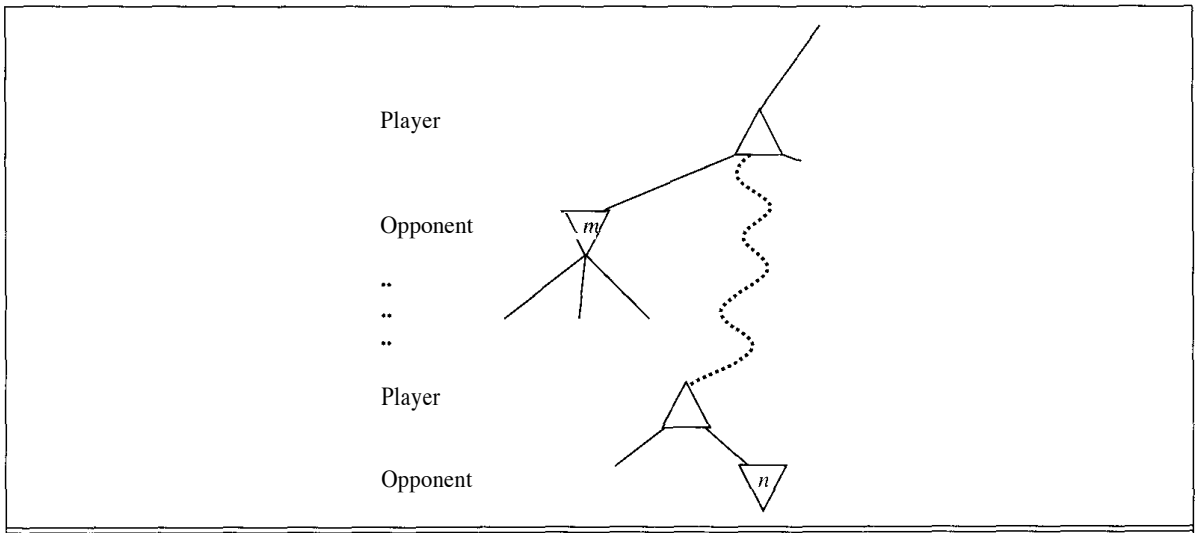


Figure 5.7 Alpha-beta pruning: the general case. If m is better than n for Player, we will never get to n in play.

which case the returned value is ignored anyway). The alpha-beta search function itself is just a copy of the MAX-VALUE function with extra code to remember and return the best move found.

Effectiveness of alpha-beta pruning

The effectiveness of alpha-beta depends on the ordering in which the successors are examined. This is clear from Figure 5.6, where we could not prune $A3$ at all because A_{31} and A_{32} (the worst moves from the point of view of MIN) were generated first. This suggests it might be worthwhile to try to examine first the successors that are likely to be best.

If we assume that this can be done,[3] then it turns out that alpha-beta only needs to examine $O(b^{d/2})$nodes to pick the best move, instead of $O(b^d)$with minimax. This means that the effective branching factor is $\sqrt{b}$ instead of b—for chess, 6 instead of 35. Put another way, this means

[3] Obviously, it cannot be done perfectly, otherwise the ordering function could be used to play a perfect game!

```
function MAX-VALUE(stategame, a, β) returns the minimax value of state
  inputs: state, current state in game
          game, game description
          a, the best score for MAX along the path to state
          β, the best score for MIN along the path to state

  if CUTOFF-TEST(state) then return EVAL(state)
  for each s in SUCCESSORS(state) do
     a ← MAX(α , MIN-VALUE(s, game, a, β))
     if a ≥ ft then return ft
  end
  return a
```

```
function MIN-VALUE(state, game, a, ft) returns the minimax value of state

  if CUTOFF-TEST(state) then return EVAL(state)
  for each s in SUCCESSORS(state) do
     ft ← MIN(ft, MAX-VALUE(s, game, a, β))
     if ft < a then return a
  end
  return ft
```

Figure 5.8 The alpha-beta search algorithm. It does the same computation as a normal minimax, but prunes the search tree.

that alpha-beta can look ahead twice as far as minimax for the same cost. Thus, by generating 150,000 nodes in the time allotment, a program can look ahead eight ply instead of four. By thinking carefully about *which computations actually affect the decision,* we are able to transform a program from a novice into an expert.

The effectiveness of alpha-beta pruning was first analyzed in depth by Knuth and Moore (1975). As well as the best case described in the previous paragraph, they analyzed the case in which successors are ordered randomly. It turns out that the asymptotic complexity is $O((b/\log b)^d)$,which seems rather dismal because the effective branching factor $b/\log b$ is not much less than b itself. On the other hand, the asymptotic formula is only accurate for $b > 1000$ or so—in other words, not for any games we can reasonably play using these techniques. For reasonable b, the total number of nodes examined will be roughly $O(b^{3d/4})$. In practice, a fairly simple ordering function (such as trying captures first, then threats, then forward moves, then backward moves) gets you fairly close to the best-case result rather than the random result. Another popular approach is to do an iterative deepening search, and use the backed-up values from one iteration to determine the ordering of successors in the next iteration.

TREE MODEL

It is also worth noting that all complexity results on games (and, in fact, on search problems in general) have to assume an idealized **tree model** in order to obtain their results. For example, the model used for the alpha-beta result in the previous paragraph assumes that all nodes have the same branching factor b; that all paths reach the fixed depth limit d; and that the leaf evaluations

are randomly distributed across the last layer of the tree. This last assumption is seriously flawed: for example, if a move higher up the tree is a disastrous blunder, then most of its descendants will look bad for the player who made the blunder. The value of a node is therefore likely to be highly correlated with the values of its siblings. The amount of correlation depends very much on the particular game and indeed the particular position at the root. Hence, there is an unavoidable component of *empirical science* involved in game-playing research, eluding the power of mathematical analysis.

HISTORY OF "HEURISTIC"

> By now the space aliens had mastered my own language, but they still made simple mistakes like using "hermeneutic" when they meant "heuristic."
> — a Louisiana factory worker in Woody Alien's *The UFO Menace*

The word "heuristic" is derived from the Greek verb *heuriskein*, meaning "to find" or "to discover." Archimedes is said to have run naked down the street shouting "*Heureka*" (I have found it) after discovering the principle of flotation in his bath. Later generations converted this to Eureka.

The technical meaning of "heuristic" has undergone several changes in the history of AI. In 1957, George Polya wrote an influential book called *How to Solve It* that used "heuristic" to refer to the study of methods for discovering and inventing problem-solving techniques, particularly for the problem of coming up with mathematical proofs. Such methods had often been deemed not amenable to explication.

Some people use heuristic as the opposite of algorithmic. For example, Newell, Shaw, and Simon stated in 1963, "A process that may solve a given problem, but offers no guarantees of doing so, is called a heuristic for that problem." But note that there is nothing random or nondeterministic about a heuristic search algorithm: it proceeds by algorithmic steps toward its result. In some cases, there is no guarantee how long the search will take, and in some cases, the quality of the solution is not guaranteed either. Nonetheless, it is important to distinguish between "nonalgorithmic" and "not precisely characterizable."

Heuristic techniques dominated early applications of artificial intelligence. The first "expert systems" laboratory, started by Ed Feigenbaum, Brace Buchanan, and Joshua Lederberg at Stanford University, was called the Heuristic Programming Project (HPP). Heuristics were viewed as "rules of thumb" that domain experts could use to generate good solutions without exhaustive search. Heuristics were initially incorporated directly into the structure of programs, but this proved too inflexible when a large number of heuristics were needed. Gradually, systems were designed that could accept heuristic information expressed as "rules," and rule-based systems were born.

Currently, heuristic is most often used as an adjective, referring to any technique that improves the average-case performance on a problem-solving task, but does not necessarily improve the worst-case performance. In the specific area of search algorithms, it refers to a function that provides an estimate of solution cost.

A good article on heuristics (and one on hermeneutics!) appears in the *Encyclopedia of AI* (Shapiro, 1992).

6.4 PROPOSITIONAL LOGIC: A VERY SIMPLE LOGIC

Despite its limited expressiveness, propositional logic serves to illustrate many of the concepts of logic just as well as first-order logic. We will describe its syntax, semantics, and associated inference procedures.

Syntax

The syntax of propositional logic is simple. The symbols of propositional logic are the logical constants *True* and *False,* proposition symbols such as *P* and *Q,* the logical connectives A, V, $\Leftrightarrow$, $\Rightarrow$, and $\neg$, and parentheses, ( ). All sentences are made by putting these symbols together using the following rules:

- The logical constants *True* and *False* are sentences by themselves.
- A propositional symbol such as *P* or *Q* is a sentence by itself.
- Wrapping parentheses around a sentence yields a sentence, for example, (P A Q).
- A sentence can be formed by combining simpler sentences with one of the five logical connectives:

CONJUNCTION (LOGIC)

DISJUNCTION

A (and). A sentence whose main connective is A, such as *P* A *(Q* V *R),* is called a **conjunction (logic);** its parts are the **conjuncts.** (The A looks like an "A" for "And.")

V (or). A sentence using V, such as A V (P A *Q),* is a **disjunction** of the **disjuncts** *A* and *(P* A *Q).* (Historically, the V comes from the Latin "vel," which means "or." For most people, it is easier to remember as an upside-down and.)

IMPLICATION
PREMISE
CONCLUSION

$\Rightarrow$ (implies). A sentence such as $(P \wedge Q) \Rightarrow R$ is called an **implication** (or conditional). Its **premise or antecedent is *P* A *Q*, and** its **conclusion or consequent is** *R*. Implications are also known as **rules or if–then** statements. The implication symbol is sometimes written in other books as $\supset$ or —.

EQUIVALENCE

$\Leftrightarrow$ (equivalent). The sentence *(P* A *Q)* $\Leftrightarrow$ *(Q* A *P)* is an **equivalence** (also called a **biconditional).**

NEGATION

$\neg$ (not). A sentence such as $\neg P$ is called the **negation** of P. All the other connectives combine two sentences into one; $\neg$ is the only connective that operates on a single sentence.

ATOMIC SENTENCES
COMPLEX SENTENCES
LITERAL

Figure 6.8 gives a formal grammar of propositional logic; see page 854 if you are not familiar with the BNF notation. The grammar introduces **atomic sentences,** which in propositional logic consist of a single symbol (e.g., *P),* and **complex sentences,** which contain connectives or parentheses (e.g., *P* A *Q).* The term literal is also used, meaning either an atomic sentences or a negated atomic sentence.

```
Sentence      — AtomicSentence | ComplexSentence

AtomicSentence — True | False
               | P | Q | R | ...
ComplexSentence — ( Sentence )
               | Sentence Connective Sentence
               | ¬Sentence

Connective    — A | V | ⇔ | ⇒
```

Figure 6.8 A BNF (Backus-Naur Form) grammar of sentences in propositional logic.

Strictly speaking, the grammar is ambiguous — a sentence such as *P* A *Q* V *R* could be parsed as either *(P* A *Q)* V *R* or as *P* A *(Q* V *R).* This is similar to the ambiguity of arithmetic expressions such as $P + Q \times R$, and the way to resolve the ambiguity is also similar: we pick an order of precedence for the operators, but use parentheses whenever there might be confusion. The order of precedence in propositional logic is (from highest to lowest): $\neg$, A, V, $\Rightarrow$, and $\Leftrightarrow$. Hence, the sentence

$$\neg P \vee Q \wedge R \Rightarrow S$$

is equivalent to the sentence

$$((\neg P) \text{ V } (\text{Q A } R)) \Rightarrow S.$$

Semantics

The **semantics** of propositional logic is also quite straightforward. We define it by specifying the interpretation of the proposition symbols and constants, and specifying the meanings of the logical connectives.

A proposition symbol can mean whatever you want. That is, its interpretation can be any arbitrary fact. The interpretation of P might be the fact that Paris is the capital of France or that the wumpus is dead. A sentence containing just a proposition symbol is satisfiable but not valid: it is true just when the fact that it refers to is the case.

With logical constants, you have no choice; the sentence *True* always has as its interpretation the way the world actually is—the true fact. The sentence *False* always has as its interpretation the way the world is not.

A complex sentence has a meaning derived from the meaning of its parts. Each connective can be thought of as a function. Just as addition is a function that takes two numbers as input and returns a number, so *and* is a function that takes two truth values as input and returns a truth value. We know that one way to define a function is to make a table that gives the output value for every possible input value. For most functions (such as addition), this is impractical because of the size of the table, but there are only two possible truth values, so a logical function with two arguments needs a table with only four entries. Such a table is called a **truth table.** We give truth tables for the logical connectives in Figure 6.9. To use the table to determine, for example, the value of *True* V *False,* first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the P V Q column to see the result: *True*.

TRUTH TABLE

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
False	*False*	*True*	*False*	*False*	*True*	*True*
False	*True*	*True*	*False*	*True*	*True*	*False*
True	*False*	*False*	*False*	*True*	*False*	*False*
True	*True*	*False*	*True*	*True*	*True*	*True*

Figure 6.9 Truth tables for the five logical connectives.

Truth tables define the semantics of sentences such as *True* A *True*. Complex sentences such as (P V 0 A $\neg S$ are defined by a process of decomposition: first, determine the meaning of *(P* A Q*)* and of $\neg S$, and then combine them using the definition of the A function. This is exactly analogous to the way a complex arithmetic expression such as $(p \times q) + -s$ is evaluated.

The truth tables for "and," "or," and "not" are in close accord with our intuitions about the English words. The main point of possible confusion is that P V Q is true when either *or both P* and Q are true. There is a different connective called "exclusive or" ("xor" for short) that gives false when both disjuncts are true.[7] There is no consensus on the symbol for exclusive or; two choices are V and $\dot{+}$.

In some ways, the implication connective $\Rightarrow$ is the most important, and its truth table might seem puzzling at first, because it does not quite fit our intuitive understanding of "P implies Q"

[7] Latin has a separate word, *aut*, for exclusive or.

or "if P then Q." For one thing, propositional logic does not require any relation of causation or relevance between P and Q. The sentence "5 is odd implies Tokyo is the capital of Japan" is a true sentence of propositional logic (under the normal interpretation), even though it is a decidedly odd sentence of English. Another point of confusion is that any implication is true whenever its antecedent is false. For example, "5 is even implies Sam is smart" is true, regardless of whether Sam is smart. This seems bizarre, but it makes sense if you think of "$P \Rightarrow$ Q" as saying, "If P is true, then I am claiming that Q is true. Otherwise I am making no claim."

Validity and inference

Truth tables can be used not only to define the connectives, but also to test for valid sentences. Given a sentence, we make a truth table with one row for each of the possible combinations of truth values for the proposition symbols in the sentence. For each row, we can calculate the truth value of the entire sentence. If the sentence is true in every row, then the sentence is valid. For example, the sentence

$$((P \vee H) \wedge \neg H) \Rightarrow P$$

is valid, as can be seen in Figure 6.10. We include some intermediate columns to make it clear how the final column is derived, but it is not important that the intermediate columns are there, as long as the entries in the final column follow the definitions of the connectives. Suppose P means that there is a wumpus in [1,3] and H means there is a wumpus in [2,2]. If at some point we learn $(P \vee H)$ and then we also learn $\neg H$, then we can use the valid sentence above to conclude that P is true—that the wumpus is in [1,3].

P	H	$P \vee H$	$(P \vee H) \wedge \neg H$	$((P \vee H) \wedge \neg H) \Rightarrow P$
False	*False*	*False*	*False*	*True*
False	*True*	*True*	*False*	*True*
True	*False*	*True*	*True*	*True*
True	*True*	*True*	*False*	*True*

Figure 6.10 Truth table showing validity of a complex sentence.

This is important. It says that if a machine has some premises and a possible conclusion, it can determine if the conclusion is true. It can do this by building a truth table for the sentence *Premises* $\Rightarrow$ *Conclusion* and checking all the rows. If every row is true, then the conclusion is entailed by the premises, which means that the fact represented by the conclusion follows from the state of affairs represented by the premises. Even though the machine has no idea what the conclusion means, the user could read the conclusions and use his or her interpretation of the proposition symbols to see what the conclusions mean—in this case, that the wumpus is in [1,3]. Thus, we have fulfilled the promise made in Section 6.3.

It will often be the case that the sentences input into the knowledge base by the user refer to a world to which the computer has no independent access, as in Figure 6.11, where it is the user who observes the world and types sentences into the computer. It is therefore essential

that a reasoning system be able to draw conclusions that follow from the premises, regardless of the world to which the sentences are intended to refer. But it is a good idea for a reasoning system to follow this principle in any case. Suppose we replace the "user" in Figure 6.11 with a camera-based visual processing system that sends input sentences to the reasoning system. It makes no difference! Even though the computer now has "direct access" to the world, inference can still take place through direct operations on the syntax of sentences, without any additional information as to their intended meaning.

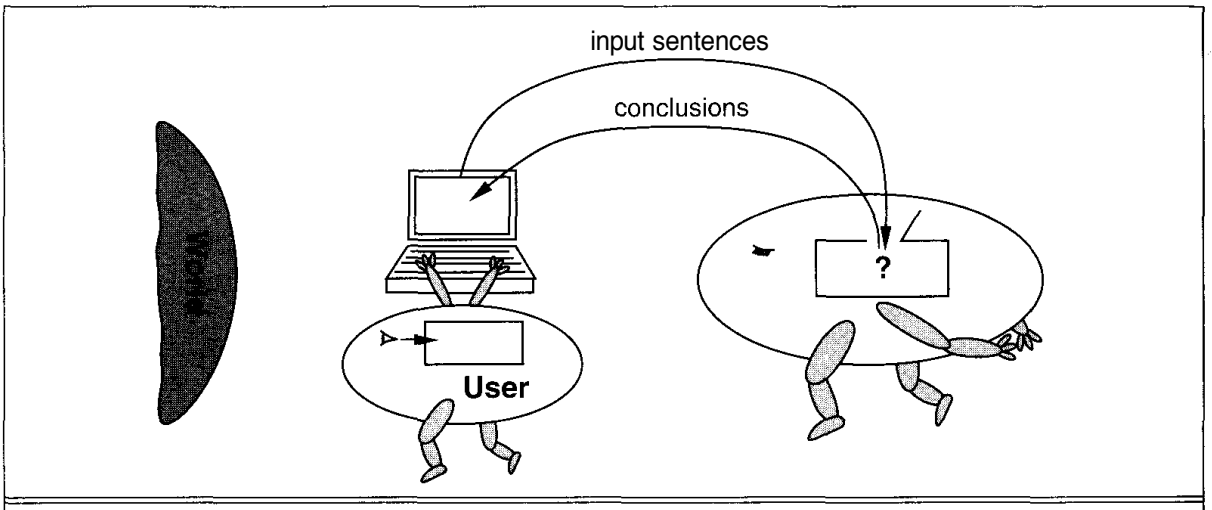


Figure 6.11 Sentences often refer to a world to which the agent has no independent access.

Models

MODEL

Any world in which a sentence is true under a particular interpretation is called a **model** of that sentence under that interpretation. Thus, the world shown in Figure 6.2 is a model of the sentence "$S_{1,2}$" under the interpretation in which it means that there is a stench in [1,2]. There are many other models of this sentence—you can make up a world that does or does not have pits and gold in various locations, and as long as the world has a stench in [1,2], it is a model of the sentence. The reason there are so many models is because "$S_{1,2}$" makes a very weak claim about the world. The more we claim (i.e., the more conjunctions we add into the knowledge base), the fewer the models there will be.

Models are very important in logic, because, to restate the definition of **entailment,** *a sentence a is entailed by a knowledge base KB if the models of KB are all models of a.* If this is the case, then whenever *KB* is true, *a* must also be true.

In fact, we could define the meaning of a sentence by means of set operations on sets of models. For example, the set of models of P A Q is the intersection of the models of P and the models of Q. Figure 6.12 diagrams the set relationships for the four binary connectives.

We have said that models are worlds. One might feel that real worlds are rather messy things on which to base a formal system. Some authors prefer to think of models as *mathematical* objects. In this view, a model in propositional logic is simply a mapping from proposition symbols

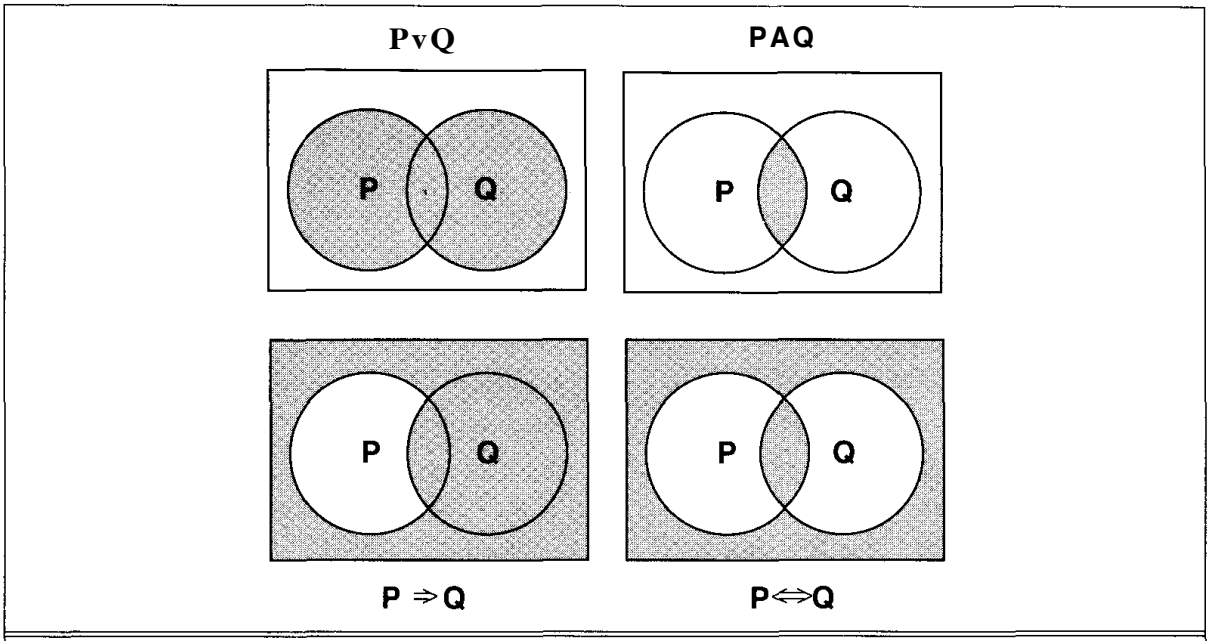


Figure 6.12 Models of complex sentences in terms of the models of their components. In each diagram, the shaded parts correspond to the models of the complex sentence.

directly to truth and falsehood, that is, the label for a row in a truth table. Then the models of a sentence are just those mappings that make the sentence true. The two views can easily be reconciled because each possible assignment of true and false to a set of proposition symbols can be viewed as an equivalence class of worlds that, under a given interpretation, have those truth values for those symbols. There may of course be many different "real worlds" that have the same truth values for those symbols. The only requirement to complete the reconciliation is that each proposition symbol be *either true orfalse* in each world. This is, of course, the basic ontological assumption of propositional logic, and is what allows us to expect that manipulations of symbols lead to conclusions with reliable counterparts in the actual world.

Rules of inference for propositional logic

The process by which the soundness of an inference is established through truth tables can be extended to entire *classes* of inferences. There are certain patterns of inferences that occur over and over again, and their soundness can be shown once and for all. Then the pattern can be captured in what is called an **inference rule.** Once a rule is established, it can be used to make inferences without going through the tedious process of building truth tables.

INFERENCE RULE

We have already seen the notation $a \vdash \beta$ to say that β can be derived from a by inference. There is an alternative notation,

$$\frac{a}{\beta}$$

which emphasizes that this is not a sentence, but rather an inference rule. Whenever something in the knowledge base matches the pattern above the line, the inference rule concludes the premise

below the line. The letters α, β, etc., are intended to match any sentence, not just individual proposition symbols. If there are several sentences, in either the premise or the conclusion, they are separated by commas. Figure 6.13 gives a list of seven commonly used inference rules.

An inference rule is sound if the conclusion is true in all cases where the premises are true. To verify soundness, we therefore construct a truth table with one line for each possible model of the proposition symbols in the premise, and show that in all models where the premise is true, the conclusion is also true. Figure 6.14 shows the truth table for the resolution rule.

◇ **Modus Ponens or Implication-Elimination:** (From an implication and the premise of the implication, you can infer the conclusion.)

$$\frac{a \Rightarrow \beta, \quad a}{\beta}$$

◇ **And-Elimination**: (From a conjunction, you can infer any of the conjuncts.)

$$\frac{\alpha_1 \text{ A } \alpha_2 \text{ A } \ldots \text{ A } \alpha_n}{\alpha_i}$$

◇ **And-Introduction**: (From a list of sentences, you can infer their conjunction.)

$$\frac{\alpha_1, \alpha_2, \bullet \bullet \bullet, \alpha_n}{\alpha_1 \text{ A } \alpha_2 \text{ A } \ldots \text{ A } \alpha_n}$$

0 Or-Introduction: (From a sentence, you can infer its disjunction with anything else at all.)

$$\frac{\alpha_i}{\alpha_1 \text{ V } \alpha_2 \text{ V } \ldots \text{ V } \alpha_n}$$

◇ **Double-Negation Elimination:** (From a doubly negated sentence, you can infer a positive sentence.)

$$\frac{\neg\neg a}{\alpha}$$

◇ **Unit Resolution:** (From a disjunction, if one of the disjuncts is false, then you can infer the other one is true.)

$$\frac{a \text{ V } \beta, \quad \neg\beta}{a}$$

◇ **Resolution:** (This is the most difficult. Because *0* cannot be both true and false, one of the other disjuncts must be true in one of the premises. Or equivalently, implication is transitive.)

$$\frac{a \vee \beta, \quad \neg\beta \text{ V } 7}{a \vee \gamma} \qquad \text{or equivalently} \qquad \frac{\neg\alpha \Rightarrow \beta, \quad \beta \Rightarrow \gamma}{\neg\alpha \Rightarrow \gamma}$$

Figure 6.13 Seven inference rules for propositional logic. The unit resolution rule is a special case of the resolution rule, which in turn is a special case of the full resolution rule for first-order logic discussed in Chapter 9.

α	β	7	$\alpha \vee \beta$	$\neg\beta$ V 7	a V 7
False	*False*	*False*	*False*	*True*	*False*
False	*False*	*True*	*False*	*True*	*True*
False	*True*	*False*	*True*	*False*	*False*
False	*True*	*True*	*True*	*True*	*True*
True	*False*	*False*	*True*	*True*	*True*
True	*False*	*True*	*True*	*True*	*True*
True	*True*	*False*	*True*	*False*	*True*
True	*True*	*True*	*True*	*True*	*True*

Figure 6.14 A truth table demonstrating the soundness of the resolution inference rule. We have underlined the rows where both premises are true.

As we mentioned above, a logical proof consists of a sequence of applications of inference rules, starting with sentences initially in the KB, and culminating in the generation of the sentence whose proof is desired. To prove that P follows from $(P \vee H)$ and $\neg H$, for example, we simply require one application of the resolution rule, with a as P, β as H, and 7 empty. The job of an inference procedure, then, is to construct proofs by finding appropriate sequences of applications of inference rules.

Complexity of propositional inference

The truth-table method of inference described on page 169 is complete, because it is always possible to enumerate the 2" rows of the table for any proof involving n proposition symbols. On the other hand, the computation time is exponential in n, and therefore impractical. One might wonder whether there is a polynomial-time proof procedure for propositional logic based on using the inference rules from Section 6.4.

In fact, a version of this very problem was the first addressed by Cook (1971) in his theory of NP-completeness. (See also the appendix on complexity.) Cook showed that checking a set of sentences for satisfiability is NP-complete, and therefore unlikely to yield to a polynomial-time algorithm. However, this does not mean that all instances of propositional inference are going to take time proportional to 2^n. In many cases, the proof of a given sentence refers only to a small subset of the KB and can be found fairly quickly. In fact, as Exercise 6.15 shows, really hard problems are quite rare.

MONOTONICITY

The use of inference rules to draw conclusions from a knowledge base relies implicitly on a general property of certain logics (including propositional and first-order logic) called **monotonicity.** Suppose that a knowledge base KB entails some set of sentences. A logic is monotonic if when we add some new sentences to the knowledge base, all the sentences entailed by the original KB are still entailed by the new larger knowledge base. Formally, we can state the property of monotonicity of a logic as follows:

if $KB_1 \models a$ **then** $(KB_1 \cup KB_2) \models a$

This is true regardless of the contents of KB_2—it can be irrelevant or even contradictory to KB_1.

It is fairly easy to show that propositional and first-order logic are monotonic in this sense; one can also show that probability theory is not monotonic (see Chapter 14). An inference rule such as Modus Ponens is **local** because its premise need only be compared with a small portion of the KB (two sentences, in fact). Were it not for monotonicity, we could not have any local inference rules because the rest of the KB might affect the soundness of the inference. This would potentially cripple any inference procedure.

LOCAL

There is also a useful class of sentences for which a polynomial-time inference procedure exists. This is the class called **Horn sentences**.[8] A Horn sentence has the form:

HORN SENTENCES

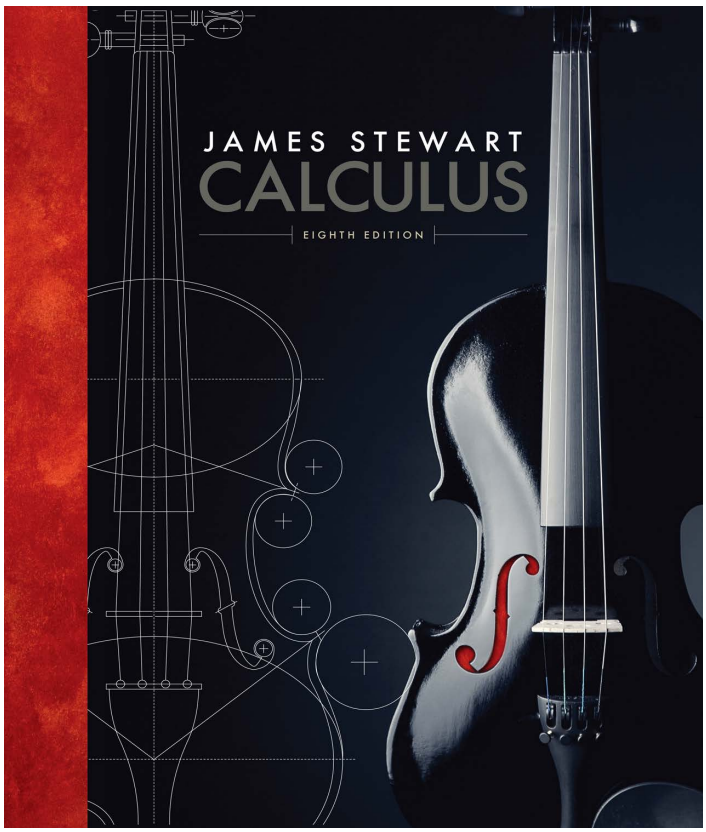
$$P_1 \text{ A } P_2 \text{ A } \ldots \text{ A } P_n \Rightarrow Q$$

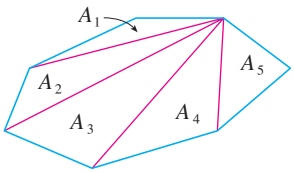
where the P_i and Q are nonnegated atoms. There are two important special cases: -First, when Q is the constant *False,* we get a sentence that is equivalent to $\neg P_1 \text{ V } \ldots \text{ V } \neg P_n$. Second, when $n = 1$ and $P_1 = True$, we get $True \Rightarrow Q$, which is equivalent to the atomic sentence Q. Not every knowledge base can be written as a collection of Horn sentences, but for those that can, we can use a simple inference procedure: apply Modus Ponens wherever possible until no new inferences remain to be made. We discuss Horn sentences and their associated inference procedures in more detail in the context of first-order logic (Section 9.4).

Calculus

James Stewart

We have compressed all of differential calculus (the finding of derivatives) into a few slides and workgroup exercise. In principle, this should be achievable, but if you've never had calculus, even in high school, it may prove challenging. If you managed the workgroup exercise easily, you can skip this chapter. If not, this chapter might help to give you some background. For more articles, see the additional materials on blackboard.





$A = A_1 + A_2 + A_3 + A_4 + A_5$

FIGURE 1

The Area Problem

The origins of calculus go back at least 2500 years to the ancient Greeks, who found areas using the "method of exhaustion." They knew how to find the area A of any polygon by dividing it into triangles as in Figure 1 and adding the areas of these triangles.

It is a much more difficult problem to find the area of a curved figure. The Greek method of exhaustion was to inscribe polygons in the figure and circumscribe polygons about the figure and then let the number of sides of the polygons increase. Figure 2 illustrates this process for the special case of a circle with inscribed regular polygons.

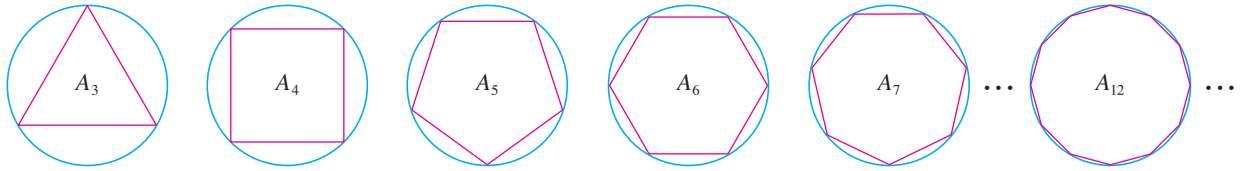


FIGURE 2

Let A_n be the area of the inscribed polygon with n sides. As n increases, it appears that A_n becomes closer and closer to the area of the circle. We say that the area of the circle is the limit of the areas of the inscribed polygons, and we write

$$A = \lim_{n \to \infty} A_n$$

TEC In the Preview Visual, you can see how areas of inscribed and circumscribed polygons approximate the area of a circle.

The Greeks themselves did not use limits explicitly. However, by indirect reasoning, Eudoxus (fifth century BC) used exhaustion to prove the familiar formula for the area of a circle: $A = \pi r^2$.

We will use a similar idea in Chapter 4 to find areas of regions of the type shown in Figure 3. We will approximate the desired area A by areas of rectangles (as in Figure 4), let the width of the rectangles decrease, and then calculate A as the limit of these sums of areas of rectangles.

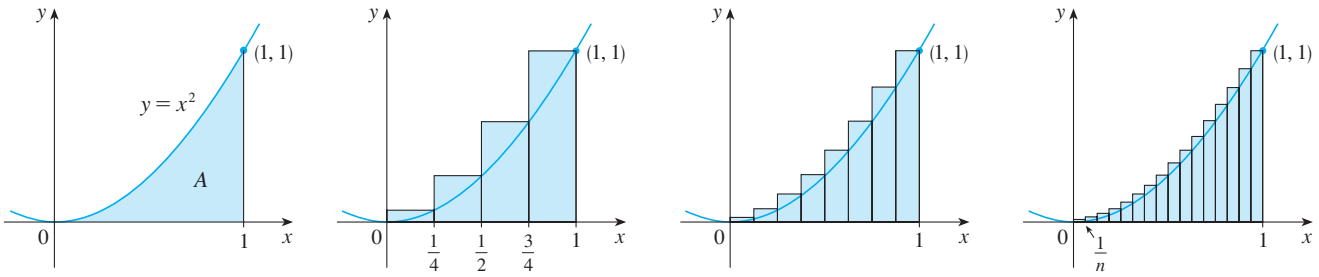


FIGURE 3

The area problem is the central problem in the branch of calculus called *integral calculus*. The techniques that we will develop in Chapter 4 for finding areas will also enable us to compute the volume of a solid, the length of a curve, the force of water against a dam, the mass and center of gravity of a rod, and the work done in pumping water out of a tank.

The Tangent Problem

Consider the problem of trying to find an equation of the tangent line t to a curve with equation $y = f(x)$ at a given point P. (We will give a precise definition of a tangent line in

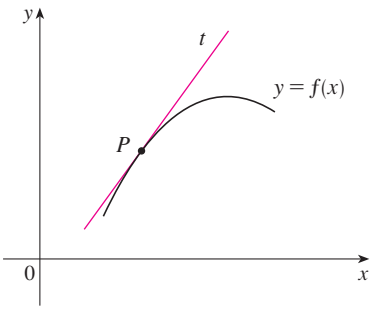


FIGURE 5
The tangent line at P

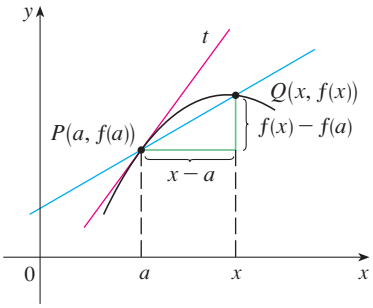


FIGURE 6
The secant line at PQ

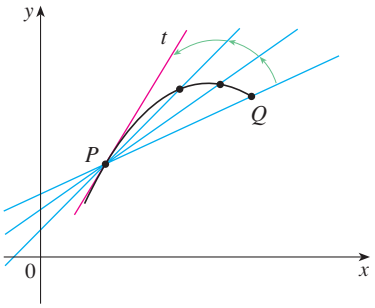


FIGURE 7
Secant lines approaching the tangent line

Chapter 1. For now you can think of it as a line that touches the curve at P as in Figure 5.) Since we know that the point P lies on the tangent line, we can find the equation of t if we know its slope m. The problem is that we need two points to compute the slope and we know only one point, P, on t. To get around the problem we first find an approximation to m by taking a nearby point Q on the curve and computing the slope m_{PQ} of the secant line PQ. From Figure 6 we see that

$$m_{PQ} = \frac{f(x) - f(a)}{x - a} \quad \boxed{1}$$

Now imagine that Q moves along the curve toward P as in Figure 7. You can see that the secant line rotates and approaches the tangent line as its limiting position. This means that the slope m_{PQ} of the secant line becomes closer and closer to the slope m of the tangent line. We write

$$m = \lim_{Q \to P} m_{PQ}$$

and we say that m is the limit of m_{PQ} as Q approaches P along the curve. Because x approaches a as Q approaches P, we could also use Equation 1 to write

$$m = \lim_{x \to a} \frac{f(x) - f(a)}{x - a} \quad \boxed{2}$$

Specific examples of this procedure will be given in Chapter 1.

The tangent problem has given rise to the branch of calculus called *differential calculus,* which was not invented until more than 2000 years after integral calculus. The main ideas behind differential calculus are due to the French mathematician Pierre Fermat (1601–1665) and were developed by the English mathematicians John Wallis (1616–1703), Isaac Barrow (1630–1677), and Isaac Newton (1642–1727) and the German mathematician Gottfried Leibniz (1646–1716).

The two branches of calculus and their chief problems, the area problem and the tangent problem, appear to be very different, but it turns out that there is a very close connection between them. The tangent problem and the area problem are inverse problems in a sense that will be described in Chapter 4.

Velocity

When we look at the speedometer of a car and read that the car is traveling at 48 mi/h, what does that information indicate to us? We know that if the velocity remains constant, then after an hour we will have traveled 48 mi. But if the velocity of the car varies, what does it mean to say that the velocity at a given instant is 48 mi/h?

In order to analyze this question, let's examine the motion of a car that travels along a straight road and assume that we can measure the distance traveled by the car (in feet) at l-second intervals as in the following chart:

t = Time elapsed (s)	0	1	2	3	4	5
d = Distance (ft)	0	2	9	24	42	71

As a first step toward finding the velocity after 2 seconds have elapsed, we find the average velocity during the time interval $2 \leqslant t \leqslant 4$:

$$\text{average velocity} = \frac{\text{change in position}}{\text{time elapsed}}$$

$$= \frac{42 - 9}{4 - 2}$$

$$= 16.5 \text{ ft/s}$$

Similarly, the average velocity in the time interval $2 \leqslant t \leqslant 3$ is

$$\text{average velocity} = \frac{24 - 9}{3 - 2} = 15 \text{ ft/s}$$

We have the feeling that the velocity at the instant $t = 2$ can't be much different from the average velocity during a short time interval starting at $t = 2$. So let's imagine that the distance traveled has been measured at 0.l-second time intervals as in the following chart:

t	2.0	2.1	2.2	2.3	2.4	2.5
d	9.00	10.02	11.16	12.45	13.96	15.80

Then we can compute, for instance, the average velocity over the time interval $[2, 2.5]$:

$$\text{average velocity} = \frac{15.80 - 9.00}{2.5 - 2} = 13.6 \text{ ft/s}$$

The results of such calculations are shown in the following chart:

Time interval	[2, 3]	[2, 2.5]	[2, 2.4]	[2, 2.3]	[2, 2.2]	[2, 2.1]
Average velocity (ft/s)	15.0	13.6	12.4	11.5	10.8	10.2

The average velocities over successively smaller intervals appear to be getting closer to a number near 10, and so we expect that the velocity at exactly $t = 2$ is about 10 ft/s. In Chapter 2 we will define the instantaneous velocity of a moving object as the limiting value of the average velocities over smaller and smaller time intervals.

In Figure 8 we show a graphical representation of the motion of the car by plotting the distance traveled as a function of time. If we write $d = f(t)$, then $f(t)$ is the number of feet traveled after t seconds. The average velocity in the time interval $[2, t]$ is

$$\text{average velocity} = \frac{\text{change in position}}{\text{time elapsed}} = \frac{f(t) - f(2)}{t - 2}$$

which is the same as the slope of the secant line PQ in Figure 8. The velocity v when $t = 2$ is the limiting value of this average velocity as t approaches 2; that is,

$$v = \lim_{t \to 2} \frac{f(t) - f(2)}{t - 2}$$

and we recognize from Equation 2 that this is the same as the slope of the tangent line to the curve at P.

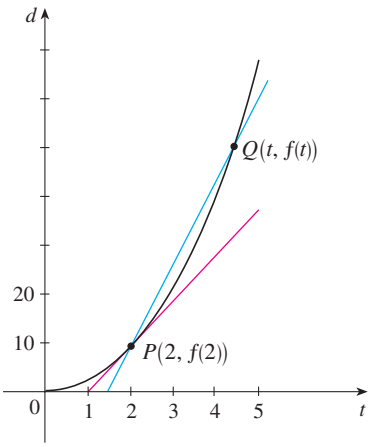


FIGURE 8

1.4 The Tangent and Velocity Problems

In this section we see how limits arise when we attempt to find the tangent to a curve or the velocity of an object.

The Tangent Problem

The word *tangent* is derived from the Latin word *tangens*, which means "touching." Thus a tangent to a curve is a line that touches the curve. In other words, a tangent line should have the same direction as the curve at the point of contact. How can this idea be made precise?

For a circle we could simply follow Euclid and say that a tangent is a line that intersects the circle once and only once, as in Figure 1(a). For more complicated curves this definition is inadequate. Figure 1(b) shows two lines l and t passing through a point P on a curve C. The line l intersects C only once, but it certainly does not look like what we think of as a tangent. The line t, on the other hand, looks like a tangent but it intersects C twice.

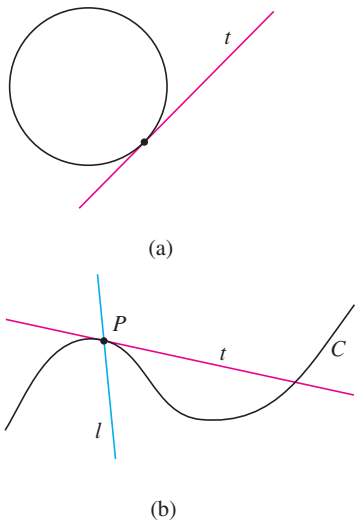


FIGURE 1

To be specific, let's look at the problem of trying to find a tangent line t to the parabola $y = x^2$ in the following example.

EXAMPLE 1 Find an equation of the tangent line to the parabola $y = x^2$ at the point $P(1, 1)$.

SOLUTION We will be able to find an equation of the tangent line t as soon as we know its slope m. The difficulty is that we know only one point, P, on t, whereas we need two points to compute the slope. But observe that we can compute an approximation to m

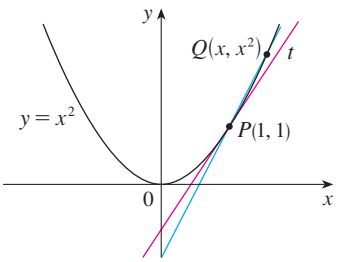


FIGURE 2

by choosing a nearby point $Q(x, x^2)$ on the parabola (as in Figure 2) and computing the slope m_{PQ} of the secant line PQ. [A **secant line**, from the Latin word *secans*, meaning cutting, is a line that cuts (intersects) a curve more than once.]

We choose $x \neq 1$ so that $Q \neq P$. Then

$$m_{PQ} = \frac{x^2 - 1}{x - 1}$$

For instance, for the point $Q(1.5, 2.25)$ we have

$$m_{PQ} = \frac{2.25 - 1}{1.5 - 1} = \frac{1.25}{0.5} = 2.5$$

x	m_{PQ}
2	3
1.5	2.5
1.1	2.1
1.01	2.01
1.001	2.001

x	m_{PQ}
0	1
0.5	1.5
0.9	1.9
0.99	1.99
0.999	1.999

The tables in the margin show the values of m_{PQ} for several values of x close to 1. The closer Q is to P, the closer x is to 1 and, it appears from the tables, the closer m_{PQ} is to 2. This suggests that the slope of the tangent line t should be $m = 2$.

We say that the slope of the tangent line is the *limit* of the slopes of the secant lines, and we express this symbolically by writing

$$\lim_{Q \to P} m_{PQ} = m \qquad \text{and} \qquad \lim_{x \to 1} \frac{x^2 - 1}{x - 1} = 2$$

Assuming that the slope of the tangent line is indeed 2, we use the point-slope form of the equation of a line [$y - y_1 = m(x - x_1)$, see Appendix B] to write the equation of the tangent line through (1, 1) as

$$y - 1 = 2(x - 1) \qquad \text{or} \qquad y = 2x - 1$$

Figure 3 illustrates the limiting process that occurs in this example. As Q approaches P along the parabola, the corresponding secant lines rotate about P and approach the tangent line t.

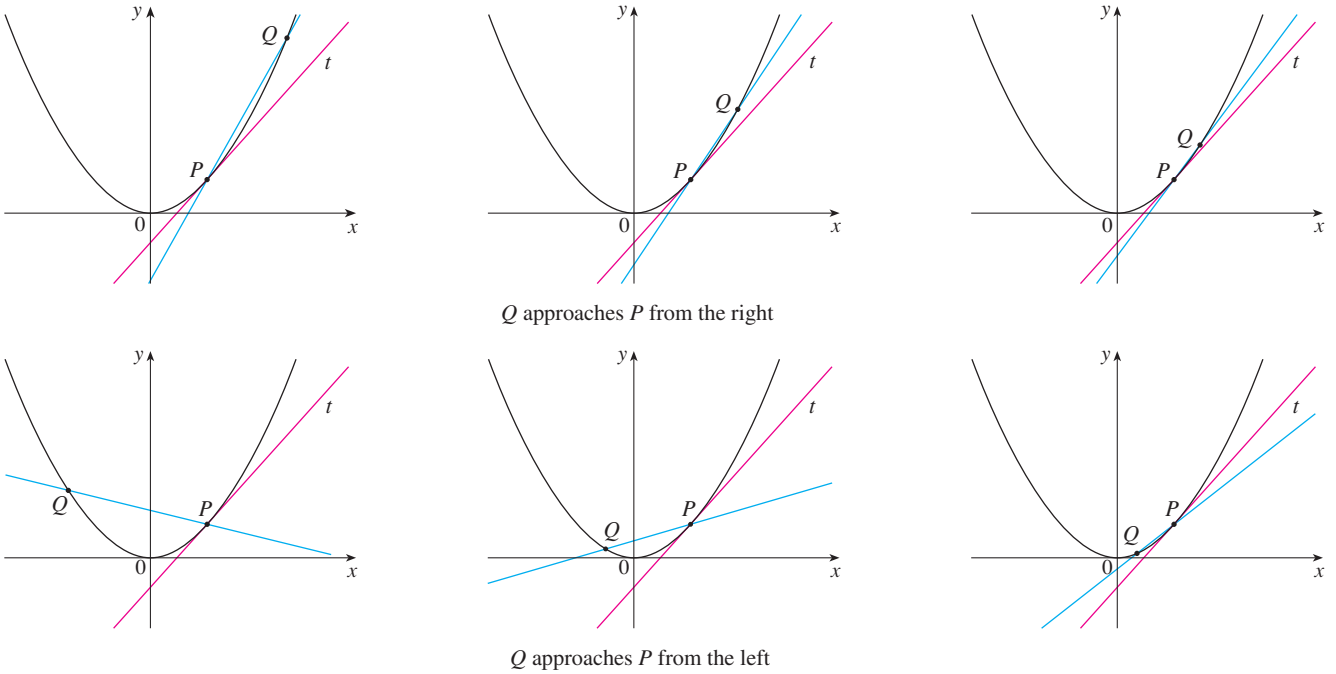


FIGURE 3

TEC In Visual 1.4 you can see how the process in Figure 3 works for additional functions.

Many functions that occur in science are not described by explicit equations; they are defined by experimental data. The next example shows how to estimate the slope of the tangent line to the graph of such a function.

EXAMPLE 2 The flash unit on a camera operates by storing charge on a capacitor and releasing it suddenly when the flash is set off. The data in the table describe the charge Q remaining on the capacitor (measured in microcoulombs) at time t (measured in seconds after the flash goes off). Use the data to draw the graph of this function and estimate the slope of the tangent line at the point where $t = 0.04$. [*Note:* The slope of the tangent line represents the electric current flowing from the capacitor to the flash bulb (measured in microamperes).]

t	Q
0.00	100.00
0.02	81.87
0.04	67.03
0.06	54.88
0.08	44.93
0.10	36.76

SOLUTION In Figure 4 we plot the given data and use them to sketch a curve that approximates the graph of the function.

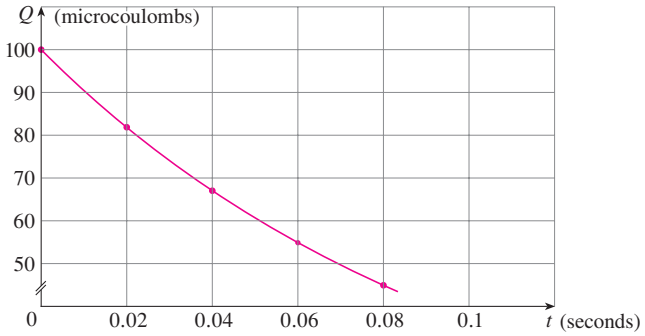


FIGURE 4

Given the points $P(0.04, 67.03)$ and $R(0.00, 100.00)$ on the graph, we find that the slope of the secant line PR is

$$m_{PR} = \frac{100.00 - 67.03}{0.00 - 0.04} = -824.25$$

R	m_{PR}
(0.00, 100.00)	−824.25
(0.02, 81.87)	−742.00
(0.06, 54.88)	−607.50
(0.08, 44.93)	−552.50
(0.10, 36.76)	−504.50

The table at the left shows the results of similar calculations for the slopes of other secant lines. From this table we would expect the slope of the tangent line at $t = 0.04$ to lie somewhere between -742 and -607.5. In fact, the average of the slopes of the two closest secant lines is

$$\tfrac{1}{2}(-742 - 607.5) = -674.75$$

So, by this method, we estimate the slope of the tangent line to be about -675.

Another method is to draw an approximation to the tangent line at P and measure the sides of the triangle ABC, as in Figure 5.

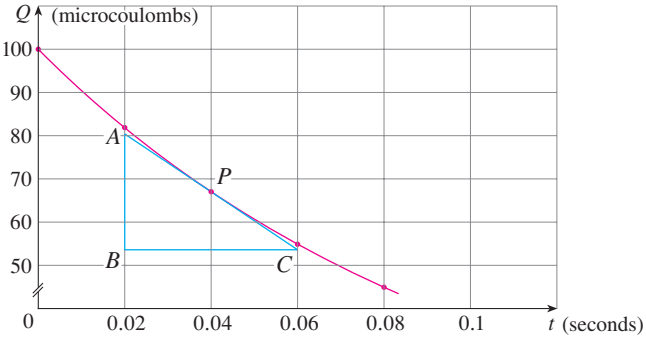


FIGURE 5

The physical meaning of the answer in Example 2 is that the electric current flowing from the capacitor to the flash bulb after 0.04 seconds is about −670 microamperes.

This gives an estimate of the slope of the tangent line as

$$-\frac{|AB|}{|BC|} \approx -\frac{80.4 - 53.6}{0.06 - 0.02} = -670$$

■

The Velocity Problem

If you watch the speedometer of a car as you travel in city traffic, you see that the speed doesn't stay the same for very long; that is, the velocity of the car is not constant. We assume from watching the speedometer that the car has a definite velocity at each moment, but how is the "instantaneous" velocity defined? Let's investigate the example of a falling ball.

Steve Allen / Stockbyte / Getty Images

The CN Tower in Toronto was the tallest freestanding building in the world for 32 years.

EXAMPLE 3 Suppose that a ball is dropped from the upper observation deck of the CN Tower in Toronto, 450 m above the ground. Find the velocity of the ball after 5 seconds.

SOLUTION Through experiments carried out four centuries ago, Galileo discovered that the distance fallen by any freely falling body is proportional to the square of the time it has been falling. (This model for free fall neglects air resistance.) If the distance fallen after t seconds is denoted by $s(t)$ and measured in meters, then Galileo's law is expressed by the equation

$$s(t) = 4.9t^2$$

The difficulty in finding the velocity after 5 seconds is that we are dealing with a single instant of time ($t = 5$), so no time interval is involved. However, we can approximate the desired quantity by computing the average velocity over the brief time interval of a tenth of a second from $t = 5$ to $t = 5.1$:

$$\text{average velocity} = \frac{\text{change in position}}{\text{time elapsed}}$$

$$= \frac{s(5.1) - s(5)}{0.1}$$

$$= \frac{4.9(5.1)^2 - 4.9(5)^2}{0.1} = 49.49 \text{ m/s}$$

The following table shows the results of similar calculations of the average velocity over successively smaller time periods.

Time interval	Average velocity (m/s)
$5 \leqslant t \leqslant 6$	53.9
$5 \leqslant t \leqslant 5.1$	49.49
$5 \leqslant t \leqslant 5.05$	49.245
$5 \leqslant t \leqslant 5.01$	49.049
$5 \leqslant t \leqslant 5.001$	49.0049

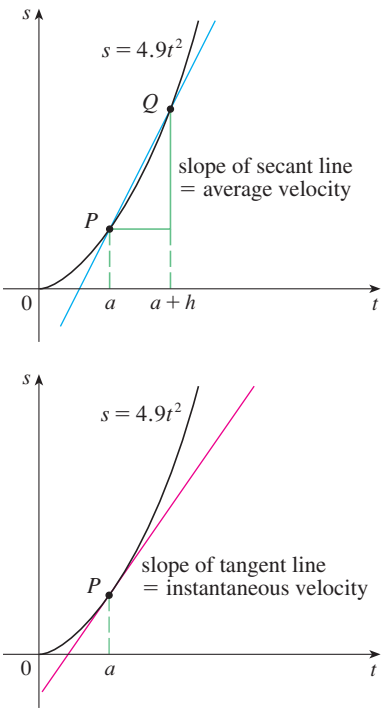


FIGURE 6

It appears that as we shorten the time period, the average velocity is becoming closer to 49 m/s. The **instantaneous velocity** when $t = 5$ is defined to be the limiting value of these average velocities over shorter and shorter time periods that start at $t = 5$. Thus it appears that the (instantaneous) velocity after 5 seconds is

$$v = 49 \text{ m/s}$$

■

You may have the feeling that the calculations used in solving this problem are very similar to those used earlier in this section to find tangents. In fact, there is a close connection between the tangent problem and the problem of finding velocities. If we draw the graph of the distance function of the ball (as in Figure 6) and we consider the points $P(a, 4.9a^2)$ and $Q(a + h, 4.9(a + h)^2)$ on the graph, then the slope of the secant line PQ is

$$m_{PQ} = \frac{4.9(a + h)^2 - 4.9a^2}{(a + h) - a}$$

which is the same as the average velocity over the time interval $[a, a + h]$. Therefore the velocity at time $t = a$ (the limit of these average velocities as h approaches 0) must be equal to the slope of the tangent line at P (the limit of the slopes of the secant lines).

Examples 1 and 3 show that in order to solve tangent and velocity problems we must be able to find limits. After studying methods for computing limits in the next four sections, we will return to the problems of finding tangents and velocities in Chapter 2.

1.5 The Limit of a Function

Having seen in the preceding section how limits arise when we want to find the tangent to a curve or the velocity of an object, we now turn our attention to limits in general and numerical and graphical methods for computing them.

Let's investigate the behavior of the function f defined by $f(x) = x^2 - x + 2$ for values of x near 2. The following table gives values of $f(x)$ for values of x close to 2 but not equal to 2.

x	$f(x)$	x	$f(x)$
1.0	2.000000	3.0	8.000000
1.5	2.750000	2.5	5.750000
1.8	3.440000	2.2	4.640000
1.9	3.710000	2.1	4.310000
1.95	3.852500	2.05	4.152500
1.99	3.970100	2.01	4.030100
1.995	3.985025	2.005	4.015025
1.999	3.997001	2.001	4.003001

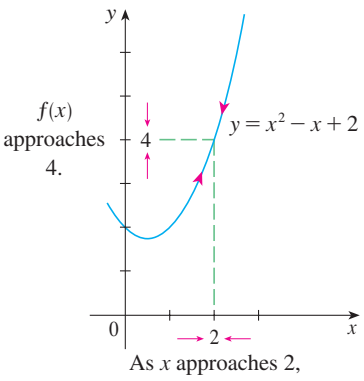


FIGURE 1

From the table and the graph of f (a parabola) shown in Figure 1 we see that the closer x is to 2 (on either side of 2), the closer $f(x)$ is to 4. In fact, it appears that we can make the values of $f(x)$ as close as we like to 4 by taking x sufficiently close to 2. We express this by saying "the limit of the function $f(x) = x^2 - x + 2$ as x approaches 2 is equal to 4." The notation for this is

$$\lim_{x \to 2} (x^2 - x + 2) = 4$$

In general, we use the following notation.

1 Intuitive Definition of a Limit Suppose $f(x)$ is defined when x is near the number a. (This means that f is defined on some open interval that contains a, except possibly at a itself.) Then we write

$$\lim_{x \to a} f(x) = L$$

and say "the limit of $f(x)$, as x approaches a, equals L"

if we can make the values of $f(x)$ arbitrarily close to L (as close to L as we like) by restricting x to be sufficiently close to a (on either side of a) but not equal to a.

Roughly speaking, this says that the values of $f(x)$ approach L as x approaches a. In other words, the values of $f(x)$ tend to get closer and closer to the number L as x gets closer and closer to the number a (from either side of a) but $x \neq a$. (A more precise definition will be given in Section 1.7.)

An alternative notation for

$$\lim_{x \to a} f(x) = L$$

is

$$f(x) \to L \quad \text{as} \quad x \to a$$

which is usually read "$f(x)$ approaches L as x approaches a."

Notice the phrase "but $x \neq a$" in the definition of limit. This means that in finding the limit of $f(x)$ as x approaches a, we never consider $x = a$. In fact, $f(x)$ need not even be defined when $x = a$. The only thing that matters is how f is defined *near* a.

Figure 2 shows the graphs of three functions. Note that in part (c), $f(a)$ is not defined and in part (b), $f(a) \neq L$. But in each case, regardless of what happens at a, it is true that $\lim_{x \to a} f(x) = L$.

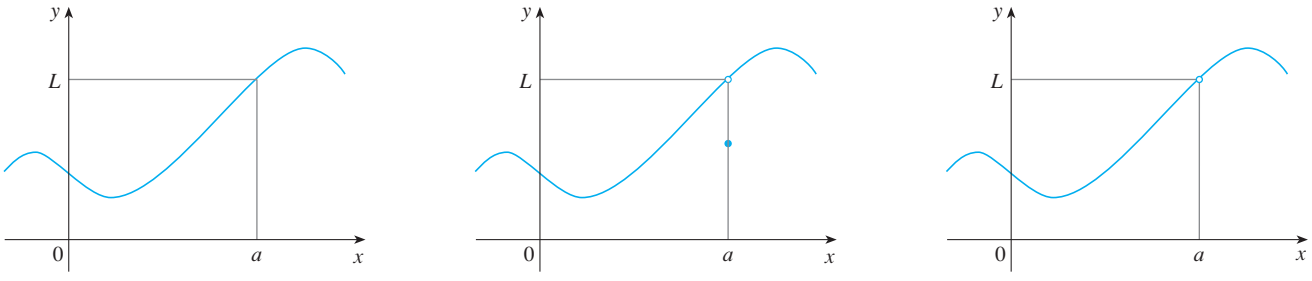


FIGURE 2 $\lim_{x \to a} f(x) = L$ in all three cases

$x < 1$	$f(x)$
0.5	0.666667
0.9	0.526316
0.99	0.502513
0.999	0.500250
0.9999	0.500025

$x > 1$	$f(x)$
1.5	0.400000
1.1	0.476190
1.01	0.497512
1.001	0.499750
1.0001	0.499975

1 0.5

EXAMPLE 1 Guess the value of $\lim_{x \to 1} \dfrac{x - 1}{x^2 - 1}$.

SOLUTION Notice that the function $f(x) = (x - 1)/(x^2 - 1)$ is not defined when $x = 1$, but that doesn't matter because the definition of $\lim_{x \to a} f(x)$ says that we consider values of x that are close to a but not equal to a.

The tables at the left give values of $f(x)$ (correct to six decimal places) for values of x that approach 1 (but are not equal to 1). On the basis of the values in the tables, we make the guess that

$$\lim_{x \to 1} \frac{x - 1}{x^2 - 1} = 0.5$$

Example 1 is illustrated by the graph of f in Figure 3. Now let's change f slightly by giving it the value 2 when $x = 1$ and calling the resulting function g:

$$g(x) = \begin{cases} \dfrac{x - 1}{x^2 - 1} & \text{if } x \neq 1 \\ 2 & \text{if } x = 1 \end{cases}$$

This new function g still has the same limit as x approaches 1. (See Figure 4.)

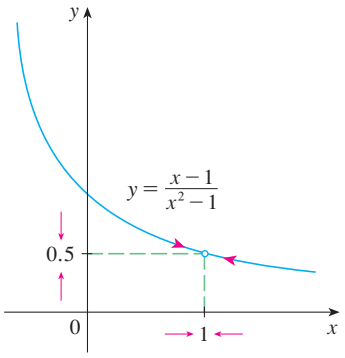


FIGURE 3

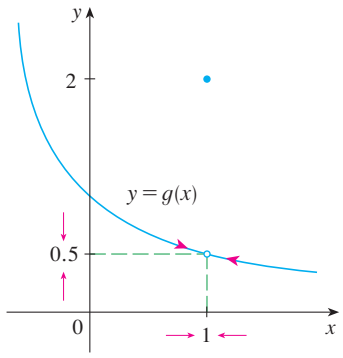


FIGURE 4

EXAMPLE 2 Estimate the value of $\lim_{t \to 0} \dfrac{\sqrt{t^2 + 9} - 3}{t^2}$.

SOLUTION The table lists values of the function for several values of t near 0.

t	$\dfrac{\sqrt{t^2 + 9} - 3}{t^2}$
± 1.0	0.162277 . . .
± 0.5	0.165525 . . .
± 0.1	0.166620 . . .
± 0.05	0.166655 . . .
± 0.01	0.166666 . . .

As t approaches 0, the values of the function seem to approach 0.1666666 . . . and so we guess that

$$\lim_{t \to 0} \frac{\sqrt{t^2 + 9} - 3}{t^2} = \frac{1}{6}$$

t	$\dfrac{\sqrt{t^2+9}-3}{t^2}$
±0.001	0.166667
±0.0001	0.166670
±0.00001	0.167000
±0.000001	0.000000

www.stewartcalculus.com
For a further explanation of why calculators sometimes give false values, click on *Lies My Calculator and Computer Told Me*. In particular, see the section called *The Perils of Subtraction*.

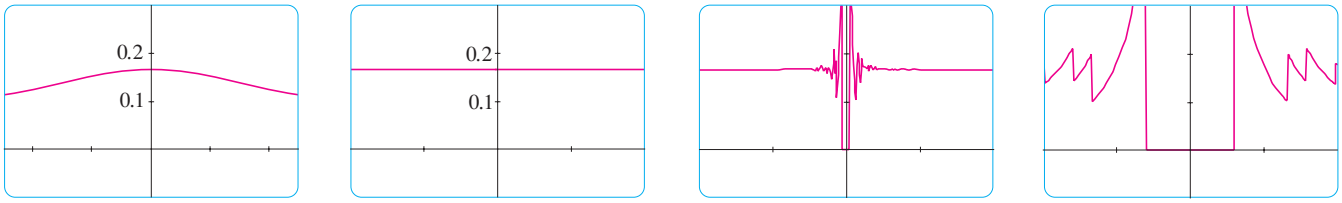
In Example 2 what would have happened if we had taken even smaller values of t? The table in the margin shows the results from one calculator; you can see that something strange seems to be happening.

If you try these calculations on your own calculator you might get different values, but eventually you will get the value 0 if you make t sufficiently small. Does this mean that the answer is really 0 instead of $\frac{1}{6}$? No, the value of the limit is $\frac{1}{6}$, as we will show in the next section. The problem is that the calculator gave false values because $\sqrt{t^2+9}$ is very close to 3 when t is small. (In fact, when t is sufficiently small, a calculator's value for $\sqrt{t^2+9}$ is 3.000... to as many digits as the calculator is capable of carrying.)

Something similar happens when we try to graph the function

$$f(t) = \frac{\sqrt{t^2+9}-3}{t^2}$$

of Example 2 on a graphing calculator or computer. Parts (a) and (b) of Figure 5 show quite accurate graphs of f, and when we use the trace mode (if available) we can estimate easily that the limit is about $\frac{1}{6}$. But if we zoom in too much, as in parts (c) and (d), then we get inaccurate graphs, again because of rounding errors from the subtraction.



(a) $-5 \leqslant t \leqslant 5$ (b) $-0.1 \leqslant t \leqslant 0.1$ (c) $-10^{-6} \leqslant t \leqslant 10^{-6}$ (d) $-10^{-7} \leqslant t \leqslant 10^{-7}$

FIGURE 5

EXAMPLE 3 Guess the value of $\lim_{x \to 0} \dfrac{\sin x}{x}$.

SOLUTION The function $f(x) = (\sin x)/x$ is not defined when $x = 0$. Using a calculator (and remembering that, if $x \in \mathbb{R}$, $\sin x$ means the sine of the angle whose *radian* measure is x), we construct a table of values correct to eight decimal places. From the table at the left and the graph in Figure 6 we guess that

x	$\dfrac{\sin x}{x}$
±1.0	0.84147098
±0.5	0.95885108
±0.4	0.97354586
±0.3	0.98506736
±0.2	0.99334665
±0.1	0.99833417
±0.05	0.99958339
±0.01	0.99998333
±0.005	0.99999583
±0.001	0.99999983

$$\lim_{x \to 0} \frac{\sin x}{x} = 1$$

This guess is in fact correct, as will be proved in Chapter 2 using a geometric argument.

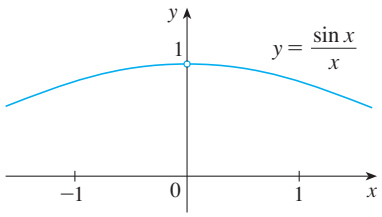


FIGURE 6

EXAMPLE 4 Investigate $\lim_{x \to 0} \sin \dfrac{\pi}{x}$.

Computer Algebra Systems
Computer algebra systems (CAS) have commands that compute limits. In order to avoid the types of pitfalls demonstrated in Examples 2, 4, and 5, they don't find limits by numerical experimentation. Instead, they use more sophisticated techniques such as computing infinite series. If you have access to a CAS, use the limit command to compute the limits in the examples of this section and to check your answers in the exercises of this chapter.

SOLUTION Again the function $f(x) = \sin(\pi/x)$ is undefined at 0. Evaluating the function for some small values of x, we get

$$f(1) = \sin \pi = 0 \qquad f\left(\tfrac{1}{2}\right) = \sin 2\pi = 0$$

$$f\left(\tfrac{1}{3}\right) = \sin 3\pi = 0 \qquad f\left(\tfrac{1}{4}\right) = \sin 4\pi = 0$$

$$f(0.1) = \sin 10\pi = 0 \qquad f(0.01) = \sin 100\pi = 0$$

Similarly, $f(0.001) = f(0.0001) = 0$. On the basis of this information we might be tempted to guess that

$$\lim_{x \to 0} \sin \frac{\pi}{x} = 0$$

but this time our guess is wrong. Note that although $f(1/n) = \sin n\pi = 0$ for any integer n, it is also true that $f(x) = 1$ for infinitely many values of x (such as $2/5$ or $2/101$) that approach 0. You can see this from the graph of f shown in Figure 7.

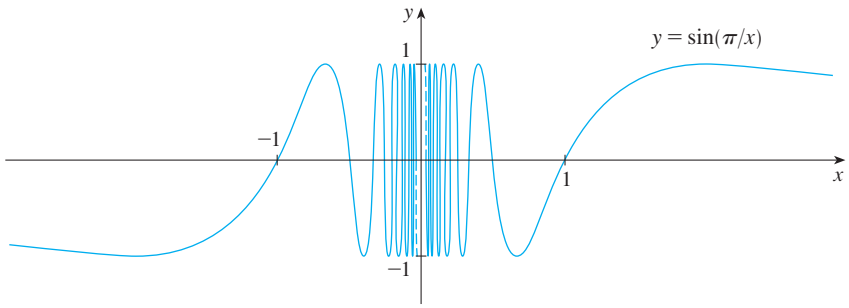


FIGURE 7

The dashed lines near the y-axis indicate that the values of $\sin(\pi/x)$ oscillate between 1 and -1 infinitely often as x approaches 0. (See Exercise 45.)

Since the values of $f(x)$ do not approach a fixed number as x approaches 0,

$$\lim_{x \to 0} \sin \frac{\pi}{x} \text{ does not exist}$$

■

x	$x^3 + \dfrac{\cos 5x}{10{,}000}$
1	1.000028
0.5	0.124920
0.1	0.001088
0.05	0.000222
0.01	0.000101

x	$x^3 + \dfrac{\cos 5x}{10{,}000}$
0.005	0.00010009
0.001	0.00010000

EXAMPLE 5 Find $\lim_{x \to 0} \left(x^3 + \dfrac{\cos 5x}{10{,}000} \right)$.

SOLUTION As before, we construct a table of values. From the first table in the margin it appears that

$$\lim_{x \to 0} \left(x^3 + \frac{\cos 5x}{10{,}000} \right) = 0$$

But if we persevere with smaller values of x, the second table suggests that

$$\lim_{x \to 0} \left(x^3 + \frac{\cos 5x}{10{,}000} \right) = 0.000100 = \frac{1}{10{,}000}$$

In Section 1.8 we will see that $\lim_{x \to 0} \cos 5x = 1$; then it follows that the limit is 0.0001. ■

Examples 4 and 5 illustrate some of the pitfalls in guessing the value of a limit. It is easy to guess the wrong value if we use inappropriate values of x, but it is difficult to

21. Find all values of a such that f is continuous on $\mathbb{R}$:

$$f(x) = \begin{cases} x + 1 & \text{if } x \leqslant a \\ x^2 & \text{if } x > a \end{cases}$$

22. A **fixed point** of a function f is a number c in its domain such that $f(c) = c$. (The function doesn't move c; it stays fixed.)

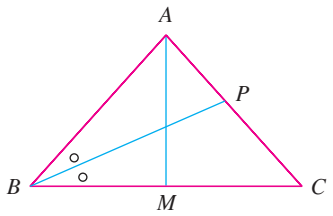
(a) Sketch the graph of a continuous function with domain $[0, 1]$ whose range also lies in $[0, 1]$. Locate a fixed point of f.

(b) Try to draw the graph of a continuous function with domain $[0, 1]$ and range in $[0, 1]$ that does *not* have a fixed point. What is the obstacle?

(c) Use the Intermediate Value Theorem to prove that any continuous function with domain $[0, 1]$ and range in $[0, 1]$ must have a fixed point.

23. If $\lim_{x \to a} [f(x) + g(x)] = 2$ and $\lim_{x \to a} [f(x) - g(x)] = 1$, find $\lim_{x \to a} [f(x)\, g(x)]$.

24. (a) The figure shows an isosceles triangle ABC with $\angle B = \angle C$. The bisector of angle B intersects the side AC at the point P. Suppose that the base BC remains fixed but the altitude $|AM|$ of the triangle approaches 0, so A approaches the midpoint M of BC. What happens to P during this process? Does it have a limiting position? If so, find it.



(b) Try to sketch the path traced out by P during this process. Then find an equation of this curve and use this equation to sketch the curve.

25. (a) If we start from 0° latitude and proceed in a westerly direction, we can let $T(x)$ denote the temperature at the point x at any given time. Assuming that T is a continuous function of x, show that at any fixed time there are at least two diametrically opposite points on the equator that have exactly the same temperature.

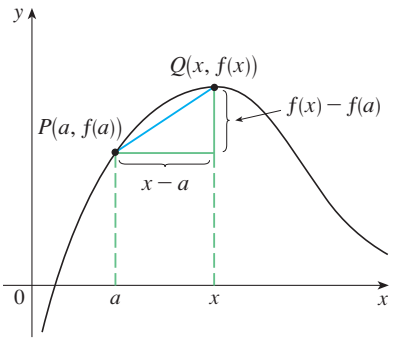
(b) Does the result in part (a) hold for points lying on any circle on the earth's surface?

(c) Does the result in part (a) hold for barometric pressure and for altitude above sea level?

2.1 Derivatives and Rates of Change

The problem of finding the tangent line to a curve and the problem of finding the velocity of an object both involve finding the same type of limit, as we saw in Section 1.4. This special type of limit is called a *derivative* and we will see that it can be interpreted as a rate of change in any of the natural or social sciences or engineering.

Tangents



If a curve C has equation $y = f(x)$ and we want to find the tangent line to C at the point $P(a, f(a))$, then we consider a nearby point $Q(x, f(x))$, where $x \neq a$, and compute the slope of the secant line PQ:

$$m_{PQ} = \frac{f(x) - f(a)}{x - a}$$

Then we let Q approach P along the curve C by letting x approach a. If m_{PQ} approaches a number m, then we define the *tangent* t to be the line through P with slope m. (This amounts to saying that the tangent line is the limiting position of the secant line PQ as Q approaches P. See Figure 1.)

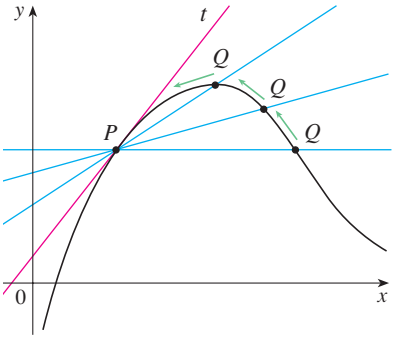


FIGURE 1

1 Definition The **tangent line** to the curve $y = f(x)$ at the point $P(a, f(a))$ is the line through P with slope

$$m = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

provided that this limit exists.

In our first example we confirm the guess we made in Example 1.4.1.

EXAMPLE 1 Find an equation of the tangent line to the parabola $y = x^2$ at the point $P(1, 1)$.

SOLUTION Here we have $a = 1$ and $f(x) = x^2$, so the slope is

$$m = \lim_{x \to 1} \frac{f(x) - f(1)}{x - 1} = \lim_{x \to 1} \frac{x^2 - 1}{x - 1}$$

$$= \lim_{x \to 1} \frac{(x - 1)(x + 1)}{x - 1}$$

$$= \lim_{x \to 1} (x + 1) = 1 + 1 = 2$$

Point-slope form for a line through the point (x_1, y_1) with slope m:

$$y - y_1 = m(x - x_1)$$

Using the point-slope form of the equation of a line, we find that an equation of the tangent line at $(1, 1)$ is

$$y - 1 = 2(x - 1) \quad \text{or} \quad y = 2x - 1$$

■

We sometimes refer to the slope of the tangent line to a curve at a point as the **slope of the curve** at the point. The idea is that if we zoom in far enough toward the point, the curve looks almost like a straight line. Figure 2 illustrates this procedure for the curve $y = x^2$ in Example 1. The more we zoom in, the more the parabola looks like a line. In other words, the curve becomes almost indistinguishable from its tangent line.

TEC Visual 2.1 shows an animation of Figure 2.

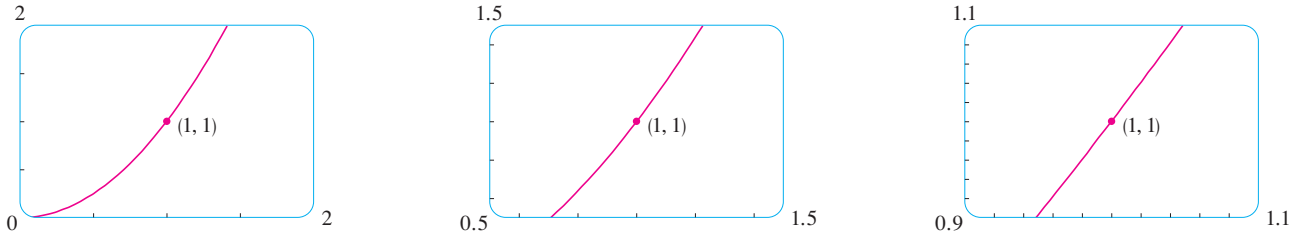


FIGURE 2 Zooming in toward the point (1, 1) on the parabola $y = x^2$

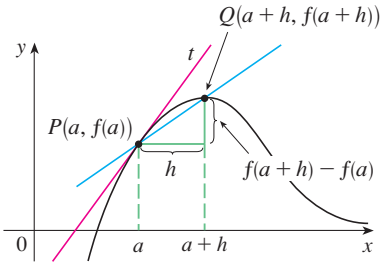


FIGURE 3

There is another expression for the slope of a tangent line that is sometimes easier to use. If $h = x - a$, then $x = a + h$ and so the slope of the secant line PQ is

$$m_{PQ} = \frac{f(a + h) - f(a)}{h}$$

(See Figure 3 where the case $h > 0$ is illustrated and Q is to the right of P. If it happened that $h < 0$, however, Q would be to the left of P.)

Notice that as x approaches a, h approaches 0 (because $h = x - a$) and so the expression for the slope of the tangent line in Definition 1 becomes

2

$$m = \lim_{h \to 0} \frac{f(a + h) - f(a)}{h}$$

EXAMPLE 2 Find an equation of the tangent line to the hyperbola $y = 3/x$ at the point (3, 1).

SOLUTION Let $f(x) = 3/x$. Then, by Equation 2, the slope of the tangent at (3, 1) is

$$\begin{aligned} m &= \lim_{h \to 0} \frac{f(3 + h) - f(3)}{h} \\ &= \lim_{h \to 0} \frac{\dfrac{3}{3 + h} - 1}{h} = \lim_{h \to 0} \frac{\dfrac{3 - (3 + h)}{3 + h}}{h} \\ &= \lim_{h \to 0} \frac{-h}{h(3 + h)} = \lim_{h \to 0} -\frac{1}{3 + h} = -\frac{1}{3} \end{aligned}$$

Therefore an equation of the tangent at the point (3, 1) is

$$y - 1 = -\tfrac{1}{3}(x - 3)$$

which simplifies to

$$x + 3y - 6 = 0$$

The hyperbola and its tangent are shown in Figure 4. ■

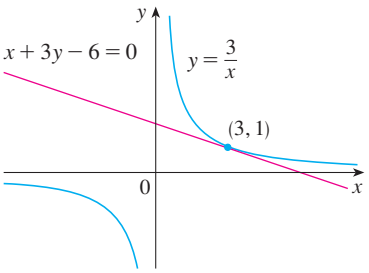


FIGURE 4

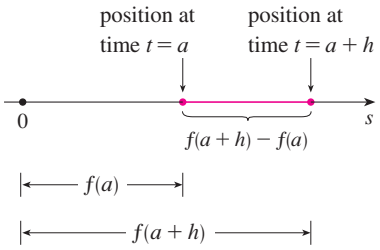
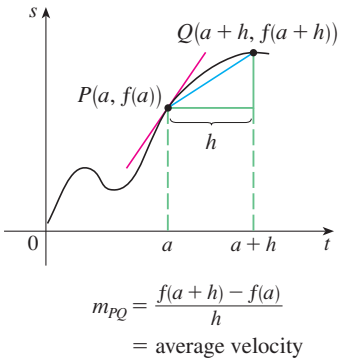


FIGURE 5



$$m_{PQ} = \frac{f(a+h) - f(a)}{h}$$
$$= \text{average velocity}$$

FIGURE 6

Velocities

In Section 1.4 we investigated the motion of a ball dropped from the CN Tower and defined its velocity to be the limiting value of average velocities over shorter and shorter time periods.

In general, suppose an object moves along a straight line according to an equation of motion $s = f(t)$, where s is the displacement (directed distance) of the object from the origin at time t. The function f that describes the motion is called the **position function** of the object. In the time interval from $t = a$ to $t = a + h$ the change in position is $f(a + h) - f(a)$. (See Figure 5.)

The average velocity over this time interval is

$$\text{average velocity} = \frac{\text{displacement}}{\text{time}} = \frac{f(a + h) - f(a)}{h}$$

which is the same as the slope of the secant line PQ in Figure 6.

Now suppose we compute the average velocities over shorter and shorter time intervals $[a, a + h]$. In other words, we let h approach 0. As in the example of the falling ball, we define the **velocity** (or **instantaneous velocity**) $v(a)$ at time $t = a$ to be the limit of these average velocities:

3
$$v(a) = \lim_{h \to 0} \frac{f(a + h) - f(a)}{h}$$

This means that the velocity at time $t = a$ is equal to the slope of the tangent line at P (compare Equations 2 and 3).

Now that we know how to compute limits, let's reconsider the problem of the falling ball.

EXAMPLE 3 Suppose that a ball is dropped from the upper observation deck of the CN Tower, 450 m above the ground.
(a) What is the velocity of the ball after 5 seconds?
(b) How fast is the ball traveling when it hits the ground?

Recall from Section 1.4: The distance (in meters) fallen after t seconds is $4.9t^2$.

SOLUTION We will need to find the velocity both when $t = 5$ and when the ball hits the ground, so it's efficient to start by finding the velocity at a general time t. Using the equation of motion $s = f(t) = 4.9t^2$, we have

$$v(t) = \lim_{h \to 0} \frac{f(t + h) - f(t)}{h} = \lim_{h \to 0} \frac{4.9(t + h)^2 - 4.9t^2}{h}$$
$$= \lim_{h \to 0} \frac{4.9(t^2 + 2th + h^2 - t^2)}{h} = \lim_{h \to 0} \frac{4.9(2th + h^2)}{h}$$
$$= \lim_{h \to 0} \frac{4.9h(2t + h)}{h} = \lim_{h \to 0} 4.9(2t + h) = 9.8t$$

(a) The velocity after 5 seconds is $v(5) = (9.8)(5) = 49$ m/s.

(b) Since the observation deck is 450 m above the ground, the ball will hit the ground at the time t when $s(t) = 450$, that is,

$$4.9t^2 = 450$$

This gives

$$t^2 = \frac{450}{4.9} \qquad \text{and} \qquad t = \sqrt{\frac{450}{4.9}} \approx 9.6 \text{ s}$$

The velocity of the ball as it hits the ground is therefore

$$v\left(\sqrt{\frac{450}{4.9}}\right) = 9.8\sqrt{\frac{450}{4.9}} \approx 94 \text{ m/s}$$

Derivatives

We have seen that the same type of limit arises in finding the slope of a tangent line (Equation 2) or the velocity of an object (Equation 3). In fact, limits of the form

$$\lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

arise whenever we calculate a rate of change in any of the sciences or engineering, such as a rate of reaction in chemistry or a marginal cost in economics. Since this type of limit occurs so widely, it is given a special name and notation.

4 Definition The **derivative of a function f at a number a**, denoted by $f'(a)$, is

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

if this limit exists.

$f'(a)$ is read "f prime of a."

If we write $x = a + h$, then we have $h = x - a$ and h approaches 0 if and only if x approaches a. Therefore an equivalent way of stating the definition of the derivative, as we saw in finding tangent lines, is

5

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

EXAMPLE 4 Find the derivative of the function $f(x) = x^2 - 8x + 9$ at the number a.

SOLUTION From Definition 4 we have

$$\begin{aligned} f'(a) &= \lim_{h \to 0} \frac{f(a+h) - f(a)}{h} \\ &= \lim_{h \to 0} \frac{[(a+h)^2 - 8(a+h) + 9] - [a^2 - 8a + 9]}{h} \\ &= \lim_{h \to 0} \frac{a^2 + 2ah + h^2 - 8a - 8h + 9 - a^2 + 8a - 9}{h} \\ &= \lim_{h \to 0} \frac{2ah + h^2 - 8h}{h} = \lim_{h \to 0} (2a + h - 8) \\ &= 2a - 8 \end{aligned}$$

Definitions 4 and 5 are equivalent, so we can use either one to compute the derivative. In practice, Definition 4 often leads to simpler computations.

We defined the tangent line to the curve $y = f(x)$ at the point $P(a, f(a))$ to be the line that passes through P and has slope m given by Equation 1 or 2. Since, by Definition 4, this is the same as the derivative $f'(a)$, we can now say the following.

> The tangent line to $y = f(x)$ at $(a, f(a))$ is the line through $(a, f(a))$ whose slope is equal to $f'(a)$, the derivative of f at a.

If we use the point-slope form of the equation of a line, we can write an equation of the tangent line to the curve $y = f(x)$ at the point $(a, f(a))$:

$$y - f(a) = f'(a)(x - a)$$

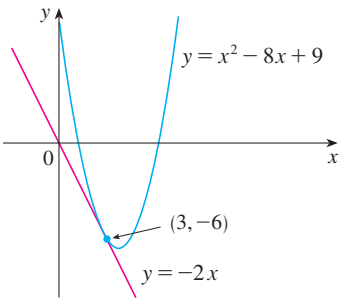


FIGURE 7

EXAMPLE 5 Find an equation of the tangent line to the parabola $y = x^2 - 8x + 9$ at the point $(3, -6)$.

SOLUTION From Example 4 we know that the derivative of $f(x) = x^2 - 8x + 9$ at the number a is $f'(a) = 2a - 8$. Therefore the slope of the tangent line at $(3, -6)$ is $f'(3) = 2(3) - 8 = -2$. Thus an equation of the tangent line, shown in Figure 7, is

$$y - (-6) = (-2)(x - 3) \qquad \text{or} \qquad y = -2x$$ ■

Rates of Change

Suppose y is a quantity that depends on another quantity x. Thus y is a function of x and we write $y = f(x)$. If x changes from x_1 to x_2, then the change in x (also called the **increment** of x) is

$$\Delta x = x_2 - x_1$$

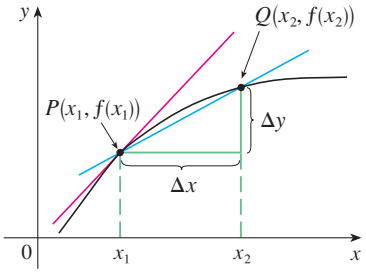
and the corresponding change in y is

$$\Delta y = f(x_2) - f(x_1)$$

The difference quotient

$$\frac{\Delta y}{\Delta x} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

is called the **average rate of change of y with respect to x** over the interval $[x_1, x_2]$ and can be interpreted as the slope of the secant line PQ in Figure 8.



average rate of change = m_{PQ}

instantaneous rate of change = slope of tangent at P

FIGURE 8

By analogy with velocity, we consider the average rate of change over smaller and smaller intervals by letting x_2 approach x_1 and therefore letting Δx approach 0. The limit of these average rates of change is called the **(instantaneous) rate of change of y with respect to x** at $x = x_1$, which (as in the case of velocity) is interpreted as the slope of the tangent to the curve $y = f(x)$ at $P(x_1, f(x_1))$:

> **6** instantaneous rate of change $= \lim_{\Delta x \to 0} \dfrac{\Delta y}{\Delta x} = \lim_{x_2 \to x_1} \dfrac{f(x_2) - f(x_1)}{x_2 - x_1}$

We recognize this limit as being the derivative $f'(x_1)$.

2.2 The Derivative as a Function

In the preceding section we considered the derivative of a function f at a fixed number a:

$$f'(a) = \lim_{h \to 0} \frac{f(a + h) - f(a)}{h} \quad \boxed{1}$$

Here we change our point of view and let the number a vary. If we replace a in Equation 1 by a variable x, we obtain

$$f'(x) = \lim_{h \to 0} \frac{f(x + h) - f(x)}{h} \quad \boxed{2}$$

Given any number x for which this limit exists, we assign to x the number $f'(x)$. So we can regard f' as a new function, called the **derivative of f** and defined by Equation 2. We know that the value of f' at x, $f'(x)$, can be interpreted geometrically as the slope of the tangent line to the graph of f at the point $(x, f(x))$.

The function f' is called the derivative of f because it has been "derived" from f by the limiting operation in Equation 2. The domain of f' is the set $\{x \mid f'(x) \text{ exists}\}$ and may be smaller than the domain of f.

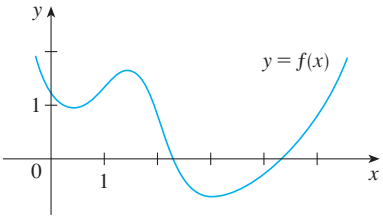


FIGURE 1

EXAMPLE 1 The graph of a function f is given in Figure 1. Use it to sketch the graph of the derivative f'.

SOLUTION We can estimate the value of the derivative at any value of x by drawing the tangent at the point $(x, f(x))$ and estimating its slope. For instance, for $x = 5$ we draw the tangent at P in Figure 2(a) and estimate its slope to be about $\frac{3}{2}$, so $f'(5) \approx 1.5$. This allows us to plot the point $P'(5, 1.5)$ on the graph of f' directly beneath P. (The slope of the graph of f becomes the y-value on the graph of f'.) Repeating this procedure at several points, we get the graph shown in Figure 2(b). Notice that the tangents at A, B, and C are horizontal, so the derivative is 0 there and the graph of f' crosses the x-axis (where $y = 0$) at the points A', B', and C', directly beneath A, B, and C. Between A and B the tangents have positive slope, so $f'(x)$ is positive there. (The graph is above the x-axis.) But between B and C the tangents have negative slope, so $f'(x)$ is negative there.

TEC Visual 2.2 shows an animation of Figure 2 for several functions.

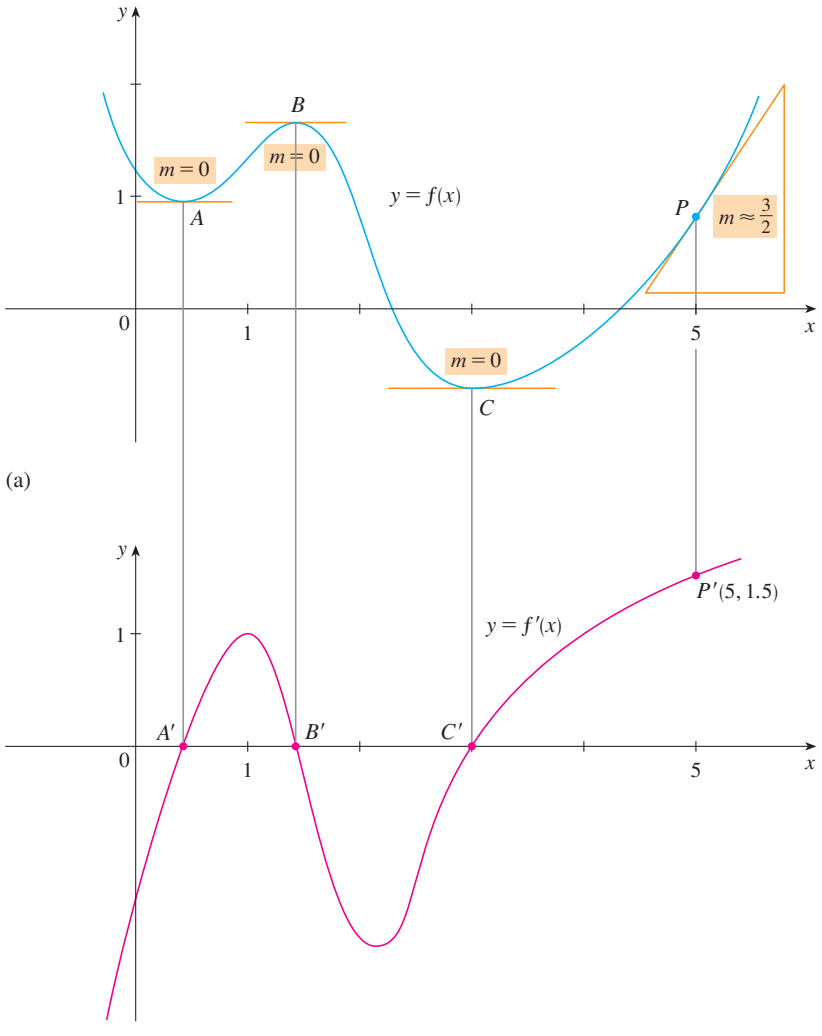


FIGURE 2

Table of Differentiation Formulas

$$\frac{d}{dx}(c) = 0 \qquad \frac{d}{dx}(x^n) = nx^{n-1}$$

$$(cf)' = cf' \qquad (f+g)' = f' + g' \qquad (f-g)' = f' - g'$$

$$(fg)' = fg' + gf' \qquad \left(\frac{f}{g}\right)' = \frac{gf' - fg'}{g^2}$$

A vector function $\mathbf{r}$ is **continuous at** a if

$$\lim_{t \to a} \mathbf{r}(t) = \mathbf{r}(a)$$

In view of Definition 1, we see that $\mathbf{r}$ is continuous at a if and only if its component functions f, g, and h are continuous at a.

Space Curves

There is a close connection between continuous vector functions and space curves. Suppose that f, g, and h are continuous real-valued functions on an interval I. Then the set C of all points (x, y, z) in space, where

$$x = f(t) \qquad y = g(t) \qquad z = h(t) \tag{2}$$

and t varies throughout the interval I, is called a **space curve**. The equations in (2) are called **parametric equations of** C and t is called a **parameter**. We can think of C as being traced out by a moving particle whose position at time t is $(f(t), g(t), h(t))$. If we now consider the vector function $\mathbf{r}(t) = \langle f(t), g(t), h(t) \rangle$, then $\mathbf{r}(t)$ is the position vector of the point $P(f(t), g(t), h(t))$ on C. Thus any continuous vector function $\mathbf{r}$ defines a space curve C that is traced out by the tip of the moving vector $\mathbf{r}(t)$, as shown in Figure 1.

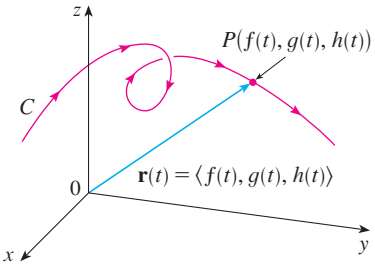


FIGURE 1
C is traced out by the tip of a moving position vector $\mathbf{r}(t)$.

EXAMPLE 3 Describe the curve defined by the vector function

$$\mathbf{r}(t) = \langle 1 + t, 2 + 5t, -1 + 6t \rangle$$

SOLUTION The corresponding parametric equations are

$$x = 1 + t \qquad y = 2 + 5t \qquad z = -1 + 6t$$

which we recognize from Equations 12.5.2 as parametric equations of a line passing through the point $(1, 2, -1)$ and parallel to the vector $\langle 1, 5, 6 \rangle$. Alternatively, we could observe that the function can be written as $\mathbf{r} = \mathbf{r}_0 + t\mathbf{v}$, where $\mathbf{r}_0 = \langle 1, 2, -1 \rangle$ and $\mathbf{v} = \langle 1, 5, 6 \rangle$, and this is the vector equation of a line as given by Equation 12.5.1. ■

TEC Visual 13.1A shows several curves being traced out by position vectors, including those in Figures 1 and 2.

Plane curves can also be represented in vector notation. For instance, the curve given by the parametric equations $x = t^2 - 2t$ and $y = t + 1$ (see Example 10.1.1) could also be described by the vector equation

$$\mathbf{r}(t) = \langle t^2 - 2t, t + 1 \rangle = (t^2 - 2t)\,\mathbf{i} + (t + 1)\,\mathbf{j}$$

where $\mathbf{i} = \langle 1, 0 \rangle$ and $\mathbf{j} = \langle 0, 1 \rangle$.

EXAMPLE 4 Sketch the curve whose vector equation is

$$\mathbf{r}(t) = \cos t\,\mathbf{i} + \sin t\,\mathbf{j} + t\,\mathbf{k}$$

SOLUTION The parametric equations for this curve are

$$x = \cos t \qquad y = \sin t \qquad z = t$$

Since $x^2 + y^2 = \cos^2 t + \sin^2 t = 1$ for all values of t, the curve must lie on the circular cylinder $x^2 + y^2 = 1$. The point (x, y, z) lies directly above the point $(x, y, 0)$, which moves counterclockwise around the circle $x^2 + y^2 = 1$ in the xy-plane. (The projection of the curve onto the xy-plane has vector equation $\mathbf{r}(t) = \langle \cos t, \sin t, 0 \rangle$. See Example 10.1.2.) Since $z = t$, the curve spirals upward around the cylinder as t increases. The curve, shown in Figure 2, is called a **helix**. ■

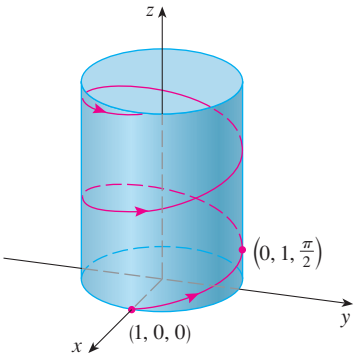


FIGURE 2

FIGURE 3
A double helix

The corkscrew shape of the helix in Example 4 is familiar from its occurrence in coiled springs. It also occurs in the model of DNA (deoxyribonucleic acid, the genetic material of living cells). In 1953 James Watson and Francis Crick showed that the structure of the DNA molecule is that of two linked, parallel helixes that are intertwined as in Figure 3.

In Examples 3 and 4 we were given vector equations of curves and asked for a geometric description or sketch. In the next two examples we are given a geometric description of a curve and are asked to find parametric equations for the curve.

EXAMPLE 5 Find a vector equation and parametric equations for the line segment that joins the point $P(1, 3, -2)$ to the point $Q(2, -1, 3)$.

SOLUTION In Section 12.5 we found a vector equation for the line segment that joins the tip of the vector $\mathbf{r}_0$ to the tip of the vector $\mathbf{r}_1$:

$$\mathbf{r}(t) = (1 - t)\mathbf{r}_0 + t\mathbf{r}_1 \qquad 0 \leqslant t \leqslant 1$$

(See Equation 12.5.4.) Here we take $\mathbf{r}_0 = \langle 1, 3, -2\rangle$ and $\mathbf{r}_1 = \langle 2, -1, 3\rangle$ to obtain a vector equation of the line segment from P to Q:

$$\mathbf{r}(t) = (1 - t)\langle 1, 3, -2\rangle + t\langle 2, -1, 3\rangle \qquad 0 \leqslant t \leqslant 1$$

or

$$\mathbf{r}(t) = \langle 1 + t, 3 - 4t, -2 + 5t\rangle \qquad 0 \leqslant t \leqslant 1$$

The corresponding parametric equations are

$$x = 1 + t \qquad y = 3 - 4t \qquad z = -2 + 5t \qquad 0 \leqslant t \leqslant 1$$ ■

Figure 4 shows the line segment PQ in Example 5.

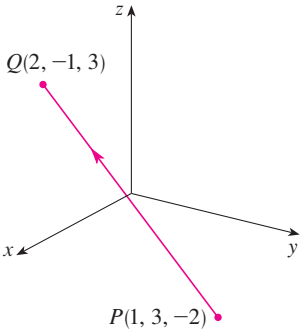


FIGURE 4

EXAMPLE 6 Find a vector function that represents the curve of intersection of the cylinder $x^2 + y^2 = 1$ and the plane $y + z = 2$.

SOLUTION Figure 5 shows how the plane and the cylinder intersect, and Figure 6 shows the curve of intersection C, which is an ellipse.

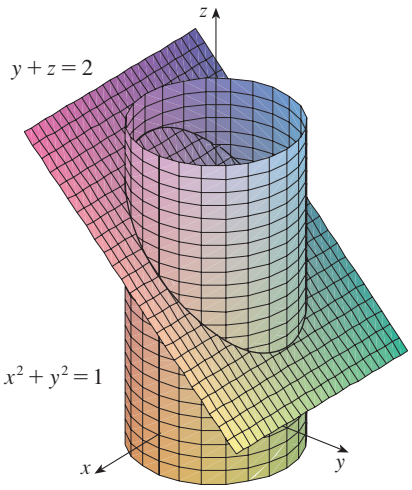


FIGURE 5

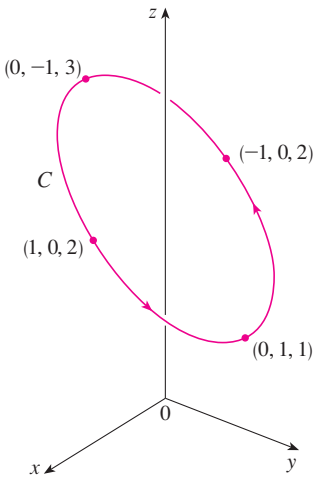


FIGURE 6

Later in this chapter we are going to use vector functions to describe the motion of planets and other objects through space. Here we prepare the way by developing the calculus of vector functions.

Derivatives

The derivative $\mathbf{r}'$ of a vector function $\mathbf{r}$ is defined in much the same way as for real-valued functions:

1

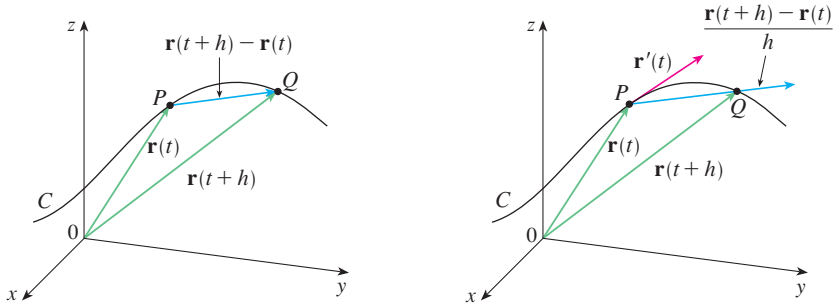
$$\frac{d\mathbf{r}}{dt} = \mathbf{r}'(t) = \lim_{h \to 0} \frac{\mathbf{r}(t+h) - \mathbf{r}(t)}{h}$$

if this limit exists. The geometric significance of this definition is shown in Figure 1. If the points P and Q have position vectors $\mathbf{r}(t)$ and $\mathbf{r}(t+h)$, then $\overrightarrow{PQ}$ represents the vector $\mathbf{r}(t+h) - \mathbf{r}(t)$, which can therefore be regarded as a secant vector. If $h > 0$, the scalar multiple $(1/h)(\mathbf{r}(t+h) - \mathbf{r}(t))$ has the same direction as $\mathbf{r}(t+h) - \mathbf{r}(t)$. As $h \to 0$, it appears that this vector approaches a vector that lies on the tangent line. For this reason, the vector $\mathbf{r}'(t)$ is called the **tangent vector** to the curve defined by $\mathbf{r}$ at the point P, provided that $\mathbf{r}'(t)$ exists and $\mathbf{r}'(t) \neq \mathbf{0}$. The **tangent line** to C at P is defined to be the line through P parallel to the tangent vector $\mathbf{r}'(t)$. We will also have occasion to consider the **unit tangent vector**, which is

Notice that when $0 < h < 1$, multiplying the secant vector by $1/h$ *stretches* the vector, as shown in Figure 1(b).

$$\mathbf{T}(t) = \frac{\mathbf{r}'(t)}{|\mathbf{r}'(t)|}$$

TEC Visual 13.2 shows an animation of Figure 1.



(a) The secant vector $\overrightarrow{PQ}$ (b) The tangent vector $\mathbf{r}'(t)$

FIGURE 1

The following theorem gives us a convenient method for computing the derivative of a vector function $\mathbf{r}$: just differentiate each component of $\mathbf{r}$.

2 Theorem If $\mathbf{r}(t) = \langle f(t), g(t), h(t) \rangle = f(t)\,\mathbf{i} + g(t)\,\mathbf{j} + h(t)\,\mathbf{k}$, where f, g, and h are differentiable functions, then

$$\mathbf{r}'(t) = \langle f'(t), g'(t), h'(t) \rangle = f'(t)\,\mathbf{i} + g'(t)\,\mathbf{j} + h'(t)\,\mathbf{k}$$

PROOF

$$\begin{aligned}
\mathbf{r}'(t) &= \lim_{\Delta t \to 0} \frac{1}{\Delta t} [\mathbf{r}(t + \Delta t) - \mathbf{r}(t)] \\
&= \lim_{\Delta t \to 0} \frac{1}{\Delta t} [\langle f(t + \Delta t), g(t + \Delta t), h(t + \Delta t) \rangle - \langle f(t), g(t), h(t) \rangle] \\
&= \lim_{\Delta t \to 0} \left\langle \frac{f(t + \Delta t) - f(t)}{\Delta t}, \frac{g(t + \Delta t) - g(t)}{\Delta t}, \frac{h(t + \Delta t) - h(t)}{\Delta t} \right\rangle \\
&= \left\langle \lim_{\Delta t \to 0} \frac{f(t + \Delta t) - f(t)}{\Delta t}, \lim_{\Delta t \to 0} \frac{g(t + \Delta t) - g(t)}{\Delta t}, \lim_{\Delta t \to 0} \frac{h(t + \Delta t) - h(t)}{\Delta t} \right\rangle \\
&= \langle f'(t), g'(t), h'(t) \rangle
\end{aligned}$$

■

Differentiation Rules

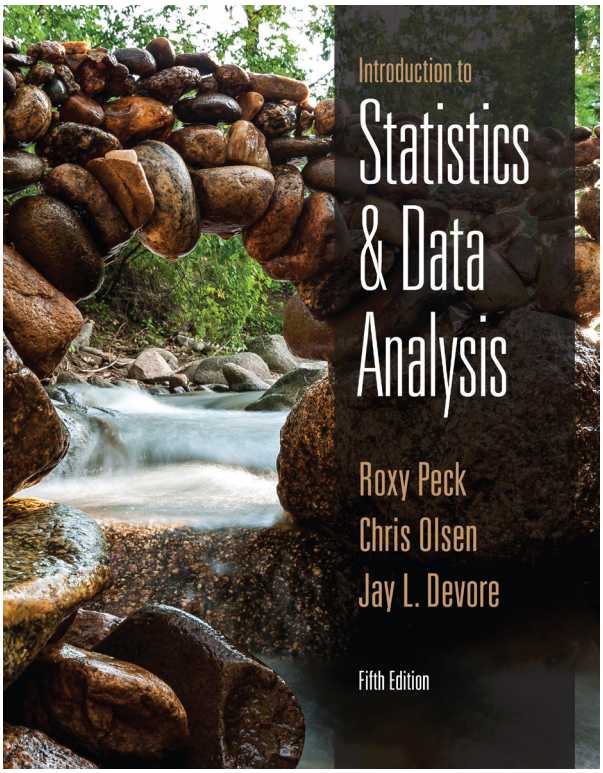
The next theorem shows that the differentiation formulas for real-valued functions have their counterparts for vector-valued functions.

3 Theorem Suppose $\mathbf{u}$ and $\mathbf{v}$ are differentiable vector functions, c is a scalar, and f is a real-valued function. Then

1. $\dfrac{d}{dt}[\mathbf{u}(t) + \mathbf{v}(t)] = \mathbf{u}'(t) + \mathbf{v}'(t)$
2. $\dfrac{d}{dt}[c\mathbf{u}(t)] = c\mathbf{u}'(t)$
3. $\dfrac{d}{dt}[f(t)\,\mathbf{u}(t)] = f'(t)\,\mathbf{u}(t) + f(t)\,\mathbf{u}'(t)$
4. $\dfrac{d}{dt}[\mathbf{u}(t) \cdot \mathbf{v}(t)] = \mathbf{u}'(t) \cdot \mathbf{v}(t) + \mathbf{u}(t) \cdot \mathbf{v}'(t)$
5. $\dfrac{d}{dt}[\mathbf{u}(t) \times \mathbf{v}(t)] = \mathbf{u}'(t) \times \mathbf{v}(t) + \mathbf{u}(t) \times \mathbf{v}'(t)$
6. $\dfrac{d}{dt}[\mathbf{u}(f(t))] = f'(t)\,\mathbf{u}'(f(t))$ (Chain Rule)

Introduction to statistics and data analysis

Roxy Peck, Chris Olsen, James L. Devore



6.1 Chance Experiments and Events

The basic ideas and terminology of probability are most easily introduced in situations that are both familiar and reasonably simple. Because of this, some of our initial examples involve such elementary activities as tossing a coin, selecting cards from a deck, and rolling a die. However, after considering a few of these simplistic examples, we will move on to more interesting and realistic situations.

Chance Experiments

When a single coin is tossed, there are two possible outcomes. The coin can land with its heads side up or its tails side up. The selection of a single card from a well-mixed standard deck may result in the ace of spades, the five of diamonds, or any one of the other 50 possibilities. Consider rolling both a red die and a green die. One possible outcome is the red die lands with four dots facing up and the green die shows one dot on its upturned face. Another outcome is the red die lands with three dots facing up and the green die also shows three dots on its upturned face. There are 36 possible outcomes in all, and we do not know in advance what the result of a particular roll will be. Situations such as these are referred to as **chance experiments**.

DEFINITION

Chance experiment: Any activity or situation in which there is uncertainty about which of two or more possible outcomes will result.

When the term chance experiment is used in a probability setting, we mean something different from what was meant by the term experiment in Chapter 2 (where experiments investigated the effect of two or more treatments on a response). For example, in an opinion poll or survey, there is uncertainty about whether an individual selected at random from the population of interest supports a school bond, and when a die is rolled, there is uncertainty about which face will land upturned. Both of these situations fit the definition of a *chance* experiment.

Consider a chance experiment to investigate whether men or women are more likely to choose a hybrid car over a traditional internal combustion engine car when purchasing a Honda Civic at a particular car dealership. The Honda Civic is available with either a hybrid or a traditional engine. In this chance experiment, a customer will be selected at random from those who purchased a Honda Civic. The type of vehicle purchased (hybrid or traditional) will be determined and the customer's gender will be recorded. Before the customer is selected the outcome of this chance experiment is unknown to us. We do know, however, what the possible outcomes are. This set of possible outcomes is called the **sample space**.

DEFINITION

Sample space: The collection of all possible outcomes of a chance experiment.

The sample space of a chance experiment can be represented in many ways. One representation is a simple list of all the possible outcomes. For the car-purchase chance experiment, the possible outcomes are:

1. A male buying hybrid
2. A female buying hybrid
3. A male buying traditional
4. A female buying traditional

We can also use set notation and ordered pairs. A male purchasing a hybrid is represented as (male, hybrid). The sample space is then

$$\text{sample space} = \begin{Bmatrix} (\text{male, hybrid}), (\text{female, hybrid}), \\ (\text{male, traditional}), (\text{female, traditional}) \end{Bmatrix}$$

Another useful representation of the sample space is a tree diagram. A tree diagram (shown in Figure 6.1) for the outcomes of the car-purchase chance experiment has two sets of branches corresponding to the two pieces of information that were gathered. To identify any particular outcome in the sample space, traverse the tree by first selecting a branch corresponding to gender and then a branch identified with a type of car.

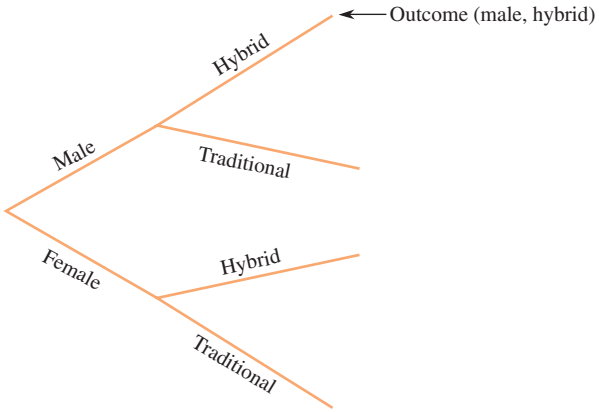


FIGURE 6.1
Tree diagram for the car purchase example.

In the tree diagram of Figure 6.1, there is no particular reason for having the gender as the first generation branch and the type of car as the second generation branch. Some chance experiments involve making observations in a particular order, in which case the order of the branches in the tree does matter. In this example, however, it would be acceptable to represent the sample space with the tree diagram shown in Figure 6.2.

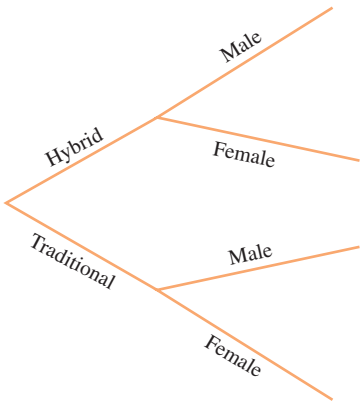


FIGURE 6.2
Another tree diagram for the car purchase example.

As we have seen, the sample space for a chance experiment can be represented in several ways, but the representations all have one thing in common: *Every* possible outcome of the chance experiment is included in the representation.

Events

In the car-purchase chance experiment, we might be interested in which particular outcome will result. Or we might focus on a group of outcomes that involve the purchase of a hybrid—the group consisting of (male, hyrid) and (female, hybrid). When we form

a collection of one or more individual outcomes, we are creating what is known as an *event*.

DEFINITION

Event: Any collection of outcomes from the sample space of a chance experiment.

Simple event: An event consisting of exactly one outcome.

We usually represent an event by an uppercase letter, such as A, B, C, and so on. On some occasions, the same letter with different numerical subscripts, such as E_1, E_2, E_3, … may also be used for this purpose.

EXAMPLE 6.1 Car Preferences

Reconsider the situation in which a person who purchased a Honda Civic was categorized by gender (M or F) and type of car purchased (H = hybrid, T = traditional). Using this notation, one possible representation of the sample space is

sample space = {MH, FH, MT, FT}

Because there are four outcomes, there are four simple events:

E_1 = MH E_2 = FH E_3 = MT E_4 = FT

One event of interest might be the event consisting of all outcomes for which a hybrid was purchased. A symbolic description of the event *hybrid* is

hybrid = {MH, FH}

Another event is the event that the purchaser is male,

male = {MH, MT} ■

EXAMPLE 6.2 Video Game

Suppose you believe that after losing to an opponent in a video game, a player is more likely to lose the next game. You conduct a chance experiment that consists of watching two consecutive games for a particular player and observing whether the player won, tied, or lost each of the two games. In this case (using W, T, and L to represent win, tie, and loss, respectively), the sample space can be represented as

sample space = {WW, WT, WL, TW, TT, TL, LW, LT, LL}

The event *lose exactly one of the two games*, denoted by L_1, could then be defined as

L_1 = {WL, TL, LW, LT} ■

Only one outcome (one simple event) occurs when a chance experiment is performed. We say that a given event occurs whenever one of the outcomes making up the event occurs. For example, if the outcome in the car purchase example is MH, then the simple event *male purchasing a hybrid* has occurred, and so has the nonsimple event *hybrid purchased*.

Forming New Events

Once some events have been specified, they can be manipulated in several useful ways to create new events.

DEFINITION

Let A and B denote two events.

***Not A*:** The event that consists of all experimental outcomes that are not in event A. *Not A* is sometimes called the *complement* of A and is usually denoted by $\boldsymbol{A^c}$, $\boldsymbol{A'}$, **or** $\boldsymbol{\overline{A}}$.

***A or B*:** The event that consists of all experimental outcomes that are in at least one of the two events, that is, in A or in B or in both of these. *A or B* is called the *union* of the two events and is denoted by $\boldsymbol{A \cup B}$.

***A and B*:** The event that consists of all experimental outcomes that are in both of the events A and B. *A and B* is called the *intersection* of the two events and is denoted by $\boldsymbol{A \cap B}$.

EXAMPLE 6.3 Turning Directions

Grant Faint/The Image Bank/Getty Images

A traffic engineer has been asked to consider whether a stop sign at the bottom of a freeway off-ramp should be replaced by a traffic light. To help in this decision, she plans to observe traffic patterns for this off-ramp. Suppose she were to record the turning direction (L = left or R = right) of each of three successive vehicles. This is a chance experiment and the sample space contains eight outcomes:

all 3 cars turn left → {LLL, RLL, LRL, LLR, RRL, RLR, LRR, RRR} ← *the 1st car turns left and the next 2 turn right*

Each of these outcomes determines a simple event. Other events include

A = event that exactly one of the cars turns right = {RLL, LRL, LLR}
B = event that at most one of the cars turns right = {LLL, RLL, LRL, LLR}
C = event that all cars turn in the same direction = {LLL, RRR}

Some other events that can be formed from those just defined are

$$\begin{aligned} \textit{not } C = C^C &= \text{event that not all cars turn in the same direction} \\ &= \{\text{RLL, LRL, LLR, RRL, RLR, LRR}\} \\ A \textit{ or } C = A \cup C &= \text{event that exactly one of the cars turns right or all cars turn in the same direction} \\ &= \{\text{RLL, LRL, LLR, LLL, RRR}\} \\ B \textit{ and } C = B \cap C &= \text{event that at most one car turns right and all cars turn in the same direction} \\ &= \{\text{LLL}\} \end{aligned}$$

■

EXAMPLE 6.4 More on Video Games

In Example 6.2, in addition to the event that a video game player loses exactly one of the two games, L_1 = {WL, TL, LW, LT}, we could also define events corresponding to L_0 = neither game is lost and L_2 = both games are lost. Then L_0 = {WW, WT, TW, TT} and L_2 = {LL}.

Some other events that can be formed from those just defined include

$$\begin{aligned} L_2^C &= \text{event that at most one game was lost} \\ &= \{\text{WW, WT, WL, TW, TT, TL, LW, LT}\} \\ L_1 \cup L_2 &= \text{event that at least one game was lost} \\ &= \{\text{WL, TL, LW, LT, LL}\} \\ L_1 \cap L_2 &= \text{event that exactly one game was lost and two games were lost} \\ &= \text{the empty set} \end{aligned}$$

■

It frequently happens that two events have no common outcomes, as was the case for events L_1 and L_2 in the video game example. Such situations are described using special terminology.

DEFINITION

Mutually exclusive: Two events are **mutually exclusive** if they have no outcomes in common. The term **disjoint** is also sometimes used to describe events that have no outcomes in common.

Saying that two events are mutually exclusive means that they can't both occur when the chance experiment is performed once.

It is sometimes useful to draw an informal picture of events to visualize relationships. In a **Venn diagram**, the collection of all possible outcomes is typically shown as the interior of a rectangle. Other events are then identified by specified regions inside this rectangle. Figure 6.3 illustrates several Venn diagrams.

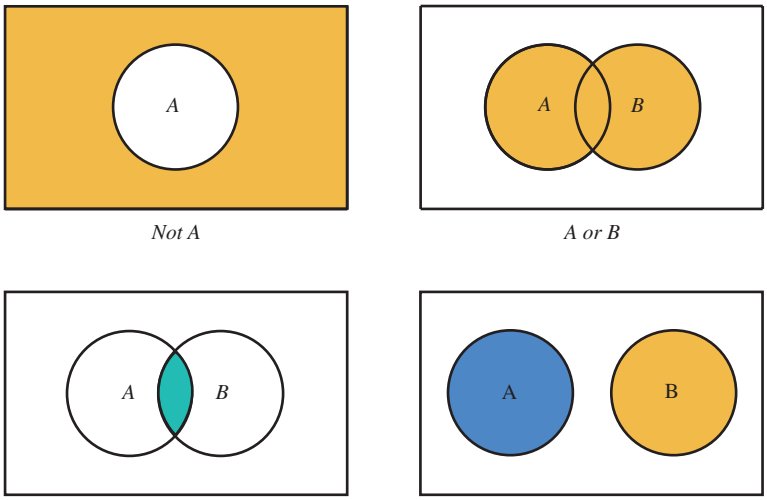


FIGURE 6.3
Venn diagrams.

The use of the *or* and *and* operations can be extended to form new events from more than two initially specified events (as illustrated in Figure 6.4).

Let $A_1, A_2, \ldots, A_k$ denote k events.

1. The event A_1 *or* A_2 *or* … *or* A_k consists of all outcomes in at least one of the individual events $A_1, A_2, \ldots, A_k$.
2. The event A_1 *and* A_2 *and* … *and* A_k consists of all outcomes that are simultaneously in every one of the individual events $A_1, A_2, \ldots, A_k$.

These k events are mutually exclusive if no two of them have any common outcomes.

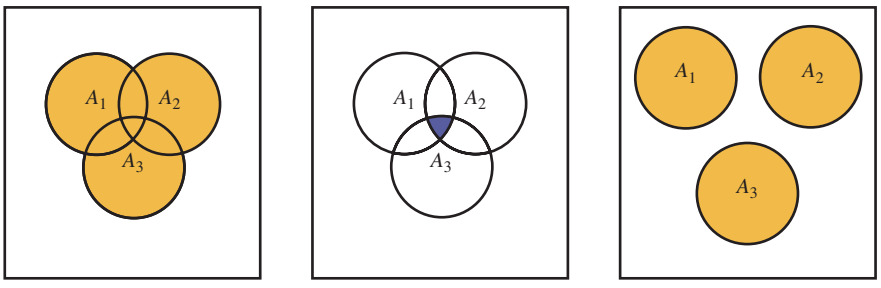


FIGURE 6.4
Venn diagrams with three events.

EXAMPLE 6.5 Asking Questions

The instructor in a seminar class consisting of four students has an unusual way of asking questions. Four slips of paper numbered 1, 2, 3, and 4 are placed in a box. The instructor determines the student to whom any particular question is to be addressed by selecting one of these four slips. Suppose that one question is to be posed during each of the next two class meetings. One possible outcome could be represented as (3, 1)—the first question is addressed to Student 3 and the second question to Student 1. There are 15 other possibilities. Consider the following events:

the event that the same student is asked both questions → $A = \{(1,1), (2,2), (3,3), (4,4)\}$

the event that Student 1 is asked at least one of the two questions → $B = \{(1,1), (1,2), (1,3), (1,4), (2,1), (3,1), (4,1)\}$

$C = \{(3,1), (2,2), (1,3)\}$

$D = \{(3,3), (3,4), (4,3)\}$

$E = \{(1,1), (1,3), (2,2), (3,1), (4,2), (3,3), (2,4), (4,4)\}$

$F = \{(1,1), (1,2), (2,1)\}$

Then

$$A \text{ or } C \text{ or } D = \{(1,1), (2,2), (3,3), (4,4), (3,1), (1,3), (3,4), (4,3)\}$$

The outcome (3,1) is contained in each of the events B, C, and E, as is the outcome (1,3). These are the only two common outcomes in all three events, so

$$B \text{ and } C \text{ and } E = \{(3,1), (1,3)\}$$

The events C, D, and F are mutually exclusive because no outcome in any one of these events is contained in either of the other two events. ■

6.2 Definition of Probability

Reasoning about games of chance and probabilistic thinking share a long history. Archeologists have found evidence that Egyptians used a small bone in mammals, the astragalus, as a sort of four-sided die as early as 3500 B.C. Games of chance were common in Greek and Roman times and during the Renaissance of Western Europe.

The formal study of probability began with the correspondence between Blaise Pascal (1623–62) and Pierre de Fermat (1601–65) in 1654. Their letters discussed some problems related to gambling that were posed by the French nobleman Chevalier de Mère. The methods and solutions that resulted from this exchange greatly enhanced the study of probability.

From this early work, new interpretations of probability have evolved, including an empirical approach preferred by many statisticians today. We begin our discussion of probability with a look at some of the different approaches to probability: the classical, relative frequency, and subjective approaches.

Classical Approach to Probability

Early mathematicians' development of the theory of probability was primarily in the context of gambling and reflected the peculiarities of games of chance. A characteristic common to most games of chance is a physical device used to generate different outcomes. In children's games, dice or a "spinner" might be used to create chance outcomes.

Dice are constructed so that the physical characteristics of each face are alike (except, of course, for the number of dots), ensuring that the different outcomes for an individual die are very close to equally likely. Therefore, it seems quite natural to assume, for example, that the probability of getting a five when a single six-sided die is rolled is one-sixth (1 chance in 6). In general, if there are N *equally likely* outcomes in a game of chance, the probability of each of the outcomes is $1/N$.

Classical Approach to Probability for Equally Likely Outcomes

When the outcomes in the sample space of a chance experiment are equally likely, the **probability of an event E,** denoted by $P(E)$**,** is the ratio of the number of outcomes favorable to E to the total number of outcomes in the sample space:

$$P(E) = \frac{\text{number of outcomes favorable to } E}{\text{number of outcomes in the sample space}}$$

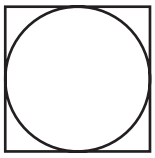
According to this definition, the calculation of a probability consists of counting the number of outcomes that make up an event, counting the number of outcomes in the sample space, and then dividing.

Chance experiments that involve tossing fair coins, rolling fair dice, or selecting cards from a well-mixed deck have equally likely outcomes. For example, if a fair die is rolled once, each outcome (simple event) has probability 1/6. With E denoting the event that the number rolled is even, $P(E) = 3/6$. This is just the number of outcomes in E divided by the total number of possible outcomes.

Limitations of the Classical Approach to Probability

The classical approach to probability works well with games of chance or other situations with a finite set of outcomes that can be regarded as equally likely. However, some situations that are clearly probabilistic in nature do not fit the classical model. Consider the accident rates of young adults driving cars. Information of this kind is used by insurance companies to set car insurance rates. Suppose that we have two individuals, one 18 years old and one 28 years old. A person purchasing a standard accident insurance policy at age 28 should have a lower annual premium than a person purchasing the same policy at age 18, because the 28-year-old has a much lower chance of an accident, based on prior experience.

However, the classical probabilist, playing by the classical rules, would have to consider that there are two outcomes for a policy year—having one or more accidents or not having an accident—*and would consider those outcomes equally likely for both the 28-year-old and the 18-year-old*. This certainly seems to defy reason and experience.



Another example of a situation that cannot be handled using the classical approach has a geometric flavor. Suppose that you have a square that is 8 inches on each side and that a circle with a diameter of 8 inches is drawn inside the square, touching the square on all four sides, as shown.

This drawing will be used as a target in a dart game. A dart will be thrown at the target and any dart that completely misses the square is a "do-over" and will be thrown again. It seems reasonable to think about the probability of a dart landing in the circle as a ratio of the area of the circle to the area of the square. That ratio is

$$\frac{\text{area of circle}}{\text{area of square}} = \frac{\pi r^2}{s^2} = \frac{\pi(4)^2}{8^2} = \frac{\pi}{4} = .785$$

A practiced dart player who is aiming for the center of the target might have an even greater probability of hitting the inside of the circle. In any case, it is not reasonable to think that the two outcomes *inside the circle* and *not inside the circle* are equally likely, so the classical approach to probability will not help here.

From the standpoint of statistics, a major limitation of the classical approach to probability is that there seems to be no way to allow past experience to inform our expectation of the future. To return to the car insurance example, suppose that in the experience of the insurance companies, 0.5% of 18-year-olds have accidents during their 19th year. It seems reasonable that, all things being equal, we could expect this percentage to be stable enough to set the prices for an insurance policy. Although the classical approach's assumption of equal probabilities is based on and consistent with the experience with dice, it cannot accommodate using an observed proportion in real life to estimate a probability. For a statistician, this is a problem. Without an understanding of probability that allows such a generalization, statistical inference would be impossible.

Relative Frequency Approach to Probability

Early scientific investigators were aware that chance experiments do not always give the same results when repeated. However, even though it is not possible to predict the outcome of a chance experiment in any particular instance, there is a dependable and stable regularity when a chance experiment is repeated many times. This became the basis for fair wagering in games of chance.

For example, suppose that two friends, Chris and Jay, meet to play a game. A fair coin is flipped. If it lands heads up, Chris pays Jay $1; otherwise, Jay pays the same amount to Chris. After many repetitions, the proportion of the time that Chris wins will be close to one-half, and the two friends will have simply enjoyed the pleasure of each other's company for a few hours. This happy circumstance occurs because *in the long run* (i.e., over many repetitions) the proportion of heads is .5. In the long run, half the time Chris wins, and half the time Jay wins.

When any given chance experiment is performed, some events are relatively likely to occur, whereas others are not as likely to occur. For a specified event E, we want to assign a

probability that gives a precise indication of how likely it is that E will occur. In the relative frequency approach to probability, the probability describes how frequently E occurs when the chance experiment is performed many times.

EXAMPLE 6.8 Tossing a Coin

Frequently, we hear a coin described as fair, or we are told that there is a 50% chance of a coin landing heads up. What is the meaning of the expression *fair*? It cannot refer to the result of a single toss, because a single toss cannot result in both a head and a tail. Might "fairness" and "50%" refer to 10 successive tosses yielding exactly five heads and five tails? Not really, because it is easy to imagine a fair coin landing heads up on only 3 or 4 of the 10 tosses.

Consider the chance experiment of tossing a coin just once. Define the simple events

H = event that the coin lands with its heads side facing up
T = event that the coin lands with its tails side facing up

Now suppose that we take a fair coin and begin to toss it over and over. After each toss, we compute the relative frequency of heads observed so far. This calculation gives the value of the ratio

$$\frac{\text{number of times event } H \text{ occurs}}{\text{number of tosses}}$$

The results of the first 10 tosses might be as follows:

Toss	1	2	3	4	5	6	7	8	9	10
Cumulative number of H's	0	1	2	3	3	3	4	5	5	5
Relative frequency of H's	0	.5	.667	.75	.6	.5	.571	.625	.556	.5

Figure 6.5 illustrates how the relative frequency of heads fluctuates during a sample sequence of 50 tosses. *As the number of tosses increases, the relative frequency of heads does not continue to fluctuate wildly but instead stabilizes and approaches some fixed number (the limiting value)*. This stabilization is illustrated for a sequence of 1000 tosses in Figure 6.6.

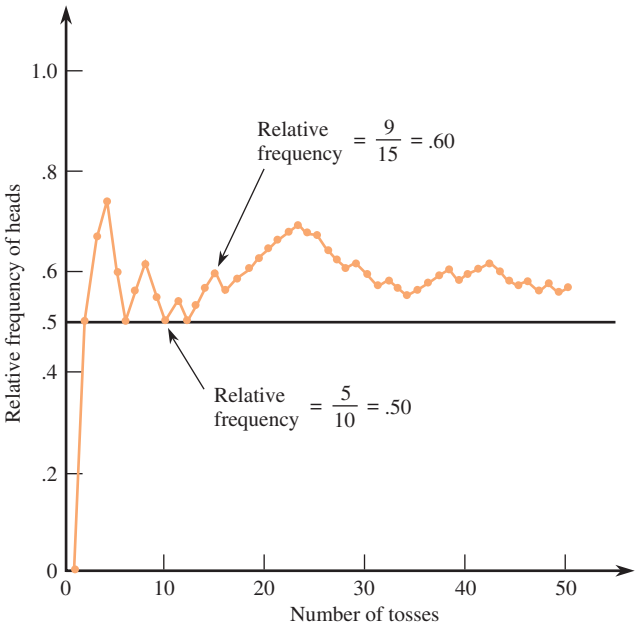


FIGURE 6.5
Relative frequency of heads in the first 50 of a long series of coin tosses.

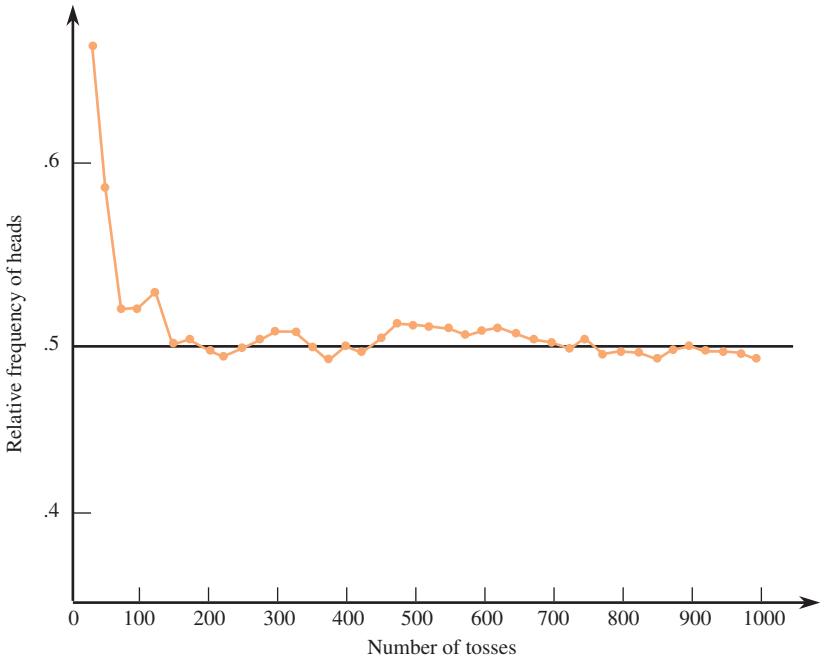


FIGURE 6.6
Stabilization of the relative frequency of heads in coin tossing.

It may seem natural to you that the proportion of H's would get closer and closer to the "real" probability of .5. That an empirically observed proportion could behave like this in real life also seemed reasonable to James Bernoulli in the early 18th century. He focused his considerable mathematical power on this topic and was able to prove mathematically what we now know as the law of large numbers.*

Law of Large Numbers

As the number of repetitions of a chance experiment increases, the chance that the relative frequency of occurrence for an event will differ from the true probability of the event by more than any small number approaches 0.

The law of large numbers tells us that, as the number of repetitions of a chance experiment increases, the proportion of the time an event E occurs gets close and stays close to the real probability of E occurring in a single chance experiment *even if the value of this probability is not known*. This means that we can observe the outcomes of repetitions of a chance experiment and then use the observed outcomes to estimate probabilities.

Relative Frequency Approach To Probability

The **probability of an event E**, denoted by $P(E)$, is defined to be the value approached by the relative frequency of occurrence of E when a chance experiment is performed many times. If the number of times the chance experiment is performed is quite large,

$$P(E) \approx \frac{\text{number of times } E \text{ occurs}}{\text{number of times the experiment is performed}}$$

*Technically, this should be referred to more specifically as the weak law of large numbers for Bernoulli trials. After Bernoulli's proof, mathematical statisticians proved more general laws of large numbers.

The relative frequency definition of probability depends on being able to repeat a chance experiment under identical conditions. Suppose that we perform a chance experiment that consists of flipping a cap from a 20-ounce bottle of soda and noting whether the cap lands with the open side up or down. The crucial difference from the previous coin-tossing chance experiment is that there is no particular reason to believe the cap is equally likely to land top up or top down.

If we assume that the chance experiment can be repeated under similar conditions (which seems reasonable), then we can flip the cap many times and compute the relative frequency of the event *top up* (the proportion of the time the event has occurred so far):

$$\frac{\text{number of times the event } \textit{top up} \text{ occurs}}{\text{number of flips}}$$

The results of the first 10 flips, with U indicating top up and D indicating top down, might be:

Flip	1	2	3	4	5	6	7	8	9	10
Outcome	U	U	D	U	D	D	U	D	U	U
Cumulative number of *ups*	1	2	2	3	3	3	4	4	5	6
Relative frequency of *ups*	1.0	1.0	.67	.75	.6	.5	.57	.5	.56	.6

Figure 6.7 illustrates how the relative frequency of the event *top up* fluctuates during a sequence of 100 flips. Based on these results and faith in the law of large numbers, it is reasonable to think that the probability of the cap landing top up is about .7.

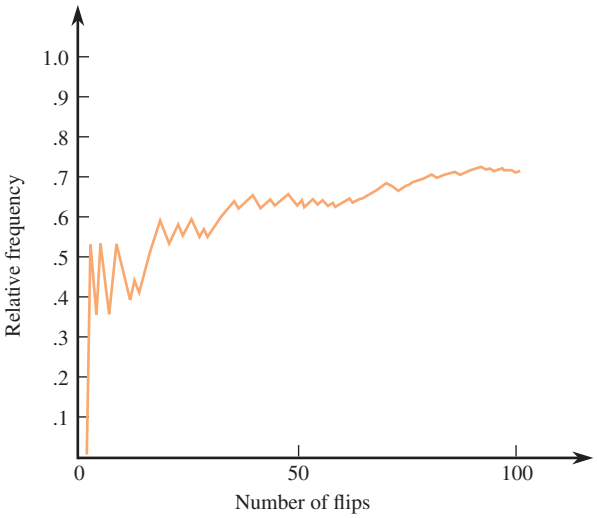


FIGURE 6.7
Stabilization of the relative frequency of a bottle cap landing top up.

The relative frequency approach to probability is based on observation. From repeated observation, we can obtain stable relative frequencies that will, in the long run, provide good estimates of the probabilities of different events. The relative frequency interpretation of probability is intuitive, widely used, and most relevant for the inferential procedures introduced in later chapters.

6.3 Basic Properties of Probability

The view of a probability as a long-run relative frequency that can be approximated empirically has some limitations when it comes to calculating probabilities in real life. The most obvious problem is time. If computing probabilities even for simple chance experiments were to require hundreds or thousands of observations, even the most ardent statistics students would avoid this task! An alternative approach that simplifies things in some situations is to study fundamental properties of probability that can be used to find probabilities of complex events. These fundamental properties are given in the accompanying box. A discussion of each property follows.

Fundamental Properties of Probability

1. For any event E, $0 \leq P(E) \leq 1$.
2. If S is the sample space for an experiment, $P(S) = 1$.
3. If two events E and F are mutually exclusive, then $P(E \text{ or } F) = P(E) + P(F)$.
4. For any event E, $P(E) + P(\text{not } E) = 1$. Therefore

$$P(\text{not } E) = 1 - P(E)$$

and

$$P(E) = 1 - P(\text{not } E).$$

Property 1: For any event E, $0 \leq P(E) \leq 1$

To understand the first property of probability, recall the previous bottle cap chance experiment. You may remember that we were keeping track of the number of flips landing with the top of the bottle cap facing up. Suppose that, after flipping the bottle cap N times, we have observed x occurrences of top up. What are the possible values of x?

The fewest that could have been counted is 0, and the most that could have been counted is N. Therefore, the relative frequency of top up falls between two numbers:

$$\frac{0}{N} \leq \text{relative frequency} \leq \frac{N}{N}$$

This means that the relative frequency must always be greater than or equal to 0 and less than or equal to 1. As N increases, the long-run value of the relative frequency, which defines the probability, must also lie between 0 and 1.

Property 2: If S is the sample space for a chance experiment, $P(S) = 1$

The probability of any event is the proportion of time an outcome in the event will occur in the long run. Because the sample space consists of all possible outcomes for a chance experiment, in the long run an outcome that is in S must occur 100% of the time. This mean that $P(S) = 1$.

Property 3: If two events E and F are mutually exclusive, then $P(E \text{ or } F) = P(E) + P(F)$

This is one of the most important properties of probability because it provides a method for computing probabilities if the number of possible outcomes (simple events) in the sample space is finite. Any nonsimple event is just a collection of outcomes from the sample space—some outcomes are in the event and others are not. Because simple events are mutually exclusive, any event can be viewed as a union of mutually exclusive events.

For example, suppose that a chance experiment has a sample space that consists of six outcomes, O_1, O_2, … , O_6. Then $P(O_1)$ can be interpreted as the long run proportion of times that the outcome O_1 will occur, and the probabilities of the other outcomes can be interpreted in a similar way. Suppose that an event E is made up of outcomes O_1, O_2, and O_4. Then E will occur whenever O_1, O_2, or O_4 occurs. Because O_1, O_2, and O_4 are mutually exclusive, the long-run proportion of time that E will occur is just the sum of the proportion of time that each of the outcomes O_1, O_2, and O_4 will occur.

Because it is always possible to express any event as a collection of mutually exclusive simple events in this way, finding the probability of an event can be reduced to finding the probabilities of the simple events (outcomes) that make up the event and then adding. If the probabilities of all the simple events are known, it is then easy to compute the probability of more complex events.

Property 4: For any event E, $P(E) + P(\textit{not } E) = 1$. Therefore $P(\textit{not } E) = 1 - P(E)$ and $P(E) = 1 - P(\textit{not } E)$

Property 4 follows from Properties 2 and 3. Property 2 tells us that $P(E)$ is the sum of the probabilities in E and that $P(\textit{not } E)$ is the sum of the probabilities for simple events corresponding to outcomes in *not* E. Every outcome in the sample space is in either E or *not* E. We know from Property 3 that the sum of the probabilities of all the simple events is 1. It follows that $P(E) + P(\textit{not } E)$ must equal 1.

The implication of Property 4, namely, that $P(E) = 1 - P(\textit{not } E)$ is surprisingly useful. There are many situations in which calculation of $P(\textit{not } E)$ is much easier than calculating $P(E)$ directly.

EXAMPLE 6.9 Medical Errors

Understand the context ⟩

Diagnostic errors in medical settings are of concern because they often lead to serious consequences and sometimes even death. The paper **"Diagnostic Error in Internal Medicine" (*Archives of Internal Medicine* [2005]: 1493–1499)** describes possible outcomes when a doctor specializing in internal medicine reaches a diagnosis for a patient. The possible diagnostic outcomes are

- O_1 Correct diagnosis
- O_2 No-fault diagnostic error (a misdiagnosis due to unusual symptoms or due to an uncooperative or deceptive patient)

O_3 System-related diagnostic error (a misdiagnosis due to equipment failure, faulty lab results, etc.)
O_4 Cognitive diagnostic error (a misdiagnosis due to faulty or inadequate doctor knowledge or failure to synthesize available information correctly)
O_5 Misdiagnosis resulting from a combination of system-related error and cognitive error

Based on a study of a large number of patients, the paper suggests that

85% of patients are correctly diagnosed
1% are misdiagnosed because of a no-fault error
3% are misdiagnosed because of a system-related error
4% are misdiagnosed because of a cognitive error
7% are misdiagnosed because of a combination of system and cognitive errors.

The following table displays the probabilities of the simple events for the chance experiment in which the diagnostic outcome is observed for a randomly selected patient:

Simple Event (Outcome)	O_1	O_2	O_3	O_4	O_5
Probability	.85	.01	.03	.04	.07

Let's create event E, the event that *a randomly selected patient is misdiagnosed.* This event consists of the outcomes O_2, O_3, O_4, and O_5. It follows that

$$\begin{aligned} P(E) &= P(O_2 \cup O_3 \cup O_4 \cup O_5) \\ &= P(O_2) + P(O_3) + P(O_4) + P(O_5) \\ &= .01 + .03 + .04 + .07 \\ &= .15 \end{aligned}$$

That is, in the long run, 15% of all patients seen by internal medicine specialists are misdiagnosed. Another way to compute this probability is to notice that the event *not E* consists only of the outcome O_1. Another way to calculate $P(E)$ is

$$P(E) = 1 - P(\textit{not } E) = 1 - P(O_1) = 1 - .85 = .15$$

If we were interested in the probability of a misdiagnosis that involved a cognitive error in some way, we could define the event F, where F is the event that *a randomly selected patient is misdiagnosed in a way that involved a cognitive error.* Then

$$F = O_4 \cup O_5$$

and

$$P(F) = P(O_4 \cup O_5) = P(O_4) + P(O_5) = .04 + .07 = .11$$

This means that, in the long run, cognitive errors play a role in the misdiagnosis of about 11% of patients. ■

Having established the fundamental properties of probability, we now present some practical probability rules that can be used to evaluate probabilities in some situations. An important aspect of each rule is the set of conditions necessary in order to use the rule. A common error for beginning statistics students is to believe that the rules can be used in *any* probability calculation. This is *not* the case. You must be careful to use the rules only after verifying that any necessary conditions are met.

Equally Likely Outcomes

The first probability rule that we consider applies only when events are equally likely. It is *not* always true for all events in all situations. Chance experiments involving tossing fair coins, rolling fair dice, or selecting cards from a well-mixed deck are examples of chance

experiments that have equally likely outcomes. If a fair die is rolled once, each possible outcome (1, 2, 3, 4, 5, and 6) has probability 1/6. With E denoting the event that the outcome is an even number,

$$P(E) = P(2 \text{ or } 4 \text{ or } 6) = P(2) + P(4) + P(6) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6}$$

This is just the ratio of the number of outcomes in E to the total number of possible outcomes. The following box presents the generalization of this result.

Calculating Probabilities When Outcomes Are Equally Likely

Consider an experiment that can result in any one of N possible outcomes. Denote the corresponding simple events by $O_1, O_2, \ldots, O_N$. If these simple events are equally likely to occur, then

1. $P(O_1) = \frac{1}{N}, P(O_2) = \frac{1}{N}, \ldots, P(O_N) = \frac{1}{N}$
2. For *any* event E,

$$P(E) = \frac{\text{number of outcomes in } E}{N}$$

Addition Rule for Mutually Exclusive Events

We have seen previously that the probability of an event can be calculated by adding together probabilities of the simple events that correspond to the outcomes making up the event. This addition process is also legitimate when calculating the probability of the union of two events that are mutually exclusive.

The Addition Rule for Mutually Exclusive Events

Let E and F be two mutually exclusive events. One of the basic properties (axioms) of probability is

$$P(E \text{ or } F) = P(E \cup F) = P(E) + P(F)$$

This property of probability is known as the addition rule for mutually exclusive events. More generally, if events $E_1, E_2, \ldots, E_k$ are all mutually exclusive, then

$$P(E_1 \text{ or } E_2 \text{ or } \ldots \text{ or } E_k) = P(E_1 \cup E_2 \cup \cdots \cup E_k) = P(E_1) + P(E_2) + \cdots + P(E_k)$$

In words, the probability that any of these k mutually exclusive events occurs is the sum of the probabilities of the individual events.

6.4 Conditional Probability

Sometimes the knowledge that one event has occurred changes our assessment of the likelihood that another event occurs. For example, consider a population in which 0.1% of all individuals have a certain disease. The presence of the disease cannot be discerned from outward appearances, but there is a diagnostic test available. Unfortunately, the test is not always correct. Of those with positive test results, 80% actually have the disease; and the other 20% who show positive test results are false-positives.

To put this in probability terms, consider the chance experiment in which an individual is randomly selected from the population. Define the following events:

E = event that the individual has the disease
F = event that the individual's diagnostic test is positive

We will use $P(E|F)$ to denote the probability of the event E *given that* the event F is known to have occurred. A new symbol has been used to indicate that a probability calculation has been made conditional on the occurrence of another event. The standard symbol for this is a vertical line, and it is read "given." For example, we would say, "the probability that an individual has the disease *given* that the diagnostic test is positive," and represent this symbolically as $P(\text{has disease}|\text{positive test})$ or $P(E|F)$. This probability is called a **conditional probability**.

The information provided then implies that

$$P(E) = .001$$
$$P(E|F) = .8$$

This means that before we have diagnostic test information, the occurrence of E is unlikely. However, once it is known that the test result is positive, the likelihood of the disease increases dramatically. (If this were not so, the diagnostic test would not be very useful!)

EXAMPLE 6.11 College Housing Options

Understand the context ⟩

At a small liberal arts college, students have three housing options. They can live in on-campus housing, off-campus housing, or at home with family. The following table gives the number of students in each housing option by year in school.

Consider the data ⟩

	Freshman	Sophomore	Junior	Senior	Total
On-campus housing	150	160	140	150	**600**
Off-campus housing	100	90	120	125	**435**
Home with family	125	140	150	150	**565**
Total	**375**	**390**	**410**	**425**	**1600**

Consider each of the following statements, and make sure that you see how each follows from the information in the table:

1. There are 160 sophomores who live in on-campus housing.
2. The number of seniors who live in off-campus housing is 125.
3. There are 435 students who live in off-campus housing.
4. There are 410 juniors at the college.
5. The total number of students at the college is 1600.

Each week, the college president selects a students at random and invites him or her to have lunch with her to discuss various issues that might be of concern to them. She feels that random selection will give her the greatest chance of hearing from a diverse group of students. What is the probability that a randomly selected student is a senior who lives on campus? Assuming that each student is equally likely to be selected, we can calculate this probability as follows:

Do the work ⟩

$$P(\text{senior who lives on campus}) = \frac{\text{number of seniors who live on campus}}{\text{total number of students}} = \frac{150}{1600} = .09375$$

Now suppose that the president's assistant records not only the student's name but also the student's year in school. The assistant has indicated that the selected student is a senior. Does this information change our assessment of the likelihood that the selected student lives on campus? Because 150 of the 425 seniors live on campus, this suggests that

$$P(\text{live on campus}|\text{senior}) = \frac{\text{number of seniors who live on campus}}{\text{total number of seniors}} = \frac{150}{425} = .3529$$

Interpret the results ⟩

The probability is calculated in this way because we know that the selected student is one of 425 seniors, each of whom is equally likely to have been the one selected. The interpretation of this conditional probability is that if we were to repeat the chance experiment of selecting a student at random, about 35.29% of the selections that resulted in a senior being selected would also result in the selection of someone who lives on campus. ■

EXAMPLE 6.12 GFI Switches

Understand the context ⟩

A GFI (ground fault interrupt) switch turns off power to a system in the event of an electrical malfunction. A spa manufacturer currently has 25 spas in stock, each equipped with a single GFI switch. Two different companies supply the switches, and some of the switches are defective, as summarized in the following table:

Consider the data ⟩

	Nondefective	Defective	Total
Company 1	10	5	**15**
Company 2	8	2	**10**
Total	**18**	**7**	

A spa is randomly selected for testing. Let

E = event that GFI switch in selected spa is from Company 1
F = event that GFI switch in selected spa is defective

Do the work ⟩ Using the information in the table, we can calculate the following probabilities:

$$P(E) = \frac{15}{25} = .60 \quad P(F) = \frac{7}{25} = .28 \quad P(E \text{ and } F) = P(E \cap F) = \frac{5}{25} = .20$$

Now suppose that testing reveals a defective switch. (This means that the chosen spa is one of the seven in the "defective" column.) How likely is it that the switch came from the first company? Because five of the seven defective switches are from Company 1,

$$P(E|F) = P(\text{company 1}|\text{defective}) = \frac{5}{7} = .714$$

This is larger than the unconditional probability $P(E)$. This is because Company 1 has a much higher defective rate than Company 2.

An alternative expression for the conditional probability is

$$P(E|F) = \frac{5}{7} = \frac{5/25}{7/25} = \frac{P(E \text{ and } F)}{P(F)} = \frac{P(E \cap F)}{P(F)}$$

Notice that $P(E|F)$ is a ratio of two previously specified probabilities. It is the probability that both events occur divided by the probability of the "conditioning event" F. Additional insight comes from the Venn diagram of Figure 6.8. Once it is known that the outcome lies in F, the chance that E also occurs is the "size" of (E *and* F) relative to the size of F.

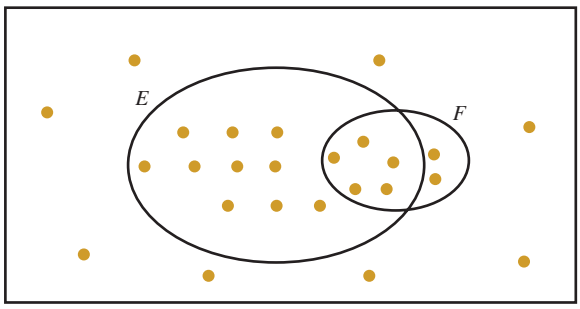


FIGURE 6.8
Venn diagram for Example 6.12 (each dot represents one GFI switch).

The results of the previous example lead us to a general definition of conditional probability.

DEFINITION

Conditional probability: Suppose that E and F are *two events with* $P(F) > 0$. **The conditional probability of the event *E* given that the event *F* has occurred,** denoted by $\boldsymbol{P(E|F)}$, is

$$P(E|F) = \frac{P(E \cap F)}{P(F)}$$

Notice the requirement that $P(F) > 0$. In addition to the standard warning about division by 0, there is another reason for requiring $P(F)$ to be positive. If the probability of F were 0, the event F would never occur. Therefore, it would not make sense to calculate the probability of another event conditional on F having occurred.

EXAMPLE 6.13 Surviving a Heart Attack

Understand the context ⟩

Medical guidelines recommend that a hospitalized patient who suffers cardiac arrest should receive defibrillation (an electric shock to the heart) within 2 minutes. The paper **"Delayed Time to Defibrillation After In-Hospital Cardiac Arrest" (*The New England Journal of Medicine* [2008]: 9–17)** describes a study of the time to defibrillation for hospitalized patients in hospitals of various sizes.

The authors examined medical records of 6716 patients who suffered cardiac arrest while hospitalized, recording the size of the hospital and whether or not defibrillation occurred in 2 minutes or less. Data from this study are summarized in the accompanying table.

Consider the data ⟩

	Time to Defibrillation		
Hospital Size	2 minutes or less	More than 2 minutes	Total
Small (Less than 250 beds)	1124	576	**1700**
Medium (250–499 beds)	2178	886	**3064**
Large (500 or more beds)	1387	565	**1952**
Total	**4689**	**2027**	**6716**

We will assume that these data are representative of the larger group of all hospitalized patients who suffer a cardiac arrest. Suppose that a hospitalized patient who suffered a cardiac arrest is selected at random. The following events are of interest:

S = event that the selected patient is at a small hospital
M = event that the selected patient is at a medium-sized hospital
L = event that the selected patient is at a large hospital
D = event that the selected patient receives defibrillation in 2 minutes or less

Do the work ⟩

We can use the information in the table to compute

$$P(D) = \frac{4689}{6716} = .698$$

This probability is interpreted as the proportion of hospitalized patients who suffer cardiac arrest that receive defibrillation in 2 minutes or less. That is, 69.8% of these patients receive timely defibrillation.

Now suppose it is known that the selected patient was at a small hospital. How likely is it that this patient received defibrillation in 2 minutes or less? To answer this question, we need to compute $P(D|S)$, the probability of defibrillation in 2 minutes or less *given* that the patient is at a small hospital. Using the formula for conditional probability, we know

$$P(D|S) = \frac{P(D \cap S)}{P(S)}$$

From the information in the table, we can compute

$$P(D \cap S) = \frac{1124}{6716} = .167$$

$$P(S) = \frac{1700}{6716} = .253$$

and so

$$P(D|S) = \frac{P(D \cap S)}{P(S)} = \frac{.167}{.253} = .660$$

Notice that this is smaller than the unconditional probability, $P(D) = .698$. This tells us that there is a smaller probability of timely defibrillation at a small hospital.

Two other conditional probabilities of interest are

$$P(D|M) = \frac{P(D \cap M)}{P(M)} = \frac{2178/6716}{3064/6716} = .711$$

and

$$P(D|L) = \frac{P(D \cap L)}{P(L)} = \frac{1387/6716}{1952/6716} = .711$$

From this, we see that the probability of timely defibrillation is the same for patients at medium-sized and large hospitals, and that this probability is higher than that for patients at small hospitals.

It is also possible to compute $P(L|D)$, the probability that a patient is at a large hospital given that the patient received timely defibrillation:

$$P(L|D) = \frac{P(L \cap D)}{P(D)} = \frac{1387/6716}{4689/6716} = .296$$

Interpret the results ⟩ Let's look carefully at the interpretation of some of these probabilities:

1. $P(D) = .698$ is interpreted as the proportion *of all hospitalized patients* suffering cardiac arrest who would receive timely defibrillation. Approximately 69.8% of these patients would receive timely defibrillation.
2. $P(D \cap L) = \frac{1387}{6716} = .207$ gives the proportion *of all hospitalized patients* who suffer cardiac arrest who are at a large hospital *and* who would receive timely defibrillation.
3. $P(D|L) = .711$ is the proportion *of patients at large hospitals* suffering cardiac arrest who would receive timely defibrillation.
4. $P(L|D) = .296$ is the proportion *of patients who receive timely defibrillation* who were at large hospitals.

Notice the difference between the unconditional probabilities in Interpretations 1 and 2 and the conditional probabilities in Interpretations 3 and 4. The reference point for the unconditional probabilities is the entire group of interest (all hospitalized patients suffering cardiac arrest), whereas the conditional probabilities are interpreted in a more restricted context defined by the "given" event. ■

Example 6.14 demonstrates the calculation of conditional probabilities and also makes the point that we must be careful when translating real-world problems—especially probability problems—into mathematical form. Not only are probability problems sometimes difficult to formulate precisely, but the answers are sometimes surprising.

EXAMPLE 6.14 Two-Kid Families

Understand the context ⟩ Consider the population of all families with two children. Representing the gender of each child using G for girl and B for boy results in four possibilities: BB, BG, GB, GG. The gender information is sequential, with the first letter indicating the gender of the older sibling. Thus, a family having a girl first and then a boy is denoted GB. If we assume that a child is equally likely to be male or female, each of the four possibilities in the sample space for the chance experiment that selects at random from families with two children is equally likely. Consider the following two questions:

1. What is the probability that the selected family has two girls, given that the family has at least one girl?